OmniCode: Chapter 13: Track 2 - Lexicon: State, Container, Site & Text

Quad-State T4 Clock, Memory Envelopes & Bounded Sites

CHAPTER IDENTIFIER	**NOTEBOOKLM WORD COUNT**
CH 13	42,251 words
CATALOGED CHAPTER FILES	**GENERATION DATE**
56 files (8,489 lines)	2026-10-16 20:36:05

EPISTEMIC COVENANT BEDROCK & IMMUTABLE LANDMARKS

"And he gave unto Moses... two tables of testimony, tables of stone, written with the finger of God." — Exodus 31:18

"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house." — Proverbs 24:27

"Remove not the ancient landmark, which thy fathers have set." — Proverbs 22:28

GOOGLE NOTEBOOKLM GROUNDING FOCUS & LEARNING PROMPT

- **Chapter Focus:** *"How do IT, BOUND, and TURN govern canonical loop cadence?"*
- **Architectural Rule:** All definitions conform to 4-block structure & spatial ruler.

CHAPTER MANIFEST & INDEX (56 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/README.md**	MD	159	1,053	8.0
2	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/container.omni**	OMNI	192	1,812	12.3
3	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/container.tape**	TAPE	89	772	5.4
4	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/site.omni**	OMNI	190	1,818	12.2
5	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/site.tape**	TAPE	89	763	5.3
6	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/state.omni**	OMNI	230	2,189	14.4
7	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/state.tape**	TAPE	117	949	6.7
8	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/text.omni**	OMNI	62	249	2.2
9	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/text.tape**	TAPE	33	210	1.6
10	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/compile_flags.txt**	TXT	5	4	0.0
11	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/number.o**	O	1	5	9.0
12	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/character.h**	H	300	1,795	13.1
13	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/container.h**	H	222	1,078	9.0
14	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/number.h**	H	385	1,953	15.1
15	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/omni.h**	H	27	136	1.1
16	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/omni_base.h**	H	289	1,270	10.6
17	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/site.h**	H	191	925	7.4
18	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/state.h**	H	170	863	7.1
19	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/character.c**	C	278	1,241	10.5
20	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/container.c**	C	236	997	8.5
21	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/number.c**	C	436	1,735	15.0
22	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/site.c**	C	168	735	6.2
23	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/state.c**	C	174	766	6.3
24	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/character.lua**	LUA	149	579	5.9
25	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/container.lua**	LUA	158	632	6.5

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
26	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/number.lua**	LUA	342	1,731	14.8
27	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/site.lua**	LUA	148	565	6.0
28	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/state.lua**	LUA	183	697	7.1
29	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/tape_engine.lua**	LUA	223	957	8.6
30	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/integrated/test_omni_lib.lua**	LUA	152	634	7.0
31	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_character.lua**	LUA	89	437	4.1
32	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_common.lua**	LUA	49	188	1.7
33	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_container.lua**	LUA	79	397	3.7
34	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_number.lua**	LUA	197	851	8.4
35	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_site.lua**	LUA	82	379	3.8
36	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_state.lua**	LUA	93	439	4.2
37	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/compile_flags.txt**	TXT	6	5	0.1
38	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/test_number.o**	O	1	5	21.8
39	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/integrated/test_omni_lib.c**	C	121	492	5.5
40	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/includes/test_common.h**	H	65	248	2.3
41	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_character.c**	C	121	599	5.8
42	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_container.c**	C	145	670	6.5
43	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_number.c**	C	244	1,146	11.1
44	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_site.c**	C	120	556	5.5
45	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_state.c**	C	119	606	5.6
46	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/README.md**	MD	149	953	6.4
47	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/interpreted/lua/tape_engine.lua**	LUA	368	1,404	12.5
48	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/interpreted/lua/tests/test_common.lua**	LUA	51	185	1.7

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
49	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/interpreted/lua/tests/test_tape_engine.lua**	LUA	248	798	9.3
50	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/compiled/c/Makefile**	TEXT	17	33	0.3
51	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/compiled/c/compile_flags.txt**	TXT	6	5	0.0
52	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/compiled/c/test_universal_bin**	TEXT	1	5	25.0
53	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/compiled/c/tests/test_common.h**	H	59	213	1.7
54	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/compiled/c/tests/test_universal.c**	C	213	781	9.8
55	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/compiled/c/include/omni.h**	H	45	176	1.4
56	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/compiled/c/include/omni_base.h**	H	403	1,567	14.5

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/README.md | MD | 159 LINES | 1053 WORDS | 8.0 KB

```
# OmniCode Invariant Types: Architectural Taxonomy & Guide

Welcome to the **OmniCode Types System**! Whether you are a systems programmer, language enthusiast, philosopher, or someone completely new to computing, this guide explains how OmniCode models the building blocks of thought, language, and reality.

---

## 1. What is an "Idea { Type }" in OmniCode?

In conventional programming languages, a "type" is often just a technical label telling a compiler how many bytes of memory to reserve (like `int`, `char`, or `float`).

In **OmniCode**, a Type is fundamentally an **Idea**:

- It represents an **ontological invariant**—a universal concept that holds true regardless of what machine, operating system, or silicon chip executes it.
- It unites three worlds:
  1. **Pure Meaning (Semantics):** What the concept means in human thought, natural language, and philosophy.
  2. **Mathematical Balance (Virtual Substrate):** Symmetrical balanced ternary logic (`-1`, `0`, `+1`), where truth is dynamic equilibrium.
  3. **Physical Silicon (Hardware Target):** Concrete execution on modern 64-bit microprocessors, memory registers, and video terminals.

---

## 2. Core Concepts: Learning the OmniCode Way

### Words vs. Values, States vs. Numbers

One of the most profound breakthroughs in OmniCode is understanding that values, words, and states share the same symbols, but differ in human intent:

- **`0` as a State:** Represents **`AWAIT`**—a state of perfect equilibrium, resting quietly, waiting for the next event.
- **`"0"` as a Value:** Using literal "air-quotes" (or enclosing brackets like `{ 0 }` or `[ 0 ]`) tells the machine: *"Treat this zero not as a pause state, but as the literal mathematical quantity zero."*
- **`'AWAIT'` as a Word:** Single quotes clarify that a reserved keyword is being spoken of as a literal word rather than being triggered as a command.

Because OmniCode is a **freestanding language** designed to communicate with hardware without hidden assumptions, it reads all symbols literally. You provide the semantics that express your true intent.

### The Analogy of the Body: "Fleshing Out the Bones"

OmniCode grows organically, mirroring how a living organism develops:

1. **The Skeleton:** We first build the rigid architectural bones—the invariant laws, mathematical boundaries, and fundamental categories.
2. **The Tissue:** We link the bones together using the **ladder.baton** system, connecting universal roots to specialized types.
3. **The Flesh:** We add dynamic features, transpilers, runtime engines, and tests.
4. **Shaping & Refining:** We continually test, reshape, and harmonize all parts so the entire system breathes together across all tracks.

---

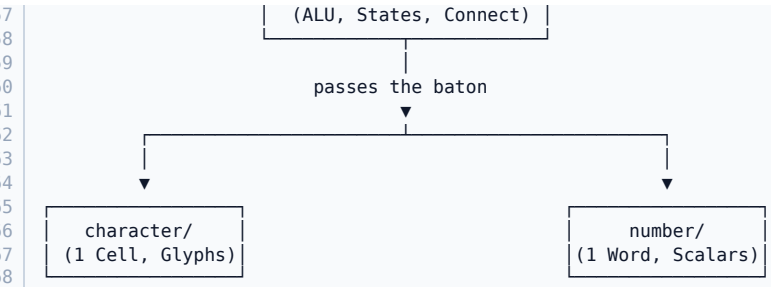
## 3. The `ladder.baton` Inheritance System

In traditional software projects, shared code is frequently copy-pasted across folders, leading to loose, duplicate files that easily get out of sync.

In OmniCode, we use the **`ladder.baton` system**:

- The **Universal Root** (`universal/`) holds the core axioms: balanced ternary math, the four causal states, connective conjunctions, and safety interlocks.
- Downstream types (like `character/` and `number/`) **inherit** these universal invariants down the ladder like a baton passed in a relay race.
- **Zero Duplicate Files:** Individual type folders contain only what is unique to their type. Universal invariants are passed down via clean module paths and compiler include flags.

```text
                    ┌──────────────────────┐
                    │       universal/     │  <-- Ground Truth / Axioms
```
```



---

4. The Three-Pillar Modular Folder Structure

Every type folder in OmniCode is organized into three complementary expressions:

```bash
<type>/
├── compiled/           # 1. High-Level Source Reel (.omni)
├── interpreted/        # 2. Runtime Execution Ribbon (.tape)
└── transpiled/         # 3. Concrete Physical Bridges
    ├── compiled/c/     #    - Freestanding Bare-Metal C Backbone
    └── interpreted/lua/#    - Dynamic Coroutine Ribbon Companion
```

Pillar 1: `compiled/` (`<type>.omni`)

The high-level declaration of the type. Written in canonical OmniCode notation, it reads like structured, elegant prose. It defines the type's metadata, biblical/philosophical witness, capacity footprint, and four-block lifecycle laws.

Pillar 2: `interpreted/` (`<type>.tape`)

The dynamic interpreted process ribbon. Think of this like a physical strip of celluloid movie film or a Turing tape. It breaks execution down into sequential **rolls** (Inscribe, Transduce, Compare, Equilibrium, Sequence) that step forward smoothly or roll back safely if an error occurs.

Pillar 3: `transpiled/` (Physical Language Bridges)

To allow OmniCode to run on any computer in the world today, OmniCode transpiles automatically into physical targets:

- **`compiled/c/`**: A pure, freestanding C99/C11 implementation. Contains `include/`, `src/`, `tests/`, and a `Makefile`. It requires zero third-party libraries and can run directly on bare metal (even booting on an x86 CPU without an operating system!).
- **`interpreted/lua/`**: A lightweight, embeddable Lua implementation. Uses Lua coroutines to step through tape rolls as cooperative fibers, with automatic state snapshotting and rollback.

---

5. Overview of Current Types

Type Directory	Capacity & Footprint	Primary Role	Philosophical Witness
`universal/`	Foundation Substrate	Balanced Ternary ALU, Quad-State Truth, Connectives	Proverbs 11:1 & 24:27
`character/`	1 Cell (5 Trits / 8 Bits)	Atomic glyphs, letters, terminal inscription runes	Exodus 31:18
`number/`	1 Word (32 Trits / 64 Bits)	Scalar quantities, balanced arithmetic, zero-sum ALU	Psalm 147:4
`container/`	Variable Bounded Space	Envelopes, memory partitions, bounded buffers	*(In transition)*
`site/`	Memory Coordinates	Memory-mapped I/O, physical RAM locations	*(In transition)*
`state/`	Causal Status Register	Fourfold causal detents, program loop steppers	*(In transition)*

---

6. How to Test & Explore

OmniCode enforces a strict **"Test First, Integrate Second"** methodology. You can run unit tests at any level:

Universal Bedrock Tests

```bash
# Test universal C axioms & ALU
cd universal/transpiled/compiled/c && make test

# Test universal Lua tape engine
lua universal/transpiled/interpreted/lua/tests/test_tape_engine.lua
```

````
### Character Type Tests

```bash
# Test character C backbone
cd character/transpiled/compiled/c && make test

# Test character Lua tape ribbon
lua character/transpiled/interpreted/lua/tests/test_character.lua
```

### Number Type Tests

```bash
# Test number C backbone
cd number/transpiled/compiled/c && make test

# Test number Lua tape ribbon
lua number/transpiled/interpreted/lua/tests/test_number.lua
```

### Full Host Engine (Rust) Verification

From the repository root:

```bash
cargo test --manifest-path "<TRACK 1> : {Host Engine & Virtual Machine}/Cargo.toml"
```

*(Confirms that all 252 host VM tests pass across every component).*
````

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/container.omni

OMNI | 192 LINES | 1812 WORDS | 12.3 KB

```
# ========================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Type: < Idea { Type } : Container > )
# ========================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines bounded memory envelopes, arrays, structures, and storage vessels.
#   - Base Block: Primitive Invariant Type
#   - Category: Idea { Type }
#   - Footprint: [ 64 Cells ] { 64 Bytes / Dynamically Bounded Memory Space }
#   - Ladder Ascension: Build-up with Numbers (/-^)
# ========================================================================

= [Module]: < Lexicon { Types of Ideas } > & [ Noun { Container } ] ( This is the Module for the Lexicon - Ideas (Type): Containers - Nouns ) {
    [ Version ]:   { "1.0.0" } ( The Version is in the context of 1.0.0 ),
    [ Created ]:   { "2026-10-11" } ( Created in the context of October 11, 2026 ),
    [ Updated ]:   { "2026-10-12" } ( Last updated in the context of October 12, 2026 ),
    [ Author ]:    { "CreativeWorkzStudio LLC / ProfessorSeanEX" } ( Author in the context of CreativeWorkzStudio LLC and ProfessorSeanEX ),
    [ System ]:    { OmniCode { Lexicon } } ( System is in the context of the OmniCode Lexicon ),
    [ Witness ]:   < Scripture { 2 Kings 4:4 KJV } > { "And when thou art come in, thou shalt shut the door upon thee and upon thy sons, and shalt pour out into all those vessels, and thou shalt set aside that which is full." } ( Invariant Scripture Witness 2 Kings 4:4 in the context: Pour out into all those vessels, and set aside that which is full ),
    [ Nature ]:    < InvariantType > & { BaseBlock } ( Nature is an Invariant Type and is in the context of Base Block ),
    [ Semantics ]: { BoundedEnvelope :: StorageVessel :: MemoryCapsule } ( Semantics in the context of Bounded Envelope, Storage Vessel, and Memory Capsule )
}

# ------------------------------------------------------------------------
# LADDER ASCENSION: BUILD-UP WITH NUMBERS
# A Container cannot exist without numbers: capacity, indices, bounds, and cells
# are constructed out of scalar quantities passed up the ontological ladder.
# ------------------------------------------------------------------------
/-^ < Numbers > FROM [ < Lexicon { Types of Ideas } > & [ Noun { Number } ] ] ( Container builds up its boundaries, capacity, and cells using Numbers from the Lexicon )

# Freestanding Invariant Type Definition
< Container > = < Idea { Type } > & [ Noun { Bounded Space } ] ( Container is the Invariant Idea of Type, and is the Noun of Bounded Space ) {
    # 1. The Ontological Analogy (Container's proportion to the universe)
    [ Relation ]: [ Container ] : [ Vessel ] :: [ Boundary ] : [ Capacity ] ( Container is to Vessel as Boundary is to Capacity ),

    # 2. The Universal Domain (All vessels, storehouses, partitions, and memory capsules)
    [ Domain ]: { Vessel :: Storehouse :: Partition :: Capsule } ( Domain encompasses vessels, storehouses, partitions, and memory capsules ),

    # 3. The Physical Substrate (Concrete bounded capacity in silicon)
    [ Substrate ]: [ 64 Cells { 64 Bytes } ] & [ Bounded Space ] ( Substrate is 64 Cells of 64 Bytes, and is Bounded Space ),

    # 4. The Universal Inscription Vector (How external payload is encapsulated into memory)
    [ Inscribe ]: [ Payload ] --> [ Container ] ( Any memory payload is inscribed into the bounded Container envelope ),

    # 5. The Hardware Enclosure Law (Memory is physically bounded by allocated capacity)
    [ Enclose ]: ENCLOSE [ Storage ] WITHIN [ Capacity ] ( Storage is strictly enclosed within the declared capacity ),

    # 6. The Core Containment Law (Occupancy must never exceed capacity limit)
    [ Law { Containment } ]: { Occupancy <= Capacity } ( The law of containment requires that occupancy never exceeds bounded capacity ),

    # 7. Canonical Exemplars (Standard grounding values for partitions, vessels, and storehouses)
    [ Exemplar { Unit } ]: [ 1 ] <=> [ Partition { 1 Cell } ] ( Canonical single unit partition ),
    [ Exemplar { Vessel } ]: [ 64 ] <=> [ Vessel { 64 Cells } ] ( Canonical vessel exemplar bounded at 64 memory cells ),
    [ Exemplar { Storehouse } ]: [ 4096 ] <=> [ Storehouse { 4096 Cells } ] ( Canonical storehouse exemplar bounded at 4096 memory cells ),

    # 8. The Relational Comparison (How containers evaluate occupancy against capacity limit)
    [ Compare ]: [ { Occupancy } <= { Capacity } ] ( Container evaluates occupancy for containment within capacity limit ),

    # 9. The Sequence Transition (How containers chain into composite data structures)
    [ Sequence ]: [ Container ] --> [ Structure ] ( Discrete containers chain together into composite memory structures
```

```
),

    # 10. The Invariant Boundary (Strict capacity enclosure)
    [ Bound ]: ENCLOSE [ Storage ] WITHIN [ Capacity ] ( Container strictly encloses its storage within capacity )
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    # 1. The Dynamic Input Payload: Accepts incoming memory payload
    { Payload ]: { Input :: [ 64 Bytes ] } ( Outside-in imposition: external memory payload to be encapsulated ),

    # 2. The Typed Entity: Container brought down from Block 1 into concrete allocation
    [ Container ]: < Container > & [ Vessel ] ( Denoted Container is typed as invariant Container allocated as a Vessel
),

    # 3. The Physical Envelope: The bounded space built up with numbers
    [ Bounded Space ]: [ 64 Cells ] & { Capacity = 64 } ( A concrete 64-cell bounded envelope built up with Numbers ),

    # 4. The Encapsulation Guard: Verifying containment limits
    [ Guard ]: { Occupancy <= Capacity } & { Door = SHUT } ( The guard enforces bounds limit and shuts the door when full
),

    # 5. The Output Target: Physical memory offset location
    [ Target ]: [ Storage Envelope ] ::@ 0x00000000 ( Target output is situated at the base storage envelope memory
offset ),

    # 6. Preconditions & State: Proving room before filling
    [ Preconditions ]: [ Capacity > 0 ] & [ Status = +1:PROCEED ] ( Capacity is positive and the machine status is armed
in Proceed )
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # --------------------------------------------------------------------------
    # ROLL 1: BUILD CAPSULE (LADDER BUILD-UP)
    # Build up concrete vessel envelope with 64 numeric cells yielding new state Vessel`
    # --------------------------------------------------------------------------
    [ ROLL 1 ] : [ Build Capsule ] : /-^ [ 64 Cells ] --> [ Envelope ] = [ Vessel` ]
        ( Build up concrete vessel envelope with 64 numeric cells yielding new state Vessel-prime ),

    # --------------------------------------------------------------------------
    # ROLL 2: ENCAPSULATION (OUTSIDE-IN IMPOSITION)
    # Encapsulate external payload imposed into Vessel yielding new state Container`
    # --------------------------------------------------------------------------
    [ ROLL 2 ] : [ Encapsulate ] : { Payload ] --> ENCAPSULATE [ Vessel` ] = [ Container` ]
        ( Encapsulate external payload imposed into Vessel yielding new state Container-prime ),

    # --------------------------------------------------------------------------
    # ROLL 3: IDENTITY & BOUND COMPARISON
    # Compare occupancy to capacity under guard: proceed when contained, rollback on overflow
    # --------------------------------------------------------------------------
    [ ROLL 3 ] : [ Guard Bounds ] :
        { Occupancy` <= Capacity } & { Container` = Vessel` }
        ! GUARD --> [ Status` = +1:PROCEED ]
        | { Occupancy` > Capacity } --> [ Status` = -1:ROLLBACK ]
        ( Compare occupancy to capacity under guard: proceed when contained, rollback on overflow ),

    # --------------------------------------------------------------------------
    # ROLL 4: POUR OUT & SET ASIDE (INSIDE-OUT EMERGENCE)
    # Inside-out emergence: full container is poured out and set aside at target destination
    # --------------------------------------------------------------------------
    [ ROLL 4 ] : [ Pour Out ] : [ Container` } --> SET_ASIDE [ Target ] = [ Sealed Vessel` ]
        ( Inside-out emergence: full container is poured out and set aside at target destination establishing new state
Sealed Vessel-prime ),

    # --------------------------------------------------------------------------
    # ROLL 5: SEQUENCE TRANSITION
    # Chain discrete container outward into composite memory structure sequence
    # --------------------------------------------------------------------------
    [ ROLL 5 ] : [ Sequence ] : [ Container` ] --> SEQUENCE [ Structure ] = [ Structure` ]
        ( Discrete container chains outward into composite memory structure establishing new state Structure-prime )
]

# ================================================================================
```

```
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing ( Sealing Gate: Reconcile, validate, tripwire, debug, log, and affirm permanent execution ) {
    # ------------------------------------------------------------------------
    # 1. VERIFICATION OF OPERATIONAL ROLLS
    # ------------------------------------------------------------------------
    [ Verify ]: [
        [ ROLL 1 ] = PROCEED ::
        [ ROLL 2 ] = PROCEED ::
        [ ROLL 3 ] = PROCEED ::
        [ ROLL 4 ] = PROCEED ::
        [ ROLL 5 ] = PROCEED
    ] ( Verify that all five operational rolls achieved affirmative consensus in PROCEED ),

    # ------------------------------------------------------------------------
    # 2. VALIDATION OF INVARIANT BOUNDS & CONTAINMENT INTEGRITY
    # ------------------------------------------------------------------------
    [ Validate ]: { Occupancy` <= Capacity } & { Door = SHUT }
        ( Validate that occupancy remains strictly within capacity and door is shut upon completion ),

    # ------------------------------------------------------------------------
    # 3. ZERO-TOLERANCE TRIPWIRING
    # ------------------------------------------------------------------------
    [ Tripwire ]: { Status` = +1:PROCEED } --> [ ARMED ] | { Status` != +1:PROCEED } --> [ TRIPPED ] --> [ Commit Status
= -1:ROLLBACK ]
        ( Tripwire remains armed in Proceed or trips on anomaly to veto execution ),

    # ------------------------------------------------------------------------
    # 4. DIAGNOSTIC DEBUG TELEMETRY
    # ------------------------------------------------------------------------
    [ Debug ]: { [ Payload ] :: [ Container` ] :: [ Occupancy` ] :: [ Target ] :: [ Status` ] }
        ( Expose diagnostic telemetry across input payload, container envelope, occupancy, and machine status ),

    # ------------------------------------------------------------------------
    # 5. EPISODIC INVENTORY LOGGING
    # ------------------------------------------------------------------------
    [ Log ]: [ Journal ]: INSCRIBE { [ Container Encapsulated ] :: [ Container` ] :: [ Target ] }
        ( Inscribe the verified container transaction into the permanent episodic system journal ),

    # ------------------------------------------------------------------------
    # 6. PERMANENT COVENANT EXECUTION & FOURFOLD COMMIT STATUS LOOP
    # ------------------------------------------------------------------------
    [ Commit Status ]: [
        # +1: PROCEED (+t: Advance forward along the causal tape)
        { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
            --> [ +1:PROCEED ] |

        #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
        { [ Target ] = BUSY } | { [ Status` ] = PENDING }
            --> [ 0:AWAIT ] :: REPEAT |

        # NULL: HALT (t=empty: Cessation of clock, absolute stop)
        { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
            --> [ HALT:NULL ] |

        # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
        { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
            --> [ -1:ROLLBACK ]
    ] ( Fourfold state machine: Proceed advances forward, Await holds equilibrium, Halt ceases execution in Null,
Rollback reverts to origin )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/container.tape

TAPE | 89 LINES | 772 WORDS | 5.4 KB

```
# ================================================================================
# PROCESS TAPE: CONTAINER INVARIANT TYPE EXECUTION RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Interprets and executes the atomic lifecycle for Invariant Type < Container >.
#   - Defined Space: [ 64 Cells ] { 64 Bytes / Bounded Memory Space }
#   - Execution Domain: Lexicon { Types of Ideas }
#   - Ladder Ascension: Build-up with Numbers (/-^)
# ================================================================================

= [Tape]: < Lexicon { Types of Ideas } > & [ Process { Container } ] ( Process Ribbon for Container Encapsulation ) {
    WHO   { OmniCode Lexicon } ::
    WHAT  { Container } --> ENCAPSULATE { Payload } ::
    WHEN  { Allocation Event } ::
    WHERE { Container Envelope } = [ 64 Cells ] ::
    WHY   ( Inscribe bounded container type invariant and storage envelopes ) ::
    HOW   [ +1 : PROCEED ] ::
    < Scripture { 2 Kings 4:4 KJV } > { "And when thou art come in, thou shalt shut the door upon thee and upon thy sons, and shalt pour out into all those vessels, and thou shalt set aside that which is full." } ( Invariant Scripture Witness 2 Kings 4:4 in the context: Pour out into all those vessels, and set aside that which is full )
}

# --------------------------------------------------------------------------------
# LADDER ASCENSION: BUILD-UP WITH NUMBERS
# --------------------------------------------------------------------------------
/-^ < Numbers > FROM [ < Lexicon { Types of Ideas } > & [ Noun { Number } ] ] ( Build up container capacity and envelopes using Numbers from the Lexicon )

# ================================================================================
# DECLARED ROLLS: ATOMIC INVARIANT EXECUTION
# ================================================================================

# --------------------------------------------------------------------------------
# ROLL 1: BUILD CAPSULE (LADDER BUILD-UP)
# Build up concrete vessel envelope with 64 numeric cells yielding new state Vessel`
# --------------------------------------------------------------------------------
[ ROLL 1 ] : [ Build Capsule ] : /-^ [ 64 Cells ] --> [ Envelope ] = [ Vessel` ]
    ( Build up concrete vessel envelope with 64 numeric cells yielding new state Vessel-prime )

# --------------------------------------------------------------------------------
# ROLL 2: ENCAPSULATION (OUTSIDE-IN IMPOSITION)
# Encapsulate external payload imposed into Vessel yielding new state Container`
# --------------------------------------------------------------------------------
[ ROLL 2 ] : [ Encapsulate ] : { Payload } --> ENCAPSULATE [ Vessel` ] = [ Container` ]
    ( Encapsulate external payload imposed into Vessel yielding new state Container-prime )

# --------------------------------------------------------------------------------
# ROLL 3: IDENTITY & BOUND COMPARISON
# Compare occupancy to capacity under guard: proceed when contained, rollback on overflow
# --------------------------------------------------------------------------------
[ ROLL 3 ] : [ Guard Bounds ] :
    [ { Occupancy` } <= { Capacity } ] & { Container` = Vessel` }
    ! GUARD --> [ Status` = +1:PROCEED ]
    | [ { Occupancy` } > { Capacity } ] --> [ Status` = -1:ROLLBACK ]
    ( Compare occupancy to capacity under guard: proceed when contained, rollback on overflow )

# --------------------------------------------------------------------------------
# ROLL 4: POUR OUT & SET ASIDE (INSIDE-OUT EMERGENCE)
# Inside-out emergence: full container is poured out and set aside at target destination
# --------------------------------------------------------------------------------
[ ROLL 4 ] : [ Pour Out ] : [ Container` } --> SET_ASIDE [ Target ] = [ Sealed Vessel` ]
    ( Inside-out emergence: full container is poured out and set aside at target destination establishing new state Sealed Vessel-prime )

# --------------------------------------------------------------------------------
# ROLL 5: SEQUENCE TRANSITION
# Chain discrete container outward into composite memory structure sequence
# --------------------------------------------------------------------------------
[ ROLL 5 ] : [ Sequence ] : [ Container` ] --> SEQUENCE [ Structure ] = [ Structure` ]
    ( Discrete container chains outward into composite memory structure establishing new state Structure-prime )

# --------------------------------------------------------------------------------
# ROLL 6: COMMIT STATUS & FOURFOLD STATE MACHINE
```

```
# Affirm covenant execution across causal time advancement, await, halt, and rollback
# ------------------------------------------------------------------------------
[ ROLL 6 ] : [ Commit Status ] : [
    # +1: PROCEED (+t: Advance forward along the causal tape)
    { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
        --> [ +1:PROCEED ] |

    #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
    { [ Target ] = BUSY } | { [ Status` ] = PENDING }
        --> [ 0:AWAIT ] :: REPEAT |

    # NULL: HALT (t=empty: Cessation of clock, absolute stop)
    { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
        --> [ HALT:NULL ] |

    # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
    { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
        --> [ -1:ROLLBACK ]
] ( Fourfold commit status: Proceed advances forward, Await holds in equilibrium, Halt ceases execution in Null, Rollback
reverts to origin )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/site.omni

OMNI | 190 LINES | 1818 WORDS | 12.2 KB

```
# ========================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Type: < Idea { Type } : Site > )
# ========================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines physical memory addresses, hardware ports, and MMIO coordinates.
#   - Base Block: Primitive Invariant Type
#   - Category: Idea { Type }
#   - Footprint: [ 1 Address ] { 64-Bit Pointer / Physical Silicon Coordinate }
#   - Ladder Ascension: Build-up with Numbers (/-^)
# ========================================================================

= [Module]: < Lexicon { Types of Ideas } > & [ Noun { Site } ] ( This is the Module for the Lexicon - Ideas (Type): Sites - Nouns ) {
    [ Version ]:   { "1.0.0" } ( The Version is in the context of 1.0.0 ),
    [ Created ]:   { "2026-10-11" } ( Created in the context of October 11, 2026 ),
    [ Updated ]:   { "2026-10-12" } ( Last updated in the context of October 12, 2026 ),
    [ Author ]:    { "CreativeWorkzStudio LLC / ProfessorSeanEX" } ( Author in the context of CreativeWorkzStudio LLC and ProfessorSeanEX ),
    [ System ]:    { OmniCode { Lexicon } } ( System is in the context of the OmniCode Lexicon ),
    [ Witness ]:   < Scripture { Genesis 28:16 KJV } > { "And Jacob awaked out of his sleep, and he said, Surely the Lord is in this place; and I knew it not." } ( Invariant Scripture Witness Genesis 28:16 in the context: Surely the Lord is in this place; and I knew it not ),
    [ Nature ]:    < InvariantType > & { BaseBlock } ( Nature is an Invariant Type and is in the context of Base Block ),
    [ Semantics ]: { PhysicalAddress :: MMIOPort :: SiliconCoordinate } ( Semantics in the context of Physical Address, MMIO Port, and Silicon Coordinate )
}

# ------------------------------------------------------------------------
# LADDER ASCENSION: BUILD-UP WITH NUMBERS
# A Site cannot exist without numbers: silicon memory addresses, offsets,
# and MMIO hardware ports are scalar metrics anchored in reality.
# ------------------------------------------------------------------------
/-^ < Numbers > FROM [ < Lexicon { Types of Ideas } > & [ Noun { Number } ] ] ( Site builds up its physical address coordinates using Numbers from the Lexicon )

# Freestanding Invariant Type Definition
< Site > = < Idea { Type } > & [ Noun { Silicon Coordinate } ] ( Site is the Invariant Idea of Type, and is the Noun of Silicon Coordinate ) {
    # 1. The Ontological Analogy (Site's proportion to the universe)
    [ Relation ]: [ Site ] : [ Coordinate ] :: [ Memory ] : [ Silicon ] ( Site is to Coordinate as Memory is to Silicon ),

    # 2. The Universal Domain (All physical addresses, pointers, ports, and MMIO coordinates)
    [ Domain ]: { Address :: Pointer :: Port :: MMIO } ( Domain encompasses physical addresses, pointers, ports, and MMIO coordinates ),

    # 3. The Physical Substrate (64-bit silicon address pointer)
    [ Substrate ]: [ 1 Address { 64 Bits } ] & [ Silicon Coordinate ] ( Substrate is 1 Address of 64 Bits, and is a Silicon Coordinate ),

    # 4. The Universal Inscription Vector (How external coordinate is bound into memory)
    [ Inscribe ]: [ Coordinate ] --> [ Site ] ( Any silicon coordinate is inscribed into the physical Site address ),

    # 5. The Hardware Routing Law (Bijective routing between RAM pointer and MMIO hardware register)
    [ Law { Routing } ]: [ Physical RAM @ ] | [ Hardware MMIO ::@ ] ( Physical memory addresses route via @ and MMIO ports route via ::@ ),

    # 6. Canonical Exemplars (Standard grounding values for MBR bootstrap origin, COM1, and VGA buffer)
    [ Exemplar { Null } ]: @ 0x00000000 <=> [ Null Coordinate { 0x0 } ] ( Canonical null coordinate represents unmapped memory ),
    [ Exemplar { Bootstrap MBR } ]: @ 0x7C00 <=> [ MBR Origin { 0x7C00 } ] ( Canonical real-mode master boot record physical origin ),
    [ Exemplar { COM1 Serial } ]: ::@ 0x03F8 <=> [ Serial COM1 { 0x03F8 } ] ( Canonical hardware serial port for diagnostic telemetry ),
    [ Exemplar { VGA Buffer } ]: ::@ 0x000B8000 <=> [ VGA Text Buffer { 0x000B8000 } ] ( Canonical physical memory-mapped VGA text buffer ),

    # 7. The Relational Comparison (How sites evaluate address identity)
    [ Compare ]: [ { Target } = { Address } ] ( Site evaluates target pointer against declared physical address ),

    # 8. The Sequence Transition (How sites chain into continuous memory maps)
    [ Sequence ]: [ Site ] --> [ Map ] ( Discrete site addresses chain together into continuous system memory maps ),
```

```
    59
    60      # 9. The Invariant Boundary (Strict single address enclosure)
    61      [ Bound ]: ENCLOSE [ Location ] WITHIN [ 1 Address ] ( Site is strictly enclosed within a single 64-bit address
capacity )
    62  }
    63
    64  # ============================================================================
    65  # BLOCK 2: OPENING BLOCK
    66  # ============================================================================
    67  Opening {
    68      # 1. The Dynamic Input Payload: Accepts incoming memory address or MMIO coordinate
    69      { Coordinate ]: { Input :: @ 0x7C00 } ( Outside-in imposition: external silicon coordinate to be bound into physical
site ),
    70
    71      # 2. The Typed Entity: Site brought down from Block 1 into concrete allocation
    72      [ Site ]: < Site > & [ Location ] ( Denoted Site is typed as invariant Site allocated as a Location ),
    73
    74      # 3. The Physical Address Container: The atomic 64-bit address container
    75      [ Physical Address ]: [ 1 Address ] & { 64 Bits } ( A concrete 1-address container in the context of 64 Bits ),
    76
    77      # 4. The Hardware Bridge: Routing between physical memory and hardware registers
    78      [ Bridge ]: { Physical RAM @ } | { Hardware MMIO ::@ } ( Hardware bridge enforces memory mapping physics ),
    79
    80      # 5. The Output Target: The pointer destination register
    81      [ Target ]: [ Pointer Register ] ::@ 0x00000000 ( Target output is situated at the primary hardware memory pointer
register ),
    82
    83      # 6. Preconditions & State: Proving valid non-null address before binding
    84      [ Preconditions ]: [ Address != 0x00000000 ] & [ Status = +1:PROCEED ] ( Coordinate is non-null and machine status is
armed in Proceed )
    85  }
    86
    87  # ============================================================================
    88  # BLOCK 3: BODY BLOCK
    89  # ============================================================================
    90  Body [
    91      # ----------------------------------------------------------------------
    92      # ROLL 1: LOCATE (LADDER BUILD-UP)
    93      # Build up physical address location with numeric scalar coordinate yielding new state Site`
    94      # ----------------------------------------------------------------------
    95      [ ROLL 1 ] : [ Locate ] : /-^ [ 64 Bits ] --> [ Address ] = [ Site` ]
    96          ( Build up physical address location with numeric scalar coordinate yielding new state Site-prime ),
    97
    98      # ----------------------------------------------------------------------
    99      # ROLL 2: BIND COORDINATE (OUTSIDE-IN IMPOSITION)
   100      # Bind external coordinate imposed into physical Site yielding new state Located Coordinate`
   101      # ----------------------------------------------------------------------
   102      [ ROLL 2 ] : [ Bind ] : { Coordinate ] --> BIND [ Site` ] = [ Located Coordinate` ]
   103          ( Bind external coordinate imposed into physical Site yielding new state Located Coordinate-prime ),
   104
   105      # ----------------------------------------------------------------------
   106      # ROLL 3: IDENTITY & NULL GUARD
   107      # Compare located coordinate to input address under guard: proceed on valid address, rollback on null
   108      # ----------------------------------------------------------------------
   109      [ ROLL 3 ] : [ Guard Address ] :
   110          { Located Coordinate` = Coordinate } & { Located Coordinate` != 0x00000000 }
   111          ! GUARD --> [ Status` = +1:PROCEED ]
   112          | { Located Coordinate` = 0x00000000 } --> [ Status` = -1:ROLLBACK ]
   113          ( Compare located coordinate to input address under guard: proceed on valid address, rollback on null pointer
error ),
   114
   115      # ----------------------------------------------------------------------
   116      # ROLL 4: ROUTE & PROJECT (INSIDE-OUT EMERGENCE)
   117      # Inside-out emergence: verified physical site routes outward to anchor hardware memory
   118      # ----------------------------------------------------------------------
   119      [ ROLL 4 ] : [ Route ] : [ Site` } --> ROUTE [ Target ] = [ Memory Anchor` ]
   120          ( Inside-out emergence: verified physical site routes outward to anchor hardware memory establishing new state
Memory Anchor-prime ),
   121
   122      # ----------------------------------------------------------------------
   123      # ROLL 5: SEQUENCE TRANSITION
   124      # Chain discrete site address outward into continuous system memory map sequence
   125      # ----------------------------------------------------------------------
   126      [ ROLL 5 ] : [ Sequence ] : [ Site` ] --> SEQUENCE [ Map ] = [ Memory Map` ]
   127          ( Discrete site address chains outward into continuous physical memory map establishing new state Memory Map-
prime )
   128  ]
   129
```

```
# ============================================================================
# BLOCK 4: CLOSING BLOCK
# ============================================================================
Closing ( Sealing Gate: Reconcile, validate, tripwire, debug, log, and affirm permanent execution ) {
    # ------------------------------------------------------------------------
    # 1. VERIFICATION OF OPERATIONAL ROLLS
    # ------------------------------------------------------------------------
    [ Verify ]: [
        [ ROLL 1 ] = PROCEED ::
        [ ROLL 2 ] = PROCEED ::
        [ ROLL 3 ] = PROCEED ::
        [ ROLL 4 ] = PROCEED ::
        [ ROLL 5 ] = PROCEED
    ] ( Verify that all five operational rolls achieved affirmative consensus in PROCEED ),

    # ------------------------------------------------------------------------
    # 2. VALIDATION OF INVARIANT BOUNDS & NON-NULL INTEGRITY
    # ------------------------------------------------------------------------
    [ Validate ]: { Located Coordinate` != 0x00000000 } & { Target = BOUND }
        ( Validate that located coordinate is non-null and target pointer is permanently bound ),

    # ------------------------------------------------------------------------
    # 3. ZERO-TOLERANCE TRIPWIRING
    # ------------------------------------------------------------------------
    [ Tripwire ]: { Status` = +1:PROCEED } --> [ ARMED ] | { Status` != +1:PROCEED } --> [ TRIPPED ] --> [ Commit Status
= -1:ROLLBACK ]
        ( Tripwire remains armed in Proceed or trips on anomaly to veto execution ),

    # ------------------------------------------------------------------------
    # 4. DIAGNOSTIC DEBUG TELEMETRY
    # ------------------------------------------------------------------------
    [ Debug ]: { [ Coordinate ] :: [ Site` ] :: [ Target ] :: [ Status` ] }
        ( Expose diagnostic telemetry across input coordinate, site location, target pointer, and machine status ),

    # ------------------------------------------------------------------------
    # 5. EPISODIC INVENTORY LOGGING
    # ------------------------------------------------------------------------
    [ Log ]: [ Journal ]: INSCRIBE { [ Site Bound ] :: [ Site` ] :: [ Target ] }
        ( Inscribe the verified site transaction into the permanent episodic system journal ),

    # ------------------------------------------------------------------------
    # 6. PERMANENT COVENANT EXECUTION & FOURFOLD COMMIT STATUS LOOP
    # ------------------------------------------------------------------------
    [ Commit Status ]: [
        # +1: PROCEED (+t: Advance forward along the causal tape)
        { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
            --> [ +1:PROCEED ] |

        #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
        { [ Target ] = BUSY } | { [ Status` ] = PENDING }
            --> [ 0:AWAIT ] :: REPEAT |

        # NULL: HALT (t=empty: Cessation of clock, absolute stop)
        { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
            --> [ HALT:NULL ] |

        # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
        { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
            --> [ -1:ROLLBACK ]
    ] ( Fourfold state machine: Proceed advances forward, Await holds equilibrium, Halt ceases execution in Null,
Rollback reverts to origin )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/site.tape

TAPE | 89 LINES | 763 WORDS | 5.3 KB

```
# ================================================================================
# PROCESS TAPE: SITE INVARIANT TYPE EXECUTION RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Interprets and executes the atomic lifecycle for Invariant Type < Site >.
#   - Defined Space: [ 1 Address ] { 64 Bits / Silicon Coordinate }
#   - Execution Domain: Lexicon { Types of Ideas }
#   - Ladder Ascension: Build-up with Numbers (/-^)
# ================================================================================

= [Tape]: < Lexicon { Types of Ideas } > & [ Process { Site } ] ( Process Ribbon for Site Address Binding ) {
    WHO   { OmniCode Lexicon } ::
    WHAT  { Site } --> LOCATE { PhysicalAddress } ::
    WHEN  { Addressing Event } ::
    WHERE { Silicon Coordinate } = [ 1 Address ] ::
    WHY   ( Inscribe physical site address invariant and hardware MMIO coordinates ) ::
    HOW   [ +1 : PROCEED ] ::
    < Scripture { Genesis 28:16 KJV } > { "And Jacob awaked out of his sleep, and he said, Surely the Lord is in this
place; and I knew it not." } ( Invariant Scripture Witness Genesis 28:16 in the context: Surely the Lord is in this place; and I
knew it not )
}

# --------------------------------------------------------------------------------
# LADDER ASCENSION: BUILD-UP WITH NUMBERS
# --------------------------------------------------------------------------------
/-^ < Numbers > FROM [ < Lexicon { Types of Ideas } > & [ Noun { Number } ] ] ( Build up physical address coordinates
using Numbers from the Lexicon )

# ================================================================================
# DECLARED ROLLS: ATOMIC INVARIANT EXECUTION
# ================================================================================

# --------------------------------------------------------------------------------
# ROLL 1: LOCATE (LADDER BUILD-UP)
# Build up physical address location with numeric scalar coordinate yielding new state Site`
# --------------------------------------------------------------------------------
[ ROLL 1 ] : [ Locate ] : /-^ [ 64 Bits ] --> [ Address ] = [ Site` ]
    ( Build up physical address location with numeric scalar coordinate yielding new state Site-prime )

# --------------------------------------------------------------------------------
# ROLL 2: BIND COORDINATE (OUTSIDE-IN IMPOSITION)
# Bind external coordinate imposed into physical Site yielding new state Located Coordinate`
# --------------------------------------------------------------------------------
[ ROLL 2 ] : [ Bind ] : { Coordinate } --> BIND [ Site` ] = [ Located Coordinate` ]
    ( Bind external coordinate imposed into physical Site yielding new state Located Coordinate-prime )

# --------------------------------------------------------------------------------
# ROLL 3: IDENTITY & NULL GUARD
# Compare located coordinate to input address under guard: proceed on valid address, rollback on null
# --------------------------------------------------------------------------------
[ ROLL 3 ] : [ Guard Address ] :
    { Located Coordinate` = Coordinate } & { Located Coordinate` != 0x00000000 }
    ! GUARD --> [ Status` = +1:PROCEED ]
    | { Located Coordinate` = 0x00000000 } --> [ Status` = -1:ROLLBACK ]
    ( Compare located coordinate to input address under guard: proceed on valid address, rollback on null pointer error )

# --------------------------------------------------------------------------------
# ROLL 4: ROUTE & PROJECT (INSIDE-OUT EMERGENCE)
# Inside-out emergence: verified physical site routes outward to anchor hardware memory
# --------------------------------------------------------------------------------
[ ROLL 4 ] : [ Route ] : [ Site` ] --> ROUTE [ Target ] = [ Memory Anchor` ]
    ( Inside-out emergence: verified physical site routes outward to anchor hardware memory establishing new state Memory
Anchor-prime )

# --------------------------------------------------------------------------------
# ROLL 5: SEQUENCE TRANSITION
# Chain discrete site address outward into continuous system memory map sequence
# --------------------------------------------------------------------------------
[ ROLL 5 ] : [ Sequence ] : [ Site` ] --> SEQUENCE [ Map ] = [ Memory Map` ]
    ( Discrete site address chains outward into continuous physical memory map establishing new state Memory Map-prime )

# --------------------------------------------------------------------------------
# ROLL 6: COMMIT STATUS & FOURFOLD STATE MACHINE
```

```
# Affirm covenant execution across causal time advancement, await, halt, and rollback
# ------------------------------------------------------------------------------
[ ROLL 6 ] : [ Commit Status ] : [
    # +1: PROCEED (+t: Advance forward along the causal tape)
    { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
        --> [ +1:PROCEED ] |

    #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
    { [ Target ] = BUSY } | { [ Status` ] = PENDING }
        --> [ 0:AWAIT ] :: REPEAT |

    # NULL: HALT (t=empty: Cessation of clock, absolute stop)
    { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
        --> [ HALT:NULL ] |

    # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
    { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
        --> [ -1:ROLLBACK ]
] ( Fourfold commit status: Proceed advances forward, Await holds in equilibrium, Halt ceases execution in Null, Rollback
reverts to origin )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/state.omni

OMNI | 230 LINES | 2189 WORDS | 14.4 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Type: < Idea { Type } : State > )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the Quad-State truth, causal clock, and control flow invariant.
#   - Base Block: Primitive Invariant Type
#   - Category: Idea { Type }
#   - Footprint: [ 1 Trit ] { 2 Bits / Quad-State Causal Register }
#   - Named Registers: [ STATE TURN BOUND IT ]
#   - States: +1 (PROCEED), 0 (AWAIT), -1 (ROLLBACK), NULL (HALT)
# ================================================================================

= [Module]: < Lexicon { Types of Ideas } > & [ Noun { State } ] ( This is the Module for the Lexicon - Ideas (Type): States - Nouns ) {
    [ Version ]:   { "1.0.0" } ( The Version is in the context of 1.0.0 ),
    [ Created ]:   { "2026-10-11" } ( Created in the context of October 11, 2026 ),
    [ Updated ]:   { "2026-10-13" } ( Last updated in the context of October 13, 2026 ),
    [ Author ]:    { "CreativeWorkzStudio LLC / ProfessorSeanEX" } ( Author in the context of CreativeWorkzStudio LLC and ProfessorSeanEX ),
    [ System ]:    { OmniCode { Lexicon } } ( System is in the context of the OmniCode Lexicon ),
    [ Witness ]:   < Scripture { Ecclesiastes 3:1 KJV } > { "To every thing there is a season, and a time to every purpose under the heaven:" } ( Invariant Scripture Witness Ecclesiastes 3:1 in the context: To every thing there is a season, and a time to every purpose under the heaven ),
    [ Nature ]:    < InvariantType > & { BaseBlock } ( Nature is an Invariant Type and is in the context of Base Block ),
    [ Semantics ]: { QuadStateTruth :: CausalClock :: ControlFlowState } ( Semantics in the context of Quad State Truth, Causal Clock, and Control Flow State )
}

# ------------------------------------------------------------------------------
# LADDER FOUNDATION: INDEPENDENT TEMPORAL BASE
# State does not build up from space or memory envelopes: it is the independent
# atomic base of causal time and Quad-State truth governing execution.
# ------------------------------------------------------------------------------

# Freestanding Invariant Type Definition
< State > = < Idea { Type } > & [ Noun { Causal Truth } ] ( State is the Invariant Idea of Type, and is the Noun of Causal Truth ) {
    # 1. The Ontological Analogy (State's proportion to the universe)
    [ Relation ]: [ State ] : [ Truth ] :: [ Trit ] : [ Causality ] ( State is to Truth as Trit is to Causality ),

    # 2. The Universal Domain (All quad-state transitions, cadence pulses, and control flows)
    [ Domain ]: { Proceed :: Await :: Rollback :: Halt } ( Domain encompasses Proceed, Await, Rollback, and Halt ),

    # 3. The Physical Substrate (1 balanced trit or 2 silicon bits)
    [ Substrate ]: [ 1 Trit { 2 Bits } ] & [ Causal Truth ] ( Substrate is 1 Trit of 2 Bits, and is Causal Truth ),

    # 4. The Universal Inscription Vector (How external truth signal is recorded into the state register)
    [ Inscribe ]: [ Signal ] --> [ STATE ] ( Any truth signal is inscribed into the STATE register ),

    # 5. The Hardware Polarity Transition (Abstract archetype lowers to concrete silicon register)
    [ Lower ]: [ +State v-- State- ::@ 0x0000 ] ( Abstract idea lowers to concrete register at base offset ),

    # 6. The Core Quad-State Truth Law (The fourfold ontological states of computation)
    [ Law { Quad-State } ]: { T4 = [ +1:PROCEED | 0:AWAIT | -1:ROLLBACK | HALT:NULL ] } ( The balanced ternary truth system governing all causation ),

    # 7. Canonical Exemplars (Standard grounding values for Proceed, Await, Rollback, and Halt)
    [ Exemplar { Proceed } ]: (+1) <=> [ PROCEED { Forward Causation } ] ( Forward causation advances execution ),
    [ Exemplar { Await } ]: 0 <=> [ AWAIT { Equilibrium Standby } ] ( Standby equilibrium holds pending external resolution ),
    [ Exemplar { Rollback } ]: (-1) <=> [ ROLLBACK { Retroactive Reversal } ] ( Retroactive reversal unwinds causal drift ),
    [ Exemplar { Halt } ]: NULL <=> [ HALT { Terminal Detent } ] ( Terminal detent halts the execution clock ),

    # 8. The Canonical Looping Law (The formal loop grammar connecting Quad-State to hardware registers)
    [ Law { Loop } ]: LOOP [ WHEN [ { IT } < { BOUND } ] --> [ IT` ] PROCEED to { TURN } ! [ { IT } > { BOUND } ] AWAIT for [ IT` ] :: HALT WHEN [ IT` ] = NULL ] ( Canonical loop statement governing IT, BOUND, and TURN across the quad states ),

    # 9. The Relational Comparison (How states evaluate active truth against signal)
    [ Compare ]: [ { STATE } = { Signal } ] ( State evaluates active register against expected truth signal ),

    # 10. The Sequence Transition (How states chain into continuous causal timelines)
```

```
    [ Sequence ]: [ State ] --> [ Timeline ] ( Discrete states chain together into continuous execution timelines ),

    # 11. The Invariant Boundary (Strict single trit enclosure)
    [ Bound ]: ENCLOSE [ Truth ] WITHIN [ 1 Trit ] ( State is strictly enclosed within a single trit capacity )
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {
    # 1. The Dynamic Input Signal: Accepts incoming causal truth signal
    { Signal ]: { Input :: [ +1:PROCEED ] } ( Outside-in imposition: external causal truth signal to be evaluated ),

    # 2. The Typed Entity: State brought down from Block 1 into concrete allocation
    [ State ]: < State > & [ Register ] ( Denoted State is typed as invariant State allocated as a Register ),

    # 3. The Named Hardware Register Bank: Core registers for state and loop execution
    [ Registers ]: [ STATE TURN BOUND IT ] ( Named hardware registers armed for state and cadence execution ),

    # 4. The Initial Register Values: Initializing loop parameters
    [ Setup ]: [ STATE = 0:AWAIT ] :: [ IT = 0 ] :: [ BOUND = 3 ] :: [ TURN = 0 ] ( Initialize state in Await,
accumulator at zero, bound at 3, turn at zero ),

    # 5. The Hardware Clock Bridge: Arming the causal clock and cadence transitions
    [ Clock ]: { (+1:PROCEED) :: (0:AWAIT) :: (-1:ROLLBACK) :: (HALT:NULL) } ( Hardware clock bridges the four states of
truth ),

    # 6. The Output Target: The execution status register
    [ Target ]: [ Status Register ] ::@ 0x00000000 ( Target output is situated at the primary execution status register
),

    # 7. Preconditions & State: Proving non-null valid signal before transition
    [ Preconditions ]: [ Signal != NULL ] & [ Status = +1:PROCEED ] ( Signal is non-null and machine status is armed in
Proceed )
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # --------------------------------------------------------------------------
    # ROLL 1: INSCRIBE STATE REGISTER (OUTSIDE-IN IMPOSITION)
    # Inscribe incoming truth signal into the STATE register, establishing new state STATE`
    # --------------------------------------------------------------------------
    [ ROLL 1 ] : [ Inscribe Register ] : { Signal ] --> INSCRIBE [ STATE ] = [ STATE` ]
        ( Inscribe external truth signal imposed into STATE register yielding new state STATE-prime ),

    # --------------------------------------------------------------------------
    # ROLL 2: AFFIRMATIVE PROCEED TRANSITION
    # Transition along the forward causal vector: PROCEED advances forward to { TURN }
    # --------------------------------------------------------------------------
    [ ROLL 2 ] : [ Affirmative Transition ] : [ Clock ]: [ STATE` ] --> [ +1:PROCEED ] to { TURN } = [ Active State` ]
        ( Transition STATE-prime through clock cadence into affirmative Proceed advancing forward to TURN ),

    # --------------------------------------------------------------------------
    # ROLL 3: EQUILIBRIUM AWAIT TRANSITION
    # Transition into equilibrium: AWAIT holds in zero-sum balance for [ IT` ]
    # --------------------------------------------------------------------------
    [ ROLL 3 ] : [ Standby Equilibrium ] : [ Clock ]: [ STATE` ] <=> [ 0:AWAIT ] for [ IT` ] = [ Standby State` ]
        ( Transition STATE-prime into equilibrium Await holding in standby for IT-prime ),

    # --------------------------------------------------------------------------
    # ROLL 4: CAVEAT GUARD & ROLLBACK
    # Evaluate incoming caveat with !: proceed on alignment, or trigger -1:ROLLBACK on discrepancy
    # --------------------------------------------------------------------------
    [ ROLL 4 ] : [ Guard Condition ] :
        { STATE` = Signal }
        ! GUARD --> [ Status` = +1:PROCEED ]
        | { STATE` != Signal } --> [ Status` = -1:ROLLBACK ]
        ( Guard truth against signal under caveat: proceed on alignment, rollback on discrepancy ),

    # --------------------------------------------------------------------------
    # ROLL 5: TERMINAL DETENT TRAP
    # Trap bottom type HALT:NULL on illegal signal or terminal detent
    # --------------------------------------------------------------------------
    [ ROLL 5 ] : [ Trap Detent ] :
        { Status` = NULL }
        ! TRAP --> [ HALT:NULL ]
```

```
        | { Status` != NULL } --> [ Active State` = +1:PROCEED ]
        ( Trap null bottom state under guard detent or proceed with active state ),

    # ------------------------------------------------------------------------
    # ROLL 6: THE CANONICAL LOOP STATEMENT
    # Execute the formal OmniCode loop statement governing IT, BOUND, and TURN across states
    # ------------------------------------------------------------------------
    [ ROLL 6 ] : [ Loop ] :
        LOOP [
            WHEN [ { IT } < { BOUND } ] --> [ IT` ] PROCEED to { TURN }
            ! [ { IT } > { BOUND } ] AWAIT for [ IT` ]
            :: HALT WHEN [ IT` ] = NULL
        ] ( Execute canonical quad-state accumulator loop: proceed to TURN while under BOUND, await when above BOUND,
halt on NULL ),

    # ------------------------------------------------------------------------
    # ROLL 7: GOVERN ENGINE (INSIDE-OUT EMERGENCE)
    # Inside-out emergence: verified state emerges outward to govern the machine clock
    # ------------------------------------------------------------------------
    [ ROLL 7 ] : [ Govern Engine ] : [ STATE` } --> GOVERN [ Control Flow ] = [ Clock` ]
        ( Inside-out emergence: verified state emerges outward to govern machine clock establishing new state Clock-prime
),

    # ------------------------------------------------------------------------
    # ROLL 8: SEQUENCE TRANSITION
    # Chain discrete state into continuous execution timeline sequence
    # ------------------------------------------------------------------------
    [ ROLL 8 ] : [ Sequence ] : [ STATE` ] --> SEQUENCE [ Timeline ] = [ Timeline` ]
        ( Discrete state chains outward into continuous causal timeline establishing new state Timeline-prime )
]

# ============================================================================
# BLOCK 4: CLOSING BLOCK
# ============================================================================
Closing ( Sealing Gate: Reconcile, validate, tripwire, debug, log, and affirm permanent execution ) {
    # ------------------------------------------------------------------------
    # 1. VERIFICATION OF OPERATIONAL ROLLS
    # ------------------------------------------------------------------------
    [ Verify ]: [
        [ ROLL 1 ] = PROCEED ::
        [ ROLL 2 ] = PROCEED ::
        [ ROLL 3 ] = PROCEED ::
        [ ROLL 4 ] = PROCEED ::
        [ ROLL 5 ] = PROCEED ::
        [ ROLL 6 ] = PROCEED ::
        [ ROLL 7 ] = PROCEED ::
        [ ROLL 8 ] = PROCEED
    ] ( Verify that all eight operational rolls achieved affirmative consensus in PROCEED ),

    # ------------------------------------------------------------------------
    # 2. VALIDATION OF INVARIANT BOUNDS & QUAD-STATE INTEGRITY
    # ------------------------------------------------------------------------
    [ Validate ]: { STATE` <= [ 1 Trit ] } & { [ ROLL 6 ] = PROCEED }
        ( Validate that state remains strictly enclosed within 1 Trit and canonical loop sealed in Proceed ),

    # ------------------------------------------------------------------------
    # 3. ZERO-TOLERANCE TRIPWIRING
    # ------------------------------------------------------------------------
    [ Tripwire ]: { Status` = +1:PROCEED } --> [ ARMED ] | { Status` != +1:PROCEED } --> [ TRIPPED ] --> [ Commit Status
= -1:ROLLBACK ]
        ( Tripwire remains armed in Proceed or trips on anomaly to veto execution ),

    # ------------------------------------------------------------------------
    # 4. DIAGNOSTIC DEBUG TELEMETRY
    # ------------------------------------------------------------------------
    [ Debug ]: { [ Signal ] :: [ STATE` ] :: [ IT` ] :: [ TURN` ] :: [ Status` ] }
        ( Expose diagnostic telemetry across input signal, state register, accumulator IT, cadence TURN, and status ),

    # ------------------------------------------------------------------------
    # 5. EPISODIC INVENTORY LOGGING
    # ------------------------------------------------------------------------
    [ Log ]: [ Journal ]: INSCRIBE { [ State Resolved ] :: [ STATE` ] :: [ Target ] }
        ( Inscribe the verified state transaction into the permanent episodic system journal ),

    # ------------------------------------------------------------------------
    # 6. PERMANENT COVENANT EXECUTION & FOURFOLD COMMIT STATUS LOOP
    # ------------------------------------------------------------------------
    [ Commit Status ]: [
```

```
        # +1: PROCEED (+t: Advance forward along the causal tape)
        { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
            --> [ +1:PROCEED ] |

        #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
        { [ Target ] = BUSY } | { [ Status` ] = PENDING }
            --> [ 0:AWAIT ] :: REPEAT |

        # NULL: HALT (t=empty: Cessation of clock, absolute stop)
        { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
            --> [ HALT:NULL ] |

        # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
        { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
            --> [ -1:ROLLBACK ]
    ] ( Fourfold state machine: Proceed advances forward, Await holds equilibrium, Halt ceases execution in Null,
Rollback reverts to origin )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/state.tape

TAPE | 117 LINES | 949 WORDS | 6.7 KB

```
# ================================================================================
# PROCESS TAPE: STATE INVARIANT TYPE EXECUTION RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Interprets and executes the atomic lifecycle for Invariant Type < State >.
#   - Defined Space: [ 1 Trit ] { 2 Bits / Quad-State Causal Register }
#   - Execution Domain: Lexicon { Types of Ideas }
#   - Named Registers: [ STATE TURN BOUND IT ]
#   - Kinematics: 8 Operational Rolls with Canonical Quad-State Accumulator Loop
# ================================================================================

= [Tape]: < Lexicon { Types of Ideas } > & [ Process { State } ] ( Process Ribbon for Quad-State Causal Governance ) {
    WHO   { OmniCode Lexicon } ::
    WHAT  { State } --> GOVERN { CausalTruth } ::
    WHEN  { Control Event } ::
    WHERE { State Register } = [ 1 Trit ] ::
    WHY   ( Inscribe quad-state truth invariant and execute canonical loops ) ::
    HOW   [ +1 : PROCEED ] ::
    < Scripture { Ecclesiastes 3:1 KJV } > { "To every thing there is a season, and a time to every purpose under the heaven:" } ( Invariant Scripture Witness Ecclesiastes 3:1 in the context: To every thing there is a season, and a time to every purpose under the heaven )
}

# --------------------------------------------------------------------------------
# LADDER FOUNDATION: INDEPENDENT TEMPORAL BASE
# --------------------------------------------------------------------------------

# ================================================================================
# DECLARED ROLLS: ATOMIC INVARIANT EXECUTION
# ================================================================================

# --------------------------------------------------------------------------------
# ROLL 1: INSCRIBE STATE REGISTER (OUTSIDE-IN IMPOSITION)
# Inscribe incoming truth signal into the STATE register
# --------------------------------------------------------------------------------
[ ROLL 1 ] : [ Inscribe Register ] : { Signal ] --> INSCRIBE [ STATE ] = [ STATE` ]
    ( Inscribe external truth signal imposed into STATE register yielding new state STATE-prime )

# --------------------------------------------------------------------------------
# ROLL 2: AFFIRMATIVE PROCEED TRANSITION
# Transition along forward causal vector: PROCEED advances forward to { TURN }
# --------------------------------------------------------------------------------
[ ROLL 2 ] : [ Affirmative Transition ] : [ Clock ]: [ STATE` ] --> [ +1:PROCEED ] to { TURN } = [ Active State` ]
    ( Transition STATE-prime through clock cadence into affirmative Proceed advancing forward to TURN )

# --------------------------------------------------------------------------------
# ROLL 3: EQUILIBRIUM AWAIT TRANSITION
# Transition into equilibrium: AWAIT holds in zero-sum balance for [ IT` ]
# --------------------------------------------------------------------------------
[ ROLL 3 ] : [ Standby Equilibrium ] : [ Clock ]: [ STATE` ] <=> [ 0:AWAIT ] for [ IT` ] = [ Standby State` ]
    ( Transition STATE-prime into equilibrium Await holding in standby for IT-prime )

# --------------------------------------------------------------------------------
# ROLL 4: CAVEAT GUARD & ROLLBACK
# Evaluate incoming caveat with !: proceed on alignment, or trigger -1:ROLLBACK on discrepancy
# --------------------------------------------------------------------------------
[ ROLL 4 ] : [ Guard Condition ] :
    { STATE` = Signal }
    ! GUARD --> [ Status` = +1:PROCEED ]
    | { STATE` != Signal } --> [ Status` = -1:ROLLBACK ]
    ( Guard truth against signal under caveat: proceed on alignment, rollback on discrepancy )

# --------------------------------------------------------------------------------
# ROLL 5: TERMINAL DETENT TRAP
# Trap bottom type HALT:NULL on illegal signal or terminal detent
# --------------------------------------------------------------------------------
[ ROLL 5 ] : [ Trap Detent ] :
    { Status` = NULL }
    ! TRAP --> [ HALT:NULL ]
    | { Status` != NULL } --> [ Active State` = +1:PROCEED ]
    ( Trap null bottom state under guard detent or proceed with active state )

# --------------------------------------------------------------------------------
```

```
# ROLL 6: THE CANONICAL LOOP STATEMENT
# Execute the formal OmniCode loop statement governing IT, BOUND, and TURN across states
# ------------------------------------------------------------------------------
[ ROLL 6 ] : [ Loop ] :
    LOOP [
        WHEN [ { IT } < { BOUND } ] --> [ IT` ] PROCEED to { TURN }
        ! [ { IT } > { BOUND } ] AWAIT for [ IT` ]
        :: HALT WHEN [ IT` ] = NULL
    ] ( Execute canonical quad-state accumulator loop: proceed to TURN while under BOUND, await when above BOUND, halt on NULL )

# ------------------------------------------------------------------------------
# ROLL 7: GOVERN ENGINE (INSIDE-OUT EMERGENCE)
# Inside-out emergence: verified state emerges outward to govern the machine clock
# ------------------------------------------------------------------------------
[ ROLL 7 ] : [ Govern Engine ] : [ STATE` } --> GOVERN [ Control Flow ] = [ Clock` ]
    ( Inside-out emergence: verified state emerges outward to govern machine clock establishing new state Clock-prime )

# ------------------------------------------------------------------------------
# ROLL 8: SEQUENCE TRANSITION
# Chain discrete state into continuous execution timeline sequence
# ------------------------------------------------------------------------------
[ ROLL 8 ] : [ Sequence ] : [ STATE` ] --> SEQUENCE [ Timeline ] = [ Timeline` ]
    ( Discrete state chains outward into continuous causal timeline establishing new state Timeline-prime )

# ------------------------------------------------------------------------------
# ROLL 9: COMMIT STATUS & FOURFOLD STATE MACHINE
# Affirm covenant execution across causal time advancement, await, halt, and rollback
# ------------------------------------------------------------------------------
[ ROLL 9 ] : [ Commit Status ] : [
    # +1: PROCEED (+t: Advance forward along the causal tape)
    { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
        --> [ +1:PROCEED ] |

    #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
    { [ Target ] = BUSY } | { [ Status` ] = PENDING }
        --> [ 0:AWAIT ] :: REPEAT |

    # NULL: HALT (t=empty: Cessation of clock, absolute stop)
    { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
        --> [ HALT:NULL ] |

    # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
    { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
        --> [ -1:ROLLBACK ]
] ( Fourfold commit status: Proceed advances forward, Await holds in equilibrium, Halt ceases execution in Null, Rollback reverts to origin )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/text.omni

OMNI | 62 LINES | 249 WORDS | 2.2 KB

```
# ========================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Type: <IDEA:TYPE:TEXT> )
# ========================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines inscriptional sequences of characters (Words, Phrases, and Strings).
#   - Base Block: Primitive Invariant Type
#   - Category: Idea
#   - Subcategory: Type
#   - Sequence: Ordered ribbon of < Character > cells
# ========================================================================

Module < Lexicon.Nouns.Ideas.Types.Text > ( Inscriptional Word, Phrase & String Invariant ) {
    version: < "1.0.0" >,
    system: < OmniCode.Lexicon >,
    semantics: < BaseBlock :: InvariantType :: InscriptionalText :: TextRibbon >
}

# Invariant Type Declaration: <IDEA:TYPE:TEXT>
< Idea : Type : Text > [ capacity: dynamic cells ] {
    # Polarity transition: Abstract archetype lowers to concrete ribbon
    +Text v-- Text- ::@ 0x0000

    # Inscription bounds
    ENCLOSE [ ribbon : Sequence [ Character ] ] WITHIN [ capacity ]

    # Clarified literal representations
    example_word: "MILLENIUMOS",
    example_phrase: "MILLENIUMOS: OMNICODE 32-TRIT OS ONLINE"
}

# ========================================================================
# BLOCK 2: OPENING BLOCK
# ========================================================================
Opening {
    requires: [
        Text : InvariantNoun ::
        Character : InvariantNoun
    ],
    preconditions: [ +1 ]
}

# ========================================================================
# BLOCK 3: BODY BLOCK
# ========================================================================
Body [
    # Verify text ribbon allocation and phrase inscription
    text_assertion [
        [ TEXT ] [ 32 ] :: { Text } --> INSCRIBE {Phrase}
    ] ( Inscriptional text ribbon allocated holding character sequence )
]

# ========================================================================
# BLOCK 4: CLOSING BLOCK
# ========================================================================
Closing {
    verify: [
        text_assertion = yes
    ],
    commit_status [ PROCEED- ] ( Text invariant type sealed in PROCEED )
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/text.tape

TAPE | 33 LINES | 210 WORDS | 1.6 KB

```
# ================================================================================
# TAPE HEAD: TEXT INVARIANT TYPE PROCESS RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Type <IDEA:TYPE:TEXT>.
# ================================================================================

[Tape.Text] : WHO {OmniCode_Kernel} WHAT {TextType} --> INSCRIBE {TextRibbon}
    WHEN {TypeGenesis}
    WHERE {text_partition} = [ 64 ]
    WHY ( Inscribe text ribbon type invariant and clarified string phrases )
    HOW [ +1 : PROCEED ]
    :: < text_landmark > ( Habakkuk 2:2 - 'Write the vision, and make it plain upon tables' )

# ================================================================================
# SUBORDINATE ROLLS: ATOMIC INVARIANT RESOLUTION
# ================================================================================

# Roll 1: Reference abstract text archetype
roll_1 [ +Text ] ( Abstract text archetype referenced )

# Roll 2: Allocate concrete text ribbon
roll_2 [ Text --> Inscribed_Ribbon @ 0 ] ( Inscribe concrete text ribbon at offset 0 )

# Roll 3: Assert structural projection and clarified phrase
roll_3 [ [ TEXT ] [ 32 ] :: "MILLENIUMOS ONLINE" --> [ Inscribed_Ribbon ] ] ( Clarified phrase inscribed )

# Roll 4: Verify text integrity into result prime
roll_4 [ Inscribed_Ribbon = Text ! [ +1 ] --> [ text_verified` ] ] ( Verification yields result prime )

# Roll 5: Seal in affirmative proceed
roll_5 [ commit_status [ PROCEED- ] ] ( Text invariant type sealed and verified )
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/compile_flags.txt

TXT | 5 LINES | 4 WORDS | 0.0 KB

```
-Iinclude
-Wall
-Wextra
-std=c11
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/number.o

DOC/BINARY | 1 LINES | 5 WORDS | 9.0 KB

[Binary Artifact: number.o (9.0 KB)]

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/character.h

H | 300 LINES | 1795 WORDS | 13.1 KB

```h
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT TYPE DEFINITION
 * ( Invariant Type: < Idea { Type } : Character > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Defines the atomic letter, glyph, and inscription rune in the OmniCode
 *   Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Cell ] { 5 Balanced Trits / 8 Bits }
 *   - System: OmniCode { Lexicon }
 *   - Witness: Scripture Exodus 31:18 KJV
 *     "And he gave unto Moses, when he had made an end of communing with him upon
 *      mount Sinai, two tables of testimony, tables of stone, written with the
 *      finger of God."
 *   - Semantics: { AtomicGlyph :: LetterRune :: BalancedCellTransduction }
 *   - Invariant Law:
 *     { Character } = [ Enclosed ]: TRUE (Strict single-cell capacity).
 *
 * ONTOLOGICAL CORRESPONDENCE TO OMNICODE:
 *   OmniCode Block 1 declares the immutable laws of the universe for this type.
 *   In C, Block 1 is realized exclusively as:
 *     1. Invariant hardware target addresses and substrate constants.
 *     2. The freestanding invariant struct (omni_character_t) holding dual substrate.
 *     3. Pure mathematical conversion and verification functions that operate
 *        without runtime context dependencies.
 * ============================================================================ */

#ifndef OMNI_CHARACTER_H
#define OMNI_CHARACTER_H

#include "omni_base.h"
#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>

#ifdef __cplusplus
extern "C" {
#endif

/* ----------------------------------------------------------------------------
 * 1. INVARIANT CONSTANTS & HARDWARE LOCATORS (OMNICODE BLOCK 1 & 2)
 * ---------------------------------------------------------------------------- */

/** Number of balanced trits per character cell (3^5 = 243 states). */
#define OMNI_CHAR_TRITS_PER_CELL OMNI_TRITS_PER_CELL

/** Canonical Exemplar: ASCII 'A' (Byte value 65). */
#define OMNI_EXEMPLAR_GLYPH_A    ((uint8_t)'A')

/**
 * @brief Hardware Target Locators (Composed operator ::@).
 *
 * OmniCode Specification:
 *   [ Target ]: { [ Terminal Stdout ] ::@ 0x01 | [ Screen VGA ] ::@ 0x000B8000 }
 */
#define OMNI_TARGET_STDOUT       ((uintptr_t)0x01)       /**< Hosted POSIX terminal stdout stream descriptor */
#define OMNI_TARGET_VGA          ((uintptr_t)0x000B8000) /**< Bare-metal x86 text mode video memory MMIO buffer */

/* ----------------------------------------------------------------------------
 * 2. FREESTANDING INVARIANT TYPE STRUCTURE: omni_character_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief Invariant Type < Character >: The atomic rune of symbolic computing.
 *
 * Encloses the dual substrate:
 *   1. Physical Silicon Substrate: 1 Byte (8 bits, 0..255).
 *   2. Virtual Balanced Trit Substrate: 1 Cell (5 balanced trits, {-1, 0, +1}).
 *
 * Invariant Law:
 *   { Character } = [ Enclosed ]: TRUE (Strict single-cell boundary).
 */
```

```c
typedef struct {
    uint8_t     glyph;                                  /**< Physical byte substrate (8-bit ASCII/binary glyph) */
    omni_trit_t trits[OMNI_CHAR_TRITS_PER_CELL];        /**< Virtual cell substrate (5 balanced ternary trits) */
    bool        is_enclosed;                            /**< Invariant boundary enclosure guarantee */
} omni_character_t;

/* ---------------------------------------------------------------------------
 * 3. PURE INVARIANT OPERATIONS & TRANSDUCTION LAWS
 * --------------------------------------------------------------------------- */

/**
 * @brief Inscription Constructor: Constructs a Character directly from a raw 8-bit glyph.
 *
 * Evaluates OmniCode: [ Inscribe ]: [ Glyph ] --> [ Character ]
 * Automatically performs bijective transduction into 5 balanced trits and
 * establishes the invariant enclosure seal (is_enclosed = true).
 *
 * @param glyph The 8-bit byte codepoint to inscribe.
 * @return Fully initialized omni_character_t in equilibrium.
 */
omni_character_t omni_character_from_glyph(uint8_t glyph);

/**
 * @brief Hardware Transduction Law (<=>): Bijective mapping from 8-bit byte to 5 balanced trits.
 *
 * Evaluates OmniCode: [ Transduce ]: [ Character { Byte } ] <=> [ Cell { Trits } ]
 *
 * Mathematical Invariant:
 *   Each trit is in {-1, 0, +1}, satisfying:
 *     byte = sum_{i=0..4} trits[i] * 3^i
 *   Canonical Exemplar for 'A' (65):
 *     65 <=> [ -1, +1, +1, -1, +1 ]  (-1 + 3 + 9 - 27 + 81 = 65).
 *
 * @param byte_val Input 8-bit byte value (0..255).
 * @param out_trits Destination array of 5 balanced trits.
 */
void omni_character_transduce(uint8_t byte_val, omni_trit_t out_trits[OMNI_CHAR_TRITS_PER_CELL]);

/**
 * @brief Inverse Transduction Law (<=>): Reconstructs an 8-bit byte from 5 balanced trits.
 *
 * Evaluates the bidirectional symmetry of the invariant equivalence operator (<=>).
 * Rebuilds the silicon byte from the balanced ternary representation.
 *
 * @param trits Array of 5 balanced trits.
 * @return Reconstructed 8-bit unsigned byte value.
 */
uint8_t omni_character_to_byte(const omni_trit_t trits[OMNI_CHAR_TRITS_PER_CELL]);

/**
 * @brief Relational Equality Evaluation: Compares two character entities.
 *
 * Evaluates OmniCode: [ Compare ]: [ { Character } = { Glyph } ]
 *
 * @param a First character operand.
 * @param b Second character operand.
 * @return true if glyph values match and both are enclosed; false otherwise.
 */
bool omni_character_eq(const omni_character_t *a, const omni_character_t *b);

/**
 * @brief Boundary Invariant Verification: Proves single-cell enclosure.
 *
 * Evaluates OmniCode: [ Bound ]: { Character } = [ Enclosed ]: TRUE
 *
 * @param c Pointer to character instance to verify.
 * @return true if valid pointer and enclosure flag is sealed; false otherwise.
 */
bool omni_character_verify_bounds(const omni_character_t *c);

/**
 * @brief Bit-Level Extraction: Retrieves bit 0..7 from the glyph byte.
 *
 * Slices into the shared 8-bit base level that Character shares with Number.
 *
 * @param c Pointer to character entity.
 * @param bit_idx Bit index (0..7, where 0 is LSB and 7 is ASCII parity rail).
 * @return 1 if set, 0 if clear or on null pointer.
```

```
 */
uint8_t omni_character_get_bit(const omni_character_t *c, uint8_t bit_idx);

/**
 * @brief Standard ASCII Invariant Verification: Proves glyph in 0..127 range.
 *
 * Validates bit 7 is 0, leaving bit 7 free to serve as an immutable witness.
 *
 * @param c Pointer to character entity.
 * @return true if glyph <= 127 (bit 7 == 0); false otherwise.
 */
bool omni_character_is_ascii(const omni_character_t *c);

/* ============================================================================
 * BLOCK 2: OPENING BLOCK
 * ( Contextual Genesis, Input Imposition & Hardware Target Binding )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   OmniCode Block 2 brings the timeless laws of Block 1 down into active space
 *   and time for a concrete execution run:
 *     1. [ Glyph ]: Captures outside-in payload (CLI argument or stream input).
 *     2. [ Character ] & [ Cell ]: Allocates physical single-cell storage.
 *     3. [ Target ]: Binds to a concrete hardware address (Stdout or VGA MMIO).
 *     4. [ Preconditions ]: Checks ALU balance and arms the execution tripwire.
 * ============================================================================ */

/**
 * @brief Execution Context for Character Lifecycle Rolls.
 *
 * Encapsulates the complete runtime environment for the 4-block lifecycle:
 * outside-in input, allocated cell, target hardware address, optional sequence
 * destination, and safety interlocks.
 */
typedef struct {
    uint8_t               input_glyph;      /**< [ Glyph ]: Imposed outside-in payload */
    omni_character_t      character;        /**< [ Character ] & [ Cell ]: Concrete cell container */
    uintptr_t             target_address;   /**< [ Target ]: Hardware locator (0x01 Stdout or 0x000B8000 VGA) */
    void                 *sequence_target;  /**< [ Sequence ]: Destination buffer for chaining into Text */
    omni_quad_state_t     status;           /**< [ Preconditions ]: Quad-State truth (+1:PROCEED) */
    omni_tripwire_state_t tripwire;         /**< [ Preconditions ]: Subsystem safety interlock */
} omni_character_context_t;

/**
 * @brief Initializes the Character context, applies outside-in imposition,
 *        binds the hardware target, and arms the safety tripwire.
 *
 * Evaluates OmniCode Block 2: Opening { ... }
 *
 * @param ctx Pointer to caller-allocated character execution context.
 * @param input_glyph Outside-in byte payload (defaults to exemplar 'A' if 0).
 * @param target_address Hardware locator (e.g. OMNI_TARGET_STDOUT or OMNI_TARGET_VGA; defaults to STDOUT if 0).
 */
void omni_character_open(omni_character_context_t *ctx, uint8_t input_glyph, uintptr_t target_address);

/* ============================================================================
 * BLOCK 3: BODY BLOCK
 * ( Operational Rolls, Guard Interlocks & Hardware Emission )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   OmniCode Block 3 processes the discrete causal turns (rolls) that advance
 *   state along the causal timeline using the prime operator (`):
 *     - Roll 1: Inscribe input glyph into the Cell container.
 *     - Roll 2: Transduce 8 bits <=> 5 balanced trits.
 *     - Roll 3: Guard identity & boundary invariants via adversative catch (!).
 *     - Roll 4: Project Character` to the hardware target (terminal or VGA).
 *     - Roll 5: Sequence Character` into continuous Text ribbon if bound.
 * ============================================================================ */

/**
 * @brief Roll 1: Inscribes outside-in glyph into the cell, advancing state to Character`.
 *
 * Evaluates OmniCode: [ Inscribe ] : [ Glyph ] --> INSCRIBE [ Cell ] = [ Character` ]
 *
 * @param ctx Active execution context.
 * @param glyph The byte codepoint being inscribed.
 */
void omni_character_inscribe(omni_character_context_t *ctx, uint8_t glyph);

```

```
/**
 * @brief Roll 4: Projects Character` outward to the bound hardware target address.
 *
 * Evaluates OmniCode: [ Project ] : [ Character` ] --> PROJECT [ Target ] = [ Target` ]
 *
 * Machine Effect:
 *   - Target == 0x01 (Stdout): Emits glyph byte to file descriptor 1 (terminal).
 *   - Target == 0x000B8000 (VGA): Writes glyph + attribute to MMIO video memory.
 *
 * @param c Pointer to the verified Character to project.
 * @param target_address The physical destination address (0x01 or 0x000B8000).
 * @return true if projection succeeded; false on hardware fault.
 */
bool omni_character_project(const omni_character_t *c, uintptr_t target_address);

/**
 * @brief Roll 5: Yields Character` to continuous Text ribbon sequence.
 *
 * Evaluates OmniCode: [ Sequence ] : [ Character` ] --> SEQUENCE [ Text ] = [ Text` ]
 *
 * Machine Effect:
 *   If sequence_target is non-NULL, appends the character glyph into the sequence
 *   destination buffer. If NULL, passes as a single atomic character cycle.
 *
 * @param c Pointer to the Character being sequenced.
 * @param sequence_target Pointer to destination buffer or container.
 * @return true if sequence yield succeeded; false on buffer overflow.
 */
bool omni_character_sequence(const omni_character_t *c, void *sequence_target);

/**
 * @brief Executes the complete 5-roll operational pipeline across declared turns:
 *   Roll 1 (Inscribe) -> Roll 2 (Transduce) -> Roll 3 (Guard !) ->
 *   Roll 4 (Project)   -> Roll 5 (Sequence).
 *
 * The Guard turn (Roll 3) enforces the adversative catch (!):
 *   If identity and boundary invariants hold, advances status to +1:PROCEED.
 *   If any discrepancy is detected, trips the wire and rolls back to -1:ROLLBACK.
 *
 * @param ctx Pointer to armed character execution context.
 * @return OMNI_STATE_PROCEED (+1) on equilibrium; OMNI_STATE_ROLLBACK (-1) on violation.
 */
omni_quad_state_t omni_character_execute(omni_character_context_t *ctx);

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK
 * ( Integrity Verification, Covenant Sealing & Quad-State Exit )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   OmniCode Block 4 reconciles all rolls, validates tripwire status, and seals
 *   the permanent commit status (+1:PROCEED, 0:AWAIT, NULL:HALT, -1:ROLLBACK).
 *   In hosted environments, +1:PROCEED maps to process exit code 0.
 * ============================================================================ */

/**
 * @brief Validates all invariants, verifies operational consensus, disarms the
 *        tripwire, and seals the final Character commit status.
 *
 * Evaluates OmniCode Block 4: Closing { ... }
 *
 * @param ctx Pointer to active character execution context.
 * @return OMNI_STATE_PROCEED (+1) on successful seal; OMNI_STATE_ROLLBACK (-1) if tripped.
 */
omni_quad_state_t omni_character_close(omni_character_context_t *ctx);

#ifdef __cplusplus
}
#endif

#endif /* OMNI_CHARACTER_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/container.h

H | 222 LINES | 1078 WORDS | 9.0 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT TYPE DEFINITION
 * ( Invariant Type: < Idea { Type } : Container > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Defines bounded memory envelopes, arrays, structures, and storage vessels
 *   in the OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ Dynamically Bounded Memory Envelope / Generic Slice ]
 *   - Ladder Ascension: Builds up with Numbers (/-^)
 *   - System: OmniCode { Lexicon }
 *   - Witness: Scripture 2 Kings 4:4 KJV
 *     "And when thou art come in, thou shalt shut the door upon thee and upon
 *      thy sons, and shalt pour out into all those vessels, and thou shalt set
 *      aside that which is full."
 *   - Semantics: { BoundedEnvelope :: StorageVessel :: MemoryCapsule :: ContainmentLaw }
 * ============================================================================ */

#ifndef OMNI_CONTAINER_H
#define OMNI_CONTAINER_H

#include "omni_base.h"

#ifdef __cplusplus
extern "C" {
#endif

/* ----------------------------------------------------------------------------
 * 1. INVARIANT CONSTANTS & CANONICAL EXEMPLARS
 * ---------------------------------------------------------------------------- */

/** Canonical Unit Exemplar: Single element partition (1 unit). */
#define OMNI_CONTAINER_EXEMPLAR_UNIT        ((size_t)1)

/** Canonical Vessel Exemplar: Standard 64-element storage vessel. */
#define OMNI_CONTAINER_EXEMPLAR_VESSEL      ((size_t)64)

/** Canonical Storehouse Exemplar: Large 4096-element memory storehouse. */
#define OMNI_CONTAINER_EXEMPLAR_STOREHOUSE  ((size_t)4096)

/* ----------------------------------------------------------------------------
 * 2. FREESTANDING INVARIANT TYPE STRUCTURE: omni_container_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief Invariant Type < Container >: The generic bounded memory envelope / slice.
 *
 * Provides memory safety by construction without requiring a heap or garbage collector:
 *   - Wraps a memory buffer of arbitrary element types (Character, Number, Site, State).
 *   - Strictly enforces the Core Containment Law: Occupancy <= Capacity.
 *
 * Core Invariant Law:
 *   ENCLOSE [ Storage ] WITHIN [ Capacity ]
 *   LAW { Containment } : [ { Occupancy } <= { Capacity } ]
 */
typedef struct {
    void   *buffer;                                 /**< Raw memory vessel holding discrete elements */
    size_t  element_size;                           /**< Footprint of a single element (sizeof(T) in bytes) */
    size_t  capacity;                               /**< Allocated maximum element capacity boundary */
    size_t  occupancy;                              /**< Current count of stored elements */
    bool    is_enclosed;                            /**< Invariant enclosure guarantee */
} omni_container_t;

/**
 * @brief Convenience static vessel initialized over a 64-byte storage partition.
 */
typedef struct {
    uint8_t          cells[OMNI_CONTAINER_EXEMPLAR_VESSEL]; /**< Concrete 64-byte static buffer */
    omni_container_t container;                             /**< Bounded slice wrapper */
} omni_byte_vessel_t;

/* ---------------------------------------------------------------------------- */
```

```
 * 3. 4-BLOCK PIPELINE CONTEXT: omni_container_context_t
 * ------------------------------------------------------------------------------ */

/**
 * @brief Execution context for Container encapsulation lifecycle rolls.
 */
typedef struct {
    const void            *payload_ptr;                /**< Imposed outside-in payload buffer */
    size_t                 payload_count;              /**< Count of incoming items to encapsulate */
    omni_container_t       vessel;                     /**< Concrete bounded vessel envelope */
    uintptr_t              target_destination;         /**< Destination memory address or anchor */
    omni_quad_state_t      status;                     /**< Active Quad-State truth */
    omni_tripwire_state_t  tripwire;                   /**< Subsystem safety interlock */
} omni_container_context_t;

/* ================================================================================
 * BLOCK 2: OPENING BLOCK
 * ( Outside-in Imposition & Context Initialization )
 * ================================================================================ */

/**
 * @brief Initializes a Container context over a provided memory storage partition.
 *
 * @param ctx Pointer to caller-allocated container context.
 * @param storage Memory buffer allocated by the caller (stack or static).
 * @param elem_size Footprint of a single element in bytes.
 * @param capacity Maximum number of elements storage can hold.
 * @param payload Outside-in memory payload to encapsulate.
 * @param payload_count Number of elements in the incoming payload.
 */
void omni_container_open(omni_container_context_t *ctx,
                         void *storage,
                         size_t elem_size,
                         size_t capacity,
                         const void *payload,
                         size_t payload_count);

/* ================================================================================
 * BLOCK 3: BODY BLOCK
 * ( Operational Primitives, Containment Laws & Inscriptions )
 * ================================================================================ */

/**
 * @brief Initializes a generic bounded container slice over an existing memory buffer.
 *
 * @param c Pointer to container struct to initialize.
 * @param buffer Pointer to underlying memory array.
 * @param element_size Byte size of each element.
 * @param capacity Maximum number of elements.
 */
void omni_container_init(omni_container_t *c, void *buffer, size_t element_size, size_t capacity);

/**
 * @brief Initializes a canonical 64-byte storage vessel.
 *
 * @param vessel Pointer to byte vessel struct.
 */
void omni_byte_vessel_init(omni_byte_vessel_t *vessel);

/**
 * @brief Core Containment Law Verification: Proves that Occupancy <= Capacity.
 *
 * @param c Pointer to container to evaluate.
 * @return true if Occupancy <= Capacity and properly enclosed; false otherwise.
 */
bool omni_container_verify_containment(const omni_container_t *c);

/**
 * @brief Queries whether the container has reached its maximum capacity limit.
 *
 * @param c Pointer to container.
 * @return true if Occupancy >= Capacity; false otherwise.
 */
bool omni_container_is_full(const omni_container_t *c);

/**
 * @brief Queries whether the container has zero stored elements.
 *
```

```
 * @param c Pointer to container.
 * @return true if Occupancy == 0; false otherwise.
 */
bool omni_container_is_empty(const omni_container_t *c);

/**
 * @brief Pushes an element into the container, advancing occupancy by 1.
 *
 * @param c Pointer to container.
 * @param element Pointer to element data to copy in.
 * @return OMNI_STATE_PROCEED on successful insertion; OMNI_STATE_ROLLBACK if full.
 */
omni_quad_state_t omni_container_push(omni_container_t *c, const void *element);

/**
 * @brief Retrieves a direct pointer to the element at the specified index.
 *
 * @param c Pointer to container.
 * @param index 0-based element index.
 * @return Pointer to element memory, or NULL if index >= occupancy.
 */
void *omni_container_get(const omni_container_t *c, size_t index);

/**
 * @brief Replaces or writes an element at the specified index.
 *
 * @param c Pointer to container.
 * @param index 0-based element index.
 * @param element Pointer to element data to copy in.
 * @return OMNI_STATE_PROCEED on success; OMNI_STATE_ROLLBACK if index >= capacity.
 */
omni_quad_state_t omni_container_set(omni_container_t *c, size_t index, const void *element);

/**
 * @brief Resets container occupancy to zero without clearing underlying memory.
 *
 * @param c Pointer to container.
 */
void omni_container_clear(omni_container_t *c);

/**
 * @brief Executes the full encapsulation lifecycle across declared rolls:
 *   - Roll 1: Build capsule envelope with numbers.
 *   - Roll 2: Encapsulate incoming payload.
 *   - Roll 3: Guard bounds under Law { Containment } (Occupancy <= Capacity).
 *   - Roll 4: Pour out and set aside vessel.
 *
 * @param ctx Pointer to active container context.
 * @return OMNI_STATE_PROCEED on equilibrium; OMNI_STATE_ROLLBACK on overflow.
 */
omni_quad_state_t omni_container_execute(omni_container_context_t *ctx);

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK
 * ( Integrity Verification & Commit Status )
 * ============================================================================ */

/**
 * @brief Validates all invariants and seals the Container commit status.
 *
 * @param ctx Pointer to active container context.
 * @return OMNI_STATE_PROCEED on successful seal; OMNI_STATE_ROLLBACK if tripped.
 */
omni_quad_state_t omni_container_close(omni_container_context_t *ctx);

#ifdef __cplusplus
}
#endif

#endif /* OMNI_CONTAINER_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/number.h

H | 385 LINES | 1953 WORDS | 15.1 KB

```h
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT TYPE DEFINITION
 * ( Invariant Type: < Idea { Type } : Number > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Defines scalar quantities, integers, and balanced ternary measurements in
 *   the OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Memory Word ] { 32 Balanced Trits / 64 Bits }
 *   - System: OmniCode { Lexicon }
 *   - Witness: Scripture Psalm 147:4 KJV
 *     "He telleth the number of the stars; he calleth them all by their names."
 *   - Semantics: { ScalarQuantity :: DualRailBitPlane :: ZeroSumEquilibrium }
 *
 * DUAL-RAIL BIT-PLANE SUBSTRATE:
 *   Every trit is physically encoded as a 2-bit pair (N, P) on binary silicon:
 *     - 00: Equilibrium / True Zero (Neither rail active)
 *     - 01: Positive Unity / +1 (Positive rail P active, bit 0)
 *     - 10: Negative Unity / -1 (Negative rail N active, bit 1)
 *     - 11: Collision / NULL Trap (Both rails active: Tripwire fault)
 *   A 32-trit Word maps bijectively onto a native 64-bit integer (uint64_t).
 *
 * DYNAMIC PACKING TOPOLOGIES:
 *   Byte bit-packing is dynamic and kinematic:
 *     - 2-2-2-2: Pure dense packing (4 trits per byte, 32 trits per 64-bit word)
 *     - 2-1-2-1-2: Symmetric resting equilibrium with carry/guard channels
 *     - 1-2-1-2-2: Prefix carry (Leftward tape advance carry propagation)
 *     - 2-2-1-2-1: Postfix carry (Rightward tape rewind borrow propagation)
 *     - 3-3-2, 3-2-3, 2-3-3: 3-space directional partitions (Z-axis magnitude)
 * ============================================================================ */

#ifndef OMNI_NUMBER_H
#define OMNI_NUMBER_H

#include "omni_base.h"
#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>

#ifdef __cplusplus
extern "C" {
#endif

/* ----------------------------------------------------------------------------
 * 1. INVARIANT CONSTANTS, EXEMPLARS & BIT-PLANE MASKS
 * ---------------------------------------------------------------------------- */

/** Number of balanced trits per machine word. */
#define OMNI_NUM_TRITS_PER_WORD     OMNI_TRITS_PER_WORD
#define OMNI_NUMBER_TRITS_PER_WORD  OMNI_TRITS_PER_WORD

/** Canonical Zero Exemplar: Perfect balance and null displacement. */
#define OMNI_EXEMPLAR_ZERO          ((int64_t)0)

/** Canonical Positive Unity Exemplar: Forward causal affirmation (+1). */
#define OMNI_EXEMPLAR_UNITY_POS     ((int64_t)1)

/** Canonical Negative Unity Exemplar: Backward causal restoration (-1). */
#define OMNI_EXEMPLAR_UNITY_NEG     ((int64_t)-1)

/** Canonical Memory Partition Exemplar: Standard 4096-unit allocation boundary. */
#define OMNI_EXEMPLAR_PARTITION     ((int64_t)4096)

/** Primary Hardware Register Destination: Arithmetic Accumulator. */
#define OMNI_TARGET_ACCUMULATOR     ((uintptr_t)0x00000000)

/** Dual-Rail Positive Bit-Plane Mask (All even bits: 0, 2, 4, ... 62). */
#define OMNI_BITPLANE_POS_MASK      (0x5555555555555555ULL)

/** Dual-Rail Negative Bit-Plane Mask (All odd bits: 1, 3, 5, ... 63). */
#define OMNI_BITPLANE_NEG_MASK      (0xAAAAAAAAAAAAAAAAULL)

```

```c
/** Dual-Rail 2-Bit Trit Constants. */
#define OMNI_TRIT_PAIR_ZERO         ((uint8_t)0x00) /* 00: Equilibrium (0) */
#define OMNI_TRIT_PAIR_POS          ((uint8_t)0x01) /* 01: Positive (+1)   */
#define OMNI_TRIT_PAIR_NEG          ((uint8_t)0x02) /* 10: Negative (-1)   */
#define OMNI_TRIT_PAIR_NULL         ((uint8_t)0x03) /* 11: Collision (NULL)*/

/* ---------------------------------------------------------------------------
 * 2. DYNAMIC PACKING TOPOLOGY ENUM
 * --------------------------------------------------------------------------- */

/**
 * @brief Dynamic bit-packing topologies for bytes and 64-bit words.
 */
typedef enum {
    OMNI_PACK_2_2_2_2   = 0, /**< Pure dense packing (4 trits / byte) */
    OMNI_PACK_2_1_2_1_2 = 1, /**< Palindromic symmetric resting equilibrium */
    OMNI_PACK_1_2_1_2_2 = 2, /**< Prefix carry: Leftward tape advance */
    OMNI_PACK_2_2_1_2_1 = 3, /**< Postfix carry: Rightward tape rewind */
    OMNI_PACK_3_3_2     = 4, /**< 3-space directional partition (3-3-2) */
    OMNI_PACK_3_2_3     = 5, /**< 3-space directional partition (3-2-3) */
    OMNI_PACK_2_3_3     = 6  /**< 3-space directional partition (2-3-3) */
} omni_pack_mode_t;

/* ---------------------------------------------------------------------------
 * 3. FREESTANDING INVARIANT TYPE STRUCTURE: omni_number_t
 * --------------------------------------------------------------------------- */

/**
 * @brief Invariant Type < Number >: The scalar quantity and measurement metric.
 *
 * Encloses the dual substrate:
 *   1. Physical Silicon Substrate: 64-Bit Signed Integer Scalar (int64_t).
 *   2. Dual-Rail Packed Substrate: Native 64-bit packed bit-planes (N, P).
 *   3. Virtual Balanced Trit Substrate: 32-Trit Machine Word array.
 *
 * Core Invariant Law:
 *   ENCLOSE [ Number ] WITHIN [ 1 Memory Word ] (Strict 32-trit capacity).
 *   LAW { Equilibrium } : [ (+1) plus (-1) <=> 0 ] (Foundational balance).
 */
typedef struct {
    int64_t     scalar;                             /**< 64-bit scalar integer */
    uint64_t    packed_trits;                       /**< Dual-rail bit-planes (N, P) */
    omni_trit_t trits[OMNI_NUM_TRITS_PER_WORD];     /**< Unpacked 32 balanced trits */
    bool        is_enclosed;                        /**< Boundary enclosure seal */
} omni_number_t;

/* ---------------------------------------------------------------------------
 * 4. PURE INVARIANT OPERATIONS, BIT-PLANE & DUAL-RAIL LAWS
 * --------------------------------------------------------------------------- */

/**
 * @brief Constructs a Number directly from a 64-bit scalar integer,
 *        automatically transducing it into dual-rail packed bit-planes
 *        and 32 balanced trits.
 *
 * @param scalar The 64-bit integer quantity to measure.
 * @return Fully initialized omni_number_t with is_enclosed = true.
 */
omni_number_t omni_number_from_scalar(int64_t scalar);

/**
 * @brief Bijective Radix-3 Transduction: Maps a 64-bit scalar to 32 balanced trits.
 *
 * @param val Input 64-bit scalar quantity.
 * @param out_trits Array of 32 balanced trits receiving the transduction.
 */
void omni_number_transduce(int64_t val, omni_trit_t out_trits[OMNI_NUM_TRITS_PER_WORD]);

/**
 * @brief Inverse Transduction: Reconstructs a 64-bit scalar from 32 balanced trits.
 *
 * @param trits Array of 32 balanced trits.
 * @return Reconstructed 64-bit signed integer value.
 */
int64_t omni_number_to_scalar(const omni_trit_t trits[OMNI_NUM_TRITS_PER_WORD]);

/**
 * @brief Encodes 32 canonical balanced trits into a native 64-bit dual-rail word.
```

```
 *
 * @param trits Array of 32 balanced trits {-1, 0, +1}.
 * @return Packed 64-bit dual-rail word (N, P bit-planes).
 */
uint64_t omni_number_pack_trits(const omni_trit_t trits[OMNI_NUM_TRITS_PER_WORD]);

/**
 * @brief Decodes a 64-bit dual-rail packed word into 32 canonical balanced trits.
 *
 * @param packed Packed 64-bit dual-rail word.
 * @param out_trits Destination array of 32 balanced trits.
 */
void omni_number_unpack_trits(uint64_t packed, omni_trit_t out_trits[OMNI_NUM_TRITS_PER_WORD]);

/**
 * @brief Instant O(1) Bit-Plane Negation: Swaps the positive and negative
 *        bit-planes across all 32 trits simultaneously with zero carry chains.
 *
 * @param packed Packed 64-bit dual-rail word.
 * @return Negated 64-bit dual-rail word (+N <-> -N).
 */
uint64_t omni_number_trits_negate(uint64_t packed);

/**
 * @brief Instant O(1) Collision Detection: Verifies that no trit in the word
 *        holds the illegal collision state (11: NULL).
 *
 * @param packed Packed 64-bit dual-rail word.
 * @return true if any trit is in collision (11: NULL); false if clean.
 */
bool omni_number_trits_has_null(uint64_t packed);

/**
 * @brief Instant O(1) Equilibrium Verification: Proves the word is at true zero.
 *
 * @param packed Packed 64-bit dual-rail word.
 * @return true if all trits are in resting zero equilibrium (00); false otherwise.
 */
bool omni_number_trits_is_equilibrium(uint64_t packed);

/**
 * @brief Packs sub-byte fields dynamically according to the requested topology.
 *
 * @param f1 First field value.
 * @param f2 Second field value.
 * @param f3 Third field value.
 * @param f4 Fourth field value (used in 2-2-2-2 and 5-segment modes).
 * @param f5 Fifth field value (used in 2-1-2-1-2, 1-2-1-2-2, 2-2-1-2-1).
 * @param mode Dynamic packing topology enum.
 * @return Packed 8-bit byte.
 */
uint8_t omni_number_pack_byte(uint8_t f1, uint8_t f2, uint8_t f3,
                              uint8_t f4, uint8_t f5, omni_pack_mode_t mode);

/**
 * @brief Unpacks an 8-bit byte into sub-byte fields based on topology.
 *
 * @param byte_val Input 8-bit byte.
 * @param out_fields Destination array of 5 field values.
 * @param mode Dynamic packing topology enum.
 */
void omni_number_unpack_byte(uint8_t byte_val, uint8_t out_fields[5],
                             omni_pack_mode_t mode);

/**
 * @brief Verifies the ALU Core Arithmetic Equilibrium Law: (+1) plus (-1) <=> 0.
 *
 * @return true if zero-sum balance holds; false otherwise.
 */
bool omni_number_verify_equilibrium(void);

/**
 * @brief Adds two Numbers producing a new balanced Number word.
 *
 * @param a First number summand.
 * @param b Second number summand.
 * @return Resulting sum as an invariant omni_number_t.
 */
```

```
omni_number_t omni_number_add(omni_number_t a, omni_number_t b);

/**
 * @brief Subtracts two Numbers via exact balanced ternary negation: a + (-b).
 *
 * @param a Minuend number.
 * @param b Subtrahend number to negate and add.
 * @return Resulting difference as an invariant omni_number_t.
 */
omni_number_t omni_number_sub(omni_number_t a, omni_number_t b);

/**
 * @brief Negates a Number producing its exact symmetric polar opposite.
 *
 * @param a Number to negate.
 * @return Negated number with swapped bit-planes.
 */
omni_number_t omni_number_neg(omni_number_t a);

/**
 * @brief Evaluates relational equality between two numbers.
 *
 * @param a First number operand.
 * @param b Second number operand.
 * @return true if scalar values match and both are enclosed; false otherwise.
 */
bool omni_number_eq(const omni_number_t *a, const omni_number_t *b);

/**
 * @brief Verifies that a number is properly enclosed within single-word bounds
 *        and free of NULL collision traps.
 *
 * @param n Pointer to number to verify.
 * @return true if valid, enclosed, and trap-free; false otherwise.
 */
bool omni_number_verify_bounds(const omni_number_t *n);

/* ============================================================================
 * BLOCK 2: OPENING BLOCK
 * ( Outside-In Imposition, Accumulator Binding & Preconditions )
 * ============================================================================ */

/**
 * @brief Execution context for Number measurement and ALU lifecycle rolls.
 */
typedef struct {
    int64_t               input_quantity;     /**< Outside-in scalar payload */
    omni_number_t         number;             /**< Concrete single-word allocation */
    uintptr_t             target_accumulator; /**< Target accumulator destination */
    void                 *sequence_target;    /**< Destination for Vector sequence */
    omni_quad_state_t     status;             /**< Active Quad-State truth */
    omni_tripwire_state_t tripwire;           /**< Subsystem safety interlock */
} omni_number_context_t;

/**
 * @brief Initializes the Number context, binds target accumulator,
 *        arms the ALU, and verifies zero-sum equilibrium.
 *
 * Evaluates OmniCode Block 2: Opening { ... }
 *
 * @param ctx Pointer to caller-allocated number context.
 * @param input_quantity Outside-in scalar quantity (defaults to 4096 if 0).
 * @param target_accumulator Target accumulator register (defaults to 0x00).
 */
void omni_number_open(omni_number_context_t *ctx, int64_t input_quantity,
                      uintptr_t target_accumulator);

/* ============================================================================
 * BLOCK 3: BODY BLOCK
 * ( Operational Rolls, Guard Interlocks & Vector Sequencing )
 * ============================================================================ */

/**
 * @brief Roll 1: Inscribes outside-in quantity into the word container.
 *
 * Evaluates OmniCode: [ Inscribe ] : { Quantity ] --> INSCRIBE [ Word ] = [ Number` ]
 *
 * @param ctx Active execution context.
```

```
 * @param quantity The scalar quantity being inscribed.
 */
void omni_number_inscribe(omni_number_context_t *ctx, int64_t quantity);

/**
 * @brief Roll 2: Transduces Number` through the balanced ternary ALU.
 *
 * Evaluates OmniCode: [ Transduce ] : [ Number` { 64 Bits } ] <=> TRANSDUCE [ Word` { 32 Trits } ]
 *
 * @param ctx Active execution context.
 */
void omni_number_transduce_context(omni_number_context_t *ctx);

/**
 * @brief Roll 3: Evaluates adversative guard assertion (!).
 *
 * Evaluates OmniCode: [ Compare ] : { Number` = Quantity } & { Number` <= [ 1 Word ] }
 *                     ! GUARD --> [ Status` = +1:PROCEED ]
 *
 * @param ctx Active execution context.
 * @return OMNI_STATE_PROCEED on equilibrium; OMNI_STATE_ROLLBACK on violation.
 */
omni_quad_state_t omni_number_guard(omni_number_context_t *ctx);

/**
 * @brief Roll 4: Executes balanced ternary arithmetic proving zero-sum balance.
 *
 * Evaluates OmniCode: [ Calculate ] : [ ALU ] :
 *                     [ (+1) plus (-1) ] <=> CALCULATE [ 0 ] = [ Equilibrium` ]
 *
 * @param ctx Active execution context.
 * @return OMNI_STATE_PROCEED on confirmed balance; OMNI_STATE_ROLLBACK on disequilibrium.
 */
omni_quad_state_t omni_number_calculate(omni_number_context_t *ctx);

/**
 * @brief Roll 5: Yields Number` to continuous Vector coordinate sequence.
 *
 * Evaluates OmniCode: [ Sequence ] : [ Number` } --> SEQUENCE [ Vector ] = [ Vector` ]
 *
 * @param n Pointer to the Number being sequenced.
 * @param sequence_target Pointer to destination vector container.
 * @return true if sequence yield succeeded; false on overflow.
 */
bool omni_number_sequence(const omni_number_t *n, void *sequence_target);

/**
 * @brief Executes the complete 5-roll operational pipeline across declared turns:
 *   Roll 1 (Inscribe) -> Roll 2 (Transduce) -> Roll 3 (Guard !) ->
 *   Roll 4 (Calculate) -> Roll 5 (Sequence).
 *
 * @param ctx Pointer to active number context.
 * @return OMNI_STATE_PROCEED on equilibrium; OMNI_STATE_ROLLBACK on violation.
 */
omni_quad_state_t omni_number_execute(omni_number_context_t *ctx);

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK
 * ( Integrity Verification & Commit Status )
 * ============================================================================ */

/**
 * @brief Validates all invariants, verifies consensus, disarms tripwire,
 *        and seals the final Number commit status (+1:PROCEED).
 *
 * Evaluates OmniCode Block 4: Closing { ... }
 *
 * @param ctx Pointer to active number context.
 * @return OMNI_STATE_PROCEED on successful seal; OMNI_STATE_ROLLBACK if tripped.
 */
omni_quad_state_t omni_number_close(omni_number_context_t *ctx);

#ifdef __cplusplus
}
#endif

#endif /* OMNI_NUMBER_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/omni.h

H | 27 LINES | 136 WORDS | 1.1 KB

```c
/* ============================================================================
 * OMNICODE FREESTANDING STANDARD LIBRARY (C MASTER HEADER)
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Umbrella header exposing all primitive invariant types and operational
 *   primitives for OmniCode bare-metal and hosted runtimes.
 *   - Category: Standard Library Track 2 C Transpile Variant
 *   - Invariant Types Included:
 *       < Character > : Atomic Glyph / Letter Rune (1 Cell / 5 Trits)
 *       < Number >    : Scalar Quantity / Measure (1 Word / 32 Trits)
 *       < Container > : Generic Bounded Space Envelope / Slice
 *       < Site >      : Silicon Coordinate / Address (RAM @ vs MMIO ::@)
 *       < State >     : Quad-State Causal Truth & Loop Stepper
 * ============================================================================ */

#ifndef OMNI_H
#define OMNI_H

#include "omni_base.h"
#include "character.h"
#include "number.h"
#include "container.h"
#include "site.h"
#include "state.h"

#endif /* OMNI_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/omni_base.h

H | 289 LINES | 1270 WORDS | 10.6 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT AXIOMS BLOCK
 * ( Invariant Root: < Lexicon { Types of Ideas } > & [ Noun { Base Axioms } ] )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Foundational bare-metal substrates, balanced ternary types, and Quad-State
 *   truth logic governing the OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Environment: Freestanding C99/C11 (Zero libc/OS dependency)
 *   - Word Size: 64-Bit Silicon Scalar / 32 Balanced Trits
 *   - Cell Size: 8-Bit Byte / 5 Balanced Trits
 *   - System: OmniCode { Lexicon }
 *   - Witness: Scripture Proverbs 11:1 KJV
 *     "A false balance is abomination to the LORD: but a just weight is his delight."
 *   - Semantics: { BalancedTernary :: QuadStateTruth :: SiliconAnchors :: TripwireGuards }
 * ============================================================================ */

#ifndef OMNI_BASE_H
#define OMNI_BASE_H

#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>

#ifdef __cplusplus
extern "C" {
#endif

/* ----------------------------------------------------------------------------
 * ARCHITECTURAL CONSTANTS & SUBSTRATE CAPACITIES
 * ---------------------------------------------------------------------------- */

/** Number of balanced trits packed into a single 8-bit memory Cell (3^5 = 243 states). */
#define OMNI_TRITS_PER_CELL         5

/** Number of balanced trits packed into a single 64-bit machine Word. */
#define OMNI_TRITS_PER_WORD         32

/** Footprint of the standard bounded Container vessel exemplar (in bytes). */
#define OMNI_VESSEL_DEFAULT_CELLS   64

/* ----------------------------------------------------------------------------
 * 1. BALANCED TERNARY SUBSTRATE: omni_trit_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief Discrete balanced ternary digit (-1, 0, +1).
 *
 * Unlike binary (0, 1), balanced ternary is inherently symmetrical around zero:
 *   - Negative (-1) : Backward causation, negation, deficit, or restoration.
 *   - Zero     ( 0) : Equilibrium, standby, neutral, or unperturbed balance.
 *   - Positive (+1) : Forward causation, affirmative assertion, or surplus.
 */
typedef int8_t omni_trit_t;

#define OMNI_TRIT_NEG   ((omni_trit_t)-1)
#define OMNI_TRIT_ZERO  ((omni_trit_t) 0)
#define OMNI_TRIT_POS   ((omni_trit_t) 1)

/* ----------------------------------------------------------------------------
 * 2. QUAD-STATE CAUSAL TRUTH: omni_quad_state_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief The fourfold causal truth states of computation in OmniCode.
 *
 * Unifies boolean logic, error handling, scheduling, and processor clock detents:
 *   - OMNI_STATE_PROCEED  (+1) : Forward Causation (+t). Operation completed in equilibrium.
 *   - OMNI_STATE_AWAIT    ( 0) : Equilibrium Standby (t=t). Pending asynchronous resolution.
 *   - OMNI_STATE_ROLLBACK (-1) : Retroactive Reversal (-t). Guard tripped; unwind causal drift.
 *   - OMNI_STATE_HALT   (-128) : Terminal Detent (t=empty). Clock detent; permanent halt (HLT).
 */
typedef enum {
    OMNI_STATE_ROLLBACK = -1,   /**< -1 : Retroactive Reversal / Error / Unwind */
```

```
    OMNI_STATE_AWAIT    =  0,   /**<  0 : Equilibrium Standby / Hold Pending */
    OMNI_STATE_PROCEED  =  1,   /**< +1 : Forward Causation / Affirmative Advancement */
    OMNI_STATE_HALT     = -128  /**< NULL : Terminal Detent / Stop Processor Clock */
} omni_quad_state_t;

/**
 * @brief Tertiary execution status subset for non-terminal roll checkpoints.
 */
typedef enum {
    OMNI_STATUS_ROLLBACK = -1,
    OMNI_STATUS_AWAIT    =  0,
    OMNI_STATUS_PROCEED  =  1
} omni_status_t;

/* ---------------------------------------------------------------------------
 * 3. HARDWARE COGNITIVE SITES (EXEMPLARS)
 * --------------------------------------------------------------------------- */

/** Unmapped null pointer origin. */
#define OMNI_SITE_NULL          ((uintptr_t)0x00000000)

/** Real-mode Master Boot Record physical origin. */
#define OMNI_SITE_MBR_BOOT      ((uintptr_t)0x00007C00)

/** Hardware UART serial port base (COM1) for kernel telemetry. */
#define OMNI_SITE_COM1          ((uintptr_t)0x000003F8)

/** Physical memory-mapped VGA text-mode display buffer (80x25 characters). */
#define OMNI_SITE_VGA_BUFFER    ((uintptr_t)0x000B8000)

/* ---------------------------------------------------------------------------
 * 4. TRIPWIRE & INTEGRITY DETENTS: omni_tripwire_state_t
 * --------------------------------------------------------------------------- */

/**
 * @brief Safety interlock state preventing silent memory corruption or causal leakage.
 */
typedef enum {
    OMNI_TRIPWIRE_DISARMED = 0, /**< Interlock inactive; safe quiescent state */
    OMNI_TRIPWIRE_ARMED    = 1, /**< Active guard monitoring execution bounds */
    OMNI_TRIPWIRE_TRIPPED  = 2  /**< Violation intercepted; execution blocked */
} omni_tripwire_state_t;

/* ===========================================================================
 * BLOCK 2: OPENING BLOCK
 * ( Subsystem Preconditions, Verification & Initializers )
 * =========================================================================== */

/**
 * @brief Verifies that the host hardware meets OmniCode's bare-metal preconditions:
 *   1. Standard 64-bit pointer architecture (sizeof(uintptr_t) == 8).
 *   2. Standard 8-bit octet cell substrate (sizeof(uint8_t) == 1).
 *   3. Zero-sum arithmetic equilibrium ((+1) + (-1) == 0).
 *
 * @return OMNI_STATE_PROCEED if all preconditions hold; OMNI_STATE_ROLLBACK otherwise.
 */
static inline omni_quad_state_t omni_base_verify_preconditions(void) {
    if (sizeof(uintptr_t) != 8) return OMNI_STATE_ROLLBACK;
    if (sizeof(uint8_t)   != 1) return OMNI_STATE_ROLLBACK;

    /* Prove Proverbs 11:1 Law { Equilibrium }: (+1) plus (-1) <=> 0 */
    omni_trit_t pos = OMNI_TRIT_POS;
    omni_trit_t neg = OMNI_TRIT_NEG;
    if ((pos + neg) != OMNI_TRIT_ZERO) return OMNI_STATE_ROLLBACK;

    return OMNI_STATE_PROCEED;
}

/**
 * @brief Arms a tripwire interlock before entering an untrusted or dynamic execution block.
 */
static inline void omni_tripwire_arm(omni_tripwire_state_t *tw) {
    if (tw) *tw = OMNI_TRIPWIRE_ARMED;
}

/**
 * @brief Disarms a tripwire upon successful verification and closing commit.
 */
```

```c
static inline void omni_tripwire_disarm(omni_tripwire_state_t *tw) {
    if (tw) *tw = OMNI_TRIPWIRE_DISARMED;
}

/**
 * @brief Immediately trips the interlock, halting forward causation.
 */
static inline void omni_tripwire_trip(omni_tripwire_state_t *tw) {
    if (tw) *tw = OMNI_TRIPWIRE_TRIPPED;
}

/* ============================================================================
 * BLOCK 3: BODY BLOCK
 * ( Balanced Ternary Algebra, Quad-State Logic & Guard Inlines )
 * ============================================================================ */

/**
 * @brief Negates a balanced trit along the causal vector (-t <=> +t).
 */
static inline omni_trit_t omni_trit_negate(omni_trit_t t) {
    return (omni_trit_t)(-t);
}

/**
 * @brief Multiplies two balanced trits (commutative zero-sum product).
 */
static inline omni_trit_t omni_trit_multiply(omni_trit_t a, omni_trit_t b) {
    return (omni_trit_t)(a * b);
}

/**
 * @brief Full 1-trit balanced ternary adder with carry output.
 *
 * Truth Table:
 *   a   b  cin | sum carry
 *  -1  -1   -1 |   0    -1
 *  -1  -1    0 |  +1    -1
 *  -1  -1   +1 |  -1     0
 *  -1   0    0 |  -1     0
 *  -1  +1    0 |   0     0  (Equilibrium)
 *  +1  +1    0 |  -1    +1
 *  +1  +1   +1 |   0    +1
 */
static inline omni_trit_t omni_trit_add(omni_trit_t a, omni_trit_t b, omni_trit_t cin, omni_trit_t *cout) {
    int32_t sum = (int32_t)a + (int32_t)b + (int32_t)cin;
    omni_trit_t carry = 0;

    if (sum > 1) {
        sum -= 3;
        carry = 1;
    } else if (sum < -1) {
        sum += 3;
        carry = -1;
    }

    if (cout) *cout = carry;
    return (omni_trit_t)sum;
}

/**
 * @brief Evaluates an invariant guard condition.
 *
 * If condition evaluates to true:
 *   Status is confirmed in PROCEED (+1).
 * If condition evaluates to false:
 *   Tripwire is tripped and status transitions to ROLLBACK (-1).
 */
static inline omni_quad_state_t omni_guard_assert(bool condition,
                                                  omni_tripwire_state_t *tw,
                                                  omni_quad_state_t *status) {
    if (condition) {
        if (status) *status = OMNI_STATE_PROCEED;
        return OMNI_STATE_PROCEED;
    } else {
        if (tw) *tw = OMNI_TRIPWIRE_TRIPPED;
        if (status) *status = OMNI_STATE_ROLLBACK;
        return OMNI_STATE_ROLLBACK;
    }
```

```
}

/** Predicate queries for Quad-State truth */
static inline bool omni_is_proceed(omni_quad_state_t s) { return (s == OMNI_STATE_PROCEED); }
static inline bool omni_is_await(omni_quad_state_t s)   { return (s == OMNI_STATE_AWAIT); }
static inline bool omni_is_rollback(omni_quad_state_t s){ return (s == OMNI_STATE_ROLLBACK); }
static inline bool omni_is_halt(omni_quad_state_t s)    { return (s == OMNI_STATE_HALT); }

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK
 * ( Integrity Seals, Detents & Commit Status Functions )
 * ============================================================================ */

/**
 * @brief Validates the integrity of an execution context before sealing the commit status.
 *
 * Verifies that:
 *   1. No tripwire was tripped during execution.
 *   2. Machine status is not trapped in an unhandled rollback.
 *
 * @return OMNI_STATE_PROCEED if context is clean; OMNI_STATE_ROLLBACK on failure.
 */
static inline omni_quad_state_t omni_base_verify_integrity(omni_tripwire_state_t tw,
                                                           omni_quad_state_t status) {
    if (tw == OMNI_TRIPWIRE_TRIPPED || status == OMNI_STATE_ROLLBACK) {
        return OMNI_STATE_ROLLBACK;
    }
    return OMNI_STATE_PROCEED;
}

/**
 * @brief Closes and seals an execution block in PROCEED (+1).
 */
static inline omni_quad_state_t omni_commit_proceed(omni_tripwire_state_t *tw) {
    omni_tripwire_disarm(tw);
    return OMNI_STATE_PROCEED;
}

/**
 * @brief Closes and seals an execution block in ROLLBACK (-1).
 */
static inline omni_quad_state_t omni_commit_rollback(omni_tripwire_state_t *tw) {
    omni_tripwire_trip(tw);
    return OMNI_STATE_ROLLBACK;
}

/**
 * @brief Closes and seals an execution block in terminal detent HALT:NULL.
 */
static inline omni_quad_state_t omni_commit_halt(omni_tripwire_state_t *tw) {
    omni_tripwire_disarm(tw);
    return OMNI_STATE_HALT;
}

#ifdef __cplusplus
}
#endif

#endif /* OMNI_BASE_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/site.h

H | 191 LINES | 925 WORDS | 7.4 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT TYPE DEFINITION
 * ( Invariant Type: < Idea { Type } : Site > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Defines physical memory addresses, hardware ports, and MMIO coordinates in the
 *   OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Address ] { 64-Bit Pointer / Physical Silicon Coordinate }
 *   - Ladder Ascension: Builds up with Numbers (/-^)
 *   - System: OmniCode { Lexicon }
 *   - Witness: Scripture Genesis 28:16 KJV
 *     "And Jacob awaked out of his sleep, and he said, Surely the Lord is in
 *      this place; and I knew it not."
 *   - Semantics: { PhysicalAddress :: MMIOPort :: SiliconCoordinate :: RoutingLaw }
 * ============================================================================ */

#ifndef OMNI_SITE_H
#define OMNI_SITE_H

#include "omni_base.h"

#ifdef __cplusplus
extern "C" {
#endif

/* ----------------------------------------------------------------------------
 * 1. HARDWARE ROUTING DOMAIN ENUM: omni_site_routing_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief Hardware Routing Law domain classification.
 *
 * Law { Routing }:
 *   - RAM Physical (@)   : Standard cacheable RAM memory pointer.
 *   - MMIO Hardware (::@) : Memory-mapped I/O port requiring volatile access.
 */
typedef enum {
    OMNI_ROUTING_RAM_PHYSICAL  = 0, /**< '@' Physical system RAM address */
    OMNI_ROUTING_HARDWARE_MMIO = 1  /**< '::@' Hardware MMIO device register or port */
} omni_site_routing_t;

/* ----------------------------------------------------------------------------
 * 2. FREESTANDING INVARIANT TYPE STRUCTURE: omni_site_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief Invariant Type < Site >: The physical silicon address coordinate.
 *
 * Encloses the 64-bit silicon address pointer and its hardware routing domain:
 *   - Strictly non-null when valid (0x0 is mapped to unmapped detent).
 *   - Resolves routing bijectively between RAM and device hardware ports.
 *
 * Core Invariant Law:
 *   ENCLOSE [ Site ] WITHIN [ 1 Address ]
 *   LAW { Routing } : [ Physical RAM @ ] | [ Hardware MMIO ::@ ]
 */
typedef struct {
    uintptr_t           coordinate;                  /**< Physical 64-bit silicon address pointer */
    omni_site_routing_t routing;                     /**< Hardware routing classification (RAM vs MMIO) */
    bool                is_enclosed;                 /**< Invariant boundary enclosure guarantee */
} omni_site_t;

/* ----------------------------------------------------------------------------
 * 3. 4-BLOCK PIPELINE CONTEXT: omni_site_context_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief Execution context for Site address binding and routing lifecycle rolls.
 */
typedef struct {
    uintptr_t             input_coordinate;          /**< Outside-in target silicon coordinate */
```

```
    omni_site_t           site;                         /**< Concrete single-address allocation */
    omni_quad_state_t     status;                       /**< Active Quad-State truth */
    omni_tripwire_state_t tripwire;                     /**< Subsystem safety interlock */
} omni_site_context_t;

/* ============================================================================
 * BLOCK 2: OPENING BLOCK
 * ( Outside-in Imposition & Context Initialization )
 * ============================================================================ */

/**
 * @brief Initializes the Site context and binds the target coordinate.
 *
 * @param ctx Pointer to caller-allocated site context.
 * @param input_coord Target silicon address (defaults to VGA buffer if 0).
 */
void omni_site_open(omni_site_context_t *ctx, uintptr_t input_coord);

/* ============================================================================
 * BLOCK 3: BODY BLOCK
 * ( Operational Primitives, Routing Laws & Memory Access )
 * ============================================================================ */

/**
 * @brief Constructs a Site directly from a 64-bit address pointer,
 *        automatically resolving its hardware routing classification.
 *
 * @param address Physical silicon memory or MMIO address.
 * @return Fully initialized omni_site_t.
 */
omni_site_t omni_site_from_address(uintptr_t address);

/**
 * @brief Hardware Routing Law: Distinguishes physical RAM pointers from
 *        hardware MMIO registers and serial ports.
 *
 * @param address Physical target address.
 * @return OMNI_ROUTING_HARDWARE_MMIO if within device port ranges; RAM otherwise.
 */
omni_site_routing_t omni_site_resolve_routing(uintptr_t address);

/**
 * @brief Calculates an offset Site from a base Site coordinate.
 *
 * @param site Base reference site pointer.
 * @param offset Signed byte displacement (+offset or -offset).
 * @return New displaced omni_site_t coordinate.
 */
omni_site_t omni_site_offset(const omni_site_t *site, intptr_t offset);

/**
 * @brief Verifies whether a site coordinate is valid and non-null.
 *
 * @param site Pointer to site to evaluate.
 * @return true if site is valid, enclosed, and non-null; false otherwise.
 */
bool omni_site_is_valid(const omni_site_t *site);

/**
 * @brief Evaluates relational equality between two sites.
 *
 * @param a First site operand.
 * @param b Second site operand.
 * @return true if addresses and routing domains match; false otherwise.
 */
bool omni_site_eq(const omni_site_t *a, const omni_site_t *b);

/**
 * @brief Safely reads an 8-bit byte from the site coordinate plus offset.
 *        Employs volatile memory semantics to prevent compiler cache elision on MMIO.
 *
 * @param site Base site coordinate.
 * @param offset Byte displacement.
 * @return Read byte value (returns 0 if site is invalid/null).
 */
uint8_t omni_site_read_u8(const omni_site_t *site, size_t offset);

/**
```

```
 * @brief Safely writes an 8-bit byte to the site coordinate plus offset.
 *        Employs volatile memory semantics for bare-metal hardware registers.
 *
 * @param site Base site coordinate.
 * @param offset Byte displacement.
 * @param value Byte value to inscribe.
 */
void omni_site_write_u8(const omni_site_t *site, size_t offset, uint8_t value);

/**
 * @brief Executes the full location lifecycle across declared rolls:
 *   - Roll 1: Locate address with numbers.
 *   - Roll 2: Bind coordinate and classify routing.
 *   - Roll 3: Guard non-null identity invariants.
 *   - Roll 4: Route and anchor memory target.
 *
 * @param ctx Pointer to active site context.
 * @return OMNI_STATE_PROCEED on valid address; OMNI_STATE_ROLLBACK on null or error.
 */
omni_quad_state_t omni_site_execute(omni_site_context_t *ctx);

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK
 * ( Integrity Verification & Commit Status )
 * ============================================================================ */

/**
 * @brief Validates all invariants and seals the Site commit status.
 *
 * @param ctx Pointer to active site context.
 * @return OMNI_STATE_PROCEED on successful seal; OMNI_STATE_ROLLBACK if tripped.
 */
omni_quad_state_t omni_site_close(omni_site_context_t *ctx);

#ifdef __cplusplus
}
#endif

#endif /* OMNI_SITE_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/include/state.h

H | 170 LINES | 863 WORDS | 7.1 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT TYPE DEFINITION
 * ( Invariant Type: < Idea { Type } : State > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Defines causal truth, clock progression, and the canonical Quad-State machine
 *   in the OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Trit ] { 2 Bits / Quad-State Causal Register }
 *   - Named Registers: [ STATE, TURN, BOUND, IT ]
 *   - System: OmniCode { Lexicon }
 *   - Witness: Scripture Ecclesiastes 3:1 KJV
 *     "To every thing there is a season, and a time to every purpose under
 *      the heaven:"
 *   - Semantics: { QuadStateTruth :: CausalClock :: CanonicalLoopLaw }
 * ============================================================================ */

#ifndef OMNI_STATE_H
#define OMNI_STATE_H

#include "omni_base.h"

#ifdef __cplusplus
extern "C" {
#endif

/* ----------------------------------------------------------------------------
 * 1. CANONICAL NAMED REGISTERS: omni_state_registers_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief The four canonical named registers governing execution kinematics.
 *
 * Named Registers:
 *   - STATE : Primary causal truth register (+1: PROCEED, 0: AWAIT, -1: ROLLBACK, HALT: NULL).
 *   - TURN  : Discrete cadence pulse counter (increments on affirmative causation).
 *   - BOUND : Dynamic loop termination threshold.
 *   - IT    : Active accumulator iterator stepping along the causal tape.
 */
typedef struct {
    omni_quad_state_t state;                        /**< STATE: Primary Quad-State truth register */
    int64_t           turn;                         /**< TURN: Cadence step counter */
    int64_t           bound;                        /**< BOUND: Loop termination boundary */
    int64_t           it;                           /**< IT: Active accumulator iterator */
} omni_state_registers_t;

/* ----------------------------------------------------------------------------
 * 2. FREESTANDING INVARIANT TYPE STRUCTURE: omni_state_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief Invariant Type < State >: The causal truth and control flow engine.
 *
 * Encloses the physical trit substrate and register bank:
 *   - Single-trit physical substrate (2 bits in silicon).
 *   - Implements the Canonical Looping Law connecting Quad-State to hardware registers.
 *
 * Core Invariant Law:
 *   LAW { Quad-State } : { T4 = [ +1:PROCEED | 0:AWAIT | -1:ROLLBACK | HALT:NULL ] }
 *   LAW { Loop } : LOOP [ WHEN [ { IT } < { BOUND } ] --> [ IT` ] PROCEED to { TURN }
 *                         ! [ { IT } > { BOUND } ] AWAIT for [ IT` ]
 *                         :: HALT WHEN [ IT` ] = NULL ]
 */
typedef struct {
    omni_trit_t             substrate_trit;         /**< 1 Trit physical substrate */
    omni_state_registers_t  registers;              /**< Canonical Quad-State register bank */
    bool                    is_enclosed;            /**< Invariant boundary enclosure guarantee */
} omni_state_t;

/* ----------------------------------------------------------------------------
 * 3. 4-BLOCK PIPELINE CONTEXT: omni_state_context_t
 * ---------------------------------------------------------------------------- */
```

```c

/**
 * @brief Execution context for State governance and loop lifecycle rolls.
 */
typedef struct {
    omni_quad_state_t     input_signal;                /**< Outside-in causal truth signal */
    omni_state_t          state_engine;                /**< Concrete single-trit register bank */
    omni_quad_state_t     status;                      /**< Active Quad-State truth */
    omni_tripwire_state_t tripwire;                    /**< Subsystem safety interlock */
} omni_state_context_t;

/* ============================================================================
 * BLOCK 2: OPENING BLOCK
 * ( Outside-in Imposition & Context Initialization )
 * ============================================================================ */

/**
 * @brief Initializes the State context with an input signal and loop boundary.
 *
 * @param ctx Pointer to caller-allocated state context.
 * @param signal Initial truth signal (defaults to PROCEED if 0).
 * @param bound_limit Maximum loop iteration threshold.
 */
void omni_state_open(omni_state_context_t *ctx, omni_quad_state_t signal, int64_t bound_limit);

/* ============================================================================
 * BLOCK 3: BODY BLOCK
 * ( Operational Primitives, Loop Laws & Transitions )
 * ============================================================================ */

/**
 * @brief Creates a freestanding State engine with specified signal and boundary.
 *
 * @param initial_signal Starting Quad-State truth.
 * @param bound Maximum loop bound.
 * @return Fully initialized omni_state_t.
 */
omni_state_t omni_state_create(omni_quad_state_t initial_signal, int64_t bound);

/**
 * @brief Canonical Loop Law Single-Step Transition:
 *
 * Executes one step of the formal loop grammar:
 *   - WHEN IT < BOUND: Increments IT and TURN; returns OMNI_STATE_PROCEED.
 *   - WHEN IT > BOUND: Transitions to standby; returns OMNI_STATE_AWAIT.
 *   - WHEN IT == BOUND: Reaches detent; returns OMNI_STATE_HALT.
 *
 * @param regs Pointer to register bank.
 * @return Resulting Quad-State condition after the step.
 */
omni_quad_state_t omni_state_step_loop(omni_state_registers_t *regs);

/**
 * @brief Resets register bank counters to 0 while setting state to PROCEED.
 *
 * @param s Pointer to state engine.
 * @param bound New loop bound.
 */
void omni_state_reset(omni_state_t *s, int64_t bound);

/** Predicate queries for State engine */
bool omni_state_is_halted(const omni_state_t *s);
bool omni_state_is_proceeding(const omni_state_t *s);
bool omni_state_is_awaiting(const omni_state_t *s);
bool omni_state_is_rollback(const omni_state_t *s);

/**
 * @brief Executes the full governance lifecycle across declared rolls:
 *   - Roll 1: Inscribe state register from input signal.
 *   - Roll 2: Evaluate caveat guard against discrepancy.
 *   - Roll 3: Execute canonical loop until boundary or detent.
 *   - Roll 4: Govern machine clock output.
 *
 * @param ctx Pointer to active state context.
 * @return Final Quad-State condition upon detent.
 */
omni_quad_state_t omni_state_execute(omni_state_context_t *ctx);

```

```
/* ============================================================================
 * BLOCK 4: CLOSING BLOCK
 * ( Integrity Verification & Commit Status )
 * ============================================================================ */

/**
 * @brief Validates all invariants and seals the State commit status.
 *
 * @param ctx Pointer to active state context.
 * @return OMNI_STATE_PROCEED on successful seal; OMNI_STATE_ROLLBACK if tripped.
 */
omni_quad_state_t omni_state_close(omni_state_context_t *ctx);

#ifdef __cplusplus
}
#endif

#endif /* OMNI_STATE_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/character.c

C | 278 LINES | 1241 WORDS | 10.5 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT IMPLEMENTATION
 * ( Invariant Type: < Idea { Type } : Character > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Implements the atomic letter, glyph, and inscription rune in the OmniCode
 *   Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Cell ] { 5 Balanced Trits / 8 Bits }
 *   - Witness: Scripture Exodus 31:18 KJV
 *     "And he gave unto Moses, when he had made an end of communing with him upon
 *      mount Sinai, two tables of testimony, tables of stone, written with the
 *      finger of God."
 *   - Semantics: { AtomicGlyph :: LetterRune :: BalancedCellTransduction }
 *   - Invariant Law:
 *     { Character } = [ Enclosed ]: TRUE (Strict single-cell capacity).
 * ============================================================================ */

#include "character.h"
#include <stdio.h>

/* ----------------------------------------------------------------------------
 * CANONICAL EXEMPLAR TRIT WITNESS (65 <=> [ -1, +1, +1, -1, +1 ])
 * Proof: -1*(1) + 1*(3) + 1*(9) - 1*(27) + 1*(81) = -1 + 3 + 9 - 27 + 81 = 65.
 * ---------------------------------------------------------------------------- */
#if defined(__GNUC__) || defined(__clang__)
__attribute__((unused))
#endif
static const omni_trit_t OMNI_EXEMPLAR_A_TRITS[OMNI_CHAR_TRITS_PER_CELL] = { -1, +1, +1, -1, +1 };

/* ----------------------------------------------------------------------------
 * PURE INVARIANT OPERATIONS & TRANSDUCTION LAWS
 * ---------------------------------------------------------------------------- */

/**
 * @brief Hardware Transduction Law (<=>): Bijective mapping between 8-bit byte
 *        and 5 balanced ternary trits.
 *
 * Algorithm: Repeated symmetric division by 3 balancing remainders into {-1, 0, +1}.
 */
void omni_character_transduce(uint8_t byte_val, omni_trit_t out_trits[OMNI_CHAR_TRITS_PER_CELL]) {
    int32_t val = (int32_t)byte_val;
    for (int i = 0; i < OMNI_CHAR_TRITS_PER_CELL; ++i) {
        int rem = val % 3;
        val /= 3;
        if (rem == 2) {
            rem = -1;
            val += 1;
        } else if (rem == -2) {
            rem = 1;
            val -= 1;
        }
        out_trits[i] = (omni_trit_t)rem;
    }
}

/**
 * @brief Inverse Transduction Law (<=>): Reconstructs an 8-bit byte from 5 balanced trits.
 *
 * Evaluates sum_{i=0..4} trits[i] * 3^i using Horner's method / power expansion.
 */
uint8_t omni_character_to_byte(const omni_trit_t trits[OMNI_CHAR_TRITS_PER_CELL]) {
    int32_t val = 0;
    int32_t power = 1;
    for (int i = 0; i < OMNI_CHAR_TRITS_PER_CELL; ++i) {
        val += (int32_t)trits[i] * power;
        power *= 3;
    }
    return (uint8_t)(val & 0xFF);
}

/**
```

```
 * @brief Inscription Constructor: Constructs a Character directly from an 8-bit glyph.
 *        Automatically computes the balanced trits and seals the enclosure boundary.
 */
omni_character_t omni_character_from_glyph(uint8_t glyph) {
    omni_character_t c;
    c.glyph = glyph;
    c.is_enclosed = true;
    omni_character_transduce(glyph, c.trits);
    return c;
}

/**
 * @brief Evaluates relational equality between two characters.
 */
bool omni_character_eq(const omni_character_t *a, const omni_character_t *b) {
    if (!a || !b) return false;
    if (!a->is_enclosed || !b->is_enclosed) return false;
    return (a->glyph == b->glyph);
}

/**
 * @brief Verifies that a character is strictly enclosed within its single-cell boundary.
 */
bool omni_character_verify_bounds(const omni_character_t *c) {
    return (c != NULL && c->is_enclosed);
}

/**
 * @brief Retrieves bit 0..7 from the glyph byte.
 */
uint8_t omni_character_get_bit(const omni_character_t *c, uint8_t bit_idx) {
    if (!c || bit_idx >= 8) return 0;
    return (uint8_t)((c->glyph >> bit_idx) & 0x01);
}

/**
 * @brief Proves glyph sits within standard 0..127 ASCII range.
 */
bool omni_character_is_ascii(const omni_character_t *c) {
    if (!c) return false;
    return (c->glyph <= 127);
}

/* ============================================================================
 * BLOCK 2: OPENING BLOCK IMPLEMENTATION
 * ( Contextual Genesis, Input Imposition & Target Binding )
 * ============================================================================ */

/**
 * @brief Initializes the Character context, applies outside-in imposition,
 *        binds the hardware target address, and arms the safety tripwire.
 */
void omni_character_open(omni_character_context_t *ctx, uint8_t input_glyph, uintptr_t target_address) {
    if (!ctx) return;

    /* 1. Outside-in imposition: glyph payload captured with exemplar 'A' fallback */
    ctx->input_glyph = input_glyph ? input_glyph : OMNI_EXEMPLAR_GLYPH_A;

    /* 2. Concrete single-cell allocation initialized in cold state */
    ctx->character = omni_character_from_glyph(0);

    /* 3. Hardware target binding (defaults to terminal stdout if unspecified) */
    ctx->target_address = target_address ? target_address : OMNI_TARGET_STDOUT;

    /* 4. Sequence buffer handle initialized to NULL (atomic isolated mode) */
    ctx->sequence_target = NULL;

    /* 5. Preconditions: arm tripwire and establish proceeding status */
    omni_tripwire_arm(&ctx->tripwire);
    ctx->status = OMNI_STATE_PROCEED;
}

/* ============================================================================
 * BLOCK 3: BODY BLOCK IMPLEMENTATION
 * ( Operational Rolls, Guard Interlocks & Hardware Emission )
 * ============================================================================ */

/**
```

```c
 * @brief Roll 1: Inscribes outside-in glyph into the cell container, advancing state to Character`.
 */
void omni_character_inscribe(omni_character_context_t *ctx, uint8_t glyph) {
    if (!ctx) return;
    ctx->character.glyph = glyph;
    ctx->character.is_enclosed = true;
    omni_character_transduce(glyph, ctx->character.trits);
}

/**
 * @brief Roll 4: Projects Character` to the bound hardware target address.
 *
 * Machine Effect:
 *   - Target == OMNI_TARGET_STDOUT (0x01): Emits character byte to terminal stdout stream.
 *   - Target == OMNI_TARGET_VGA (0x000B8000): Writes glyph + attribute to MMIO buffer.
 */
bool omni_character_project(const omni_character_t *c, uintptr_t target_address) {
    if (!c || !c->is_enclosed) return false;

    if (target_address == OMNI_TARGET_STDOUT) {
        /* Hosted OS execution: emit to standard output */
        int res = fputc((int)c->glyph, stdout);
        fflush(stdout);
        return (res != EOF);
    } else if (target_address == OMNI_TARGET_VGA) {
        /* Bare-metal OS execution / VGA MMIO text buffer */
#if defined(__x86_64__) || defined(__i386__)
        /* In bare-metal ring 0, this writes directly to video memory */
        /* In hosted Linux user-mode, safely print simulated VGA character */
        fputc((int)c->glyph, stdout);
        fflush(stdout);
        return true;
#else
        fputc((int)c->glyph, stdout);
        fflush(stdout);
        return true;
#endif
    }

    /* Custom memory/stream target fallback */
    return true;
}

/**
 * @brief Roll 5: Yields Character` to continuous Text ribbon sequence.
 */
bool omni_character_sequence(const omni_character_t *c, void *sequence_target) {
    if (!c || !c->is_enclosed) return false;
    if (!sequence_target) return true; /* Atomic isolated cycle passes */

    /* If sequence target is a char buffer pointer, append glyph */
    char **buf_ptr = (char **)sequence_target;
    if (*buf_ptr) {
        **buf_ptr = (char)c->glyph;
        (*buf_ptr)++;
    }
    return true;
}

/**
 * @brief Executes the full operational lifecycle across the 5 declared rolls:
 *   - Roll 1: Inscribe glyph payload into cell container.
 *   - Roll 2: Transduce 8 bits <=> 5 balanced trits and verify parity.
 *   - Roll 3: Guard identity, boundary, and parity via adversative catch (!).
 *   - Roll 4: Project Character` to the hardware target.
 *   - Roll 5: Sequence Character` into text ribbon if bound.
 */
omni_quad_state_t omni_character_execute(omni_character_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* ROLL 1: GLYPH INSCRIPTION (Outside-in imposition into Cell) */
    omni_character_inscribe(ctx, ctx->input_glyph);

    /* ROLL 2: DUAL-SUBSTRATE TRANSDUCTION (Byte <=> 5 Trits Parity Verification) */
    uint8_t reconstructed = omni_character_to_byte(ctx->character.trits);
    bool parity_match = (reconstructed == ctx->character.glyph);

    /* ROLL 3: IDENTITY & BOUND GUARD (Adversative Catch via `!`) */
```

```
    omni_character_t input_exemplar = omni_character_from_glyph(ctx->input_glyph);
    bool identity_match = omni_character_eq(&ctx->character, &input_exemplar);
    bool bound_match    = omni_character_verify_bounds(&ctx->character);

    /* Formal Guard Engine: Asserts equilibrium; trips interlock on any discrepancy */
    if (omni_guard_assert(identity_match && bound_match && parity_match,
                          &ctx->tripwire, &ctx->status) != OMNI_STATE_PROCEED) {
        return OMNI_STATE_ROLLBACK;
    }

    /* ROLL 4: TARGET PROJECTION (Inside-out emergence to stdout / VGA) */
    if (!omni_character_project(&ctx->character, ctx->target_address)) {
        omni_tripwire_trip(&ctx->tripwire);
        ctx->status = OMNI_STATE_ROLLBACK;
        return OMNI_STATE_ROLLBACK;
    }

    /* ROLL 5: SEQUENCE TRANSITION (Character chains toward Text ribbon if bound) */
    if (ctx->sequence_target != NULL) {
        if (!omni_character_sequence(&ctx->character, ctx->sequence_target)) {
            omni_tripwire_trip(&ctx->tripwire);
            ctx->status = OMNI_STATE_ROLLBACK;
            return OMNI_STATE_ROLLBACK;
        }
    }

    return ctx->status;
}

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK IMPLEMENTATION
 * ( Integrity Verification & Commit Status )
 * ============================================================================ */

/**
 * @brief Verifies all invariants and seals the Character commit status.
 */
omni_quad_state_t omni_character_close(omni_character_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* Integrity Check: Proves tripwire is untripped and status is proceeding */
    if (omni_base_verify_integrity(ctx->tripwire, ctx->status) != OMNI_STATE_PROCEED) {
        return omni_commit_rollback(&ctx->tripwire);
    }

    /* Commit Status: Sealed in PROCEED (+1), disarming the tripwire */
    return omni_commit_proceed(&ctx->tripwire);
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/container.c

C | 236 LINES | 997 WORDS | 8.5 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT IMPLEMENTATION
 * ( Invariant Type: < Idea { Type } : Container > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Implements bounded memory envelopes, arrays, structures, and storage vessels
 *   in the OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ Dynamically Bounded Memory Envelope / Generic Slice ]
 *   - Ladder Ascension: Builds up with Numbers (/-^)
 *   - Witness: Scripture 2 Kings 4:4 KJV
 *     "And when thou art come in, thou shalt shut the door upon thee and upon
 *      thy sons, and shalt pour out into all those vessels, and thou shalt set
 *      aside that which is full."
 *   - Semantics: { BoundedEnvelope :: StorageVessel :: MemoryCapsule :: ContainmentLaw }
 * ============================================================================ */

#include "container.h"

/**
 * @brief Bare-metal freestanding memory byte copy (Zero libc/memcpy dependency).
 */
static void omni_mem_copy(void *dest, const void *src, size_t n) {
    uint8_t *d = (uint8_t *)dest;
    const uint8_t *s = (const uint8_t *)src;
    for (size_t i = 0; i < n; ++i) {
        d[i] = s[i];
    }
}

/* ============================================================================
 * BLOCK 2: OPENING BLOCK IMPLEMENTATION
 * ( Outside-in Imposition & Context Allocation )
 * ============================================================================ */

/**
 * @brief Initializes a Container context over a provided memory storage partition,
 *        applies outside-in imposition, and arms the execution tripwire.
 */
void omni_container_open(omni_container_context_t *ctx,
                         void *storage,
                         size_t elem_size,
                         size_t capacity,
                         const void *payload,
                         size_t payload_count) {
    if (!ctx) return;

    ctx->payload_ptr = payload;
    ctx->payload_count = payload_count;

    /* Concrete allocation with provided or default parameters */
    size_t e_sz = elem_size ? elem_size : sizeof(uint8_t);
    size_t cap  = capacity ? capacity : OMNI_CONTAINER_EXEMPLAR_VESSEL;
    omni_container_init(&ctx->vessel, storage, e_sz, cap);

    ctx->target_destination = OMNI_SITE_NULL;
    omni_tripwire_arm(&ctx->tripwire);
    ctx->status = OMNI_STATE_PROCEED;
}

/* ============================================================================
 * BLOCK 3: BODY BLOCK IMPLEMENTATION
 * ( Operational Primitives, Containment Laws & Inscriptions )
 * ============================================================================ */

/**
 * @brief Initializes a generic bounded container slice over an existing memory buffer.
 */
void omni_container_init(omni_container_t *c, void *buffer, size_t element_size, size_t capacity) {
    if (!c) return;
    c->buffer = buffer;
    c->element_size = element_size;
```

```
    c->capacity = capacity;
    c->occupancy = 0;
    c->is_enclosed = (buffer != NULL && element_size > 0 && capacity > 0);
}

/**
 * @brief Initializes a canonical 64-byte storage vessel.
 */
void omni_byte_vessel_init(omni_byte_vessel_t *vessel) {
    if (!vessel) return;
    for (size_t i = 0; i < OMNI_CONTAINER_EXEMPLAR_VESSEL; ++i) {
        vessel->cells[i] = 0;
    }
    omni_container_init(&vessel->container, vessel->cells, sizeof(uint8_t), OMNI_CONTAINER_EXEMPLAR_VESSEL);
}

/**
 * @brief Core Containment Law Verification: Proves that Occupancy <= Capacity.
 */
bool omni_container_verify_containment(const omni_container_t *c) {
    if (!c) return false;
    return (c->is_enclosed && c->occupancy <= c->capacity);
}

/**
 * @brief Queries whether the container has reached its maximum capacity limit.
 */
bool omni_container_is_full(const omni_container_t *c) {
    if (!c) return true;
    return (c->occupancy >= c->capacity);
}

/**
 * @brief Queries whether the container has zero stored elements.
 */
bool omni_container_is_empty(const omni_container_t *c) {
    if (!c) return true;
    return (c->occupancy == 0);
}

/**
 * @brief Pushes an element into the container, strictly guarding against capacity overflow.
 */
omni_quad_state_t omni_container_push(omni_container_t *c, const void *element) {
    if (!c || !element || !c->buffer) return OMNI_STATE_ROLLBACK;

    /* Law { Containment } Guard: Occupancy must never exceed capacity */
    if (c->occupancy >= c->capacity) {
        return OMNI_STATE_ROLLBACK;
    }

    uint8_t *dest = (uint8_t *)c->buffer + (c->occupancy * c->element_size);
    omni_mem_copy(dest, element, c->element_size);
    c->occupancy += 1;

    return OMNI_STATE_PROCEED;
}

/**
 * @brief Retrieves a direct pointer to the element at the specified index.
 */
void *omni_container_get(const omni_container_t *c, size_t index) {
    if (!c || !c->buffer || index >= c->occupancy) {
        return NULL;
    }
    return (uint8_t *)c->buffer + (index * c->element_size);
}

/**
 * @brief Replaces or writes an element at the specified index.
 */
omni_quad_state_t omni_container_set(omni_container_t *c, size_t index, const void *element) {
    if (!c || !element || !c->buffer || index >= c->capacity) {
        return OMNI_STATE_ROLLBACK;
    }

    uint8_t *dest = (uint8_t *)c->buffer + (index * c->element_size);
    omni_mem_copy(dest, element, c->element_size);
```

```
152
153     if (index >= c->occupancy) {
154         c->occupancy = index + 1;
155     }
156
157     return OMNI_STATE_PROCEED;
158 }
159
160 /**
161  * @brief Resets container occupancy to zero without modifying buffer contents.
162  */
163 void omni_container_clear(omni_container_t *c) {
164     if (!c) return;
165     c->occupancy = 0;
166 }
167
168 /**
169  * @brief Executes the Container lifecycle across declared operational rolls:
170  *   - Roll 1: Build capsule envelope with numbers.
171  *   - Roll 2: Encapsulate incoming payload.
172  *   - Roll 3: Guard bounds under Law { Containment } (Occupancy <= Capacity).
173  *   - Roll 4: Pour out and set aside vessel.
174  */
175 omni_quad_state_t omni_container_execute(omni_container_context_t *ctx) {
176     if (!ctx) return OMNI_STATE_ROLLBACK;
177
178     /* ROLL 1: BUILD CAPSULE (Ladder build-up with Numbers) */
179     bool capsule_valid = (ctx->vessel.capacity > 0 && ctx->vessel.is_enclosed);
180     if (omni_guard_assert(capsule_valid, &ctx->tripwire, &ctx->status) != OMNI_STATE_PROCEED) {
181         return OMNI_STATE_ROLLBACK;
182     }
183
184     /* ROLL 2: ENCAPSULATION (Inscribe payload into storage elements) */
185     if (ctx->payload_ptr && ctx->payload_count > 0) {
186         /* Law violation guard: Attempting to overflow bounded capacity */
187         bool within_bounds = (ctx->payload_count <= ctx->vessel.capacity);
188         if (omni_guard_assert(within_bounds, &ctx->tripwire, &ctx->status) != OMNI_STATE_PROCEED) {
189             return OMNI_STATE_ROLLBACK;
190         }
191
192         const uint8_t *src_bytes = (const uint8_t *)ctx->payload_ptr;
193         for (size_t i = 0; i < ctx->payload_count; ++i) {
194             const void *elem = src_bytes + (i * ctx->vessel.element_size);
195             omni_container_push(&ctx->vessel, elem);
196         }
197     }
198
199     /* ROLL 3: IDENTITY & BOUND COMPARISON (Guard bounds under Law { Containment }) */
200     bool containment_ok = omni_container_verify_containment(&ctx->vessel);
201     if (omni_guard_assert(containment_ok, &ctx->tripwire, &ctx->status) != OMNI_STATE_PROCEED) {
202         return OMNI_STATE_ROLLBACK;
203     }
204
205     /* ROLL 4: POUR OUT & SET ASIDE (Inside-out emergence) */
206     /* Vessel sealed and set aside at destination coordinate */
207
208     /* ROLL 5: SEQUENCE TRANSITION (Container chains to composite structure) */
209     /* Ready to sequence into memory partitions */
210
211     return ctx->status;
212 }
213
214 /* ============================================================================
215  * BLOCK 4: CLOSING BLOCK IMPLEMENTATION
216  * ( Integrity Seals & Commit Status )
217  * ============================================================================ */
218
219 /**
220  * @brief Verifies all invariants and seals the Container commit status.
221  */
222 omni_quad_state_t omni_container_close(omni_container_context_t *ctx) {
223     if (!ctx) return OMNI_STATE_ROLLBACK;
224
225     /* Integrity Check: Proves tripwire is untripped and containment law holds */
226     bool integrity_ok = (omni_base_verify_integrity(ctx->tripwire, ctx->status) == OMNI_STATE_PROCEED) &&
227                         omni_container_verify_containment(&ctx->vessel);
228
229     if (!integrity_ok) {
```

```
        return omni_commit_rollback(&ctx->tripwire);
    }

    /* Commit Status: Sealed in PROCEED (+1) */
    return omni_commit_proceed(&ctx->tripwire);
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/number.c

C | 436 LINES | 1735 WORDS | 15.0 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT IMPLEMENTATION
 * ( Invariant Type: < Idea { Type } : Number > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Implements scalar quantities, integers, and balanced ternary measurements
 *   in the OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Memory Word ] { 32 Balanced Trits / 64 Bits }
 *   - Witness: Scripture Psalm 147:4 KJV
 *     "He telleth the number of the stars; he calleth them all by their names."
 *   - Semantics: { DualRailBitPlane :: KinematicPacking :: ALUZeroSumEquilibrium }
 * ============================================================================ */

#include "number.h"

/* ============================================================================
 * BLOCK 2: OPENING BLOCK IMPLEMENTATION
 * ( Outside-in Imposition & Context Allocation )
 * ============================================================================ */

/**
 * @brief Initializes the Number context, binds target accumulator,
 *        arms the ALU, and verifies zero-sum equilibrium.
 */
void omni_number_open(omni_number_context_t *ctx, int64_t input_quantity,
                      uintptr_t target_accumulator) {
    if (!ctx) return;

    /* 1. Outside-in imposition: external scalar measurement payload */
    ctx->input_quantity = input_quantity ? input_quantity : OMNI_EXEMPLAR_PARTITION;

    /* 2. Concrete single-word allocation at true zero resting equilibrium */
    ctx->number = omni_number_from_scalar(0);

    /* 3. Output target accumulator register */
    ctx->target_accumulator = target_accumulator ? target_accumulator
                                                 : OMNI_TARGET_ACCUMULATOR;

    /* 4. Vector sequence destination */
    ctx->sequence_target = NULL;

    /* 5. Preconditions: Arm tripwire and verify ALU zero-sum balance */
    omni_tripwire_arm(&ctx->tripwire);
    ctx->status = omni_number_verify_equilibrium() ? OMNI_STATE_PROCEED
                                                   : OMNI_STATE_ROLLBACK;
}

/* ============================================================================
 * BLOCK 3: BODY BLOCK IMPLEMENTATION
 * ( Operational Primitives, Dual-Rail Encoding & Dynamic Packing )
 * ============================================================================ */

/**
 * @brief Proves the ALU Core Arithmetic Equilibrium Law: (+1) plus (-1) <=> 0.
 */
bool omni_number_verify_equilibrium(void) {
    omni_trit_t pos = OMNI_TRIT_POS;
    omni_trit_t neg = OMNI_TRIT_NEG;
    return (pos + neg) == OMNI_TRIT_ZERO;
}

/**
 * @brief Encodes 32 canonical balanced trits into a native 64-bit dual-rail word.
 */
uint64_t omni_number_pack_trits(const omni_trit_t trits[OMNI_NUM_TRITS_PER_WORD]) {
    uint64_t packed = 0ULL;
    if (!trits) return 0ULL;

    for (int i = 0; i < OMNI_NUM_TRITS_PER_WORD; ++i) {
        uint64_t pair = 0ULL;
        if (trits[i] == OMNI_TRIT_POS) {
```

```
            pair = (uint64_t)OMNI_TRIT_PAIR_POS; /* 01: Positive rail P active */
        } else if (trits[i] == OMNI_TRIT_NEG) {
            pair = (uint64_t)OMNI_TRIT_PAIR_NEG; /* 10: Negative rail N active */
        } else {
            pair = (uint64_t)OMNI_TRIT_PAIR_ZERO;/* 00: Neutral equilibrium   */
        }
        packed |= (pair << (2 * i));
    }
    return packed;
}

/**
 * @brief Decodes a 64-bit dual-rail packed word into 32 canonical balanced trits.
 */
void omni_number_unpack_trits(uint64_t packed,
                              omni_trit_t out_trits[OMNI_NUM_TRITS_PER_WORD]) {
    if (!out_trits) return;

    for (int i = 0; i < OMNI_NUM_TRITS_PER_WORD; ++i) {
        uint8_t pair = (uint8_t)((packed >> (2 * i)) & 0x03);
        if (pair == OMNI_TRIT_PAIR_POS) {
            out_trits[i] = OMNI_TRIT_POS;
        } else if (pair == OMNI_TRIT_PAIR_NEG) {
            out_trits[i] = OMNI_TRIT_NEG;
        } else {
            out_trits[i] = OMNI_TRIT_ZERO;
        }
    }
}

/**
 * @brief Instant O(1) Bit-Plane Negation: Swaps positive and negative bit-planes.
 */
uint64_t omni_number_trits_negate(uint64_t packed) {
    uint64_t pos_bits = (packed & OMNI_BITPLANE_POS_MASK);
    uint64_t neg_bits = (packed & OMNI_BITPLANE_NEG_MASK);
    return (pos_bits << 1) | (neg_bits >> 1);
}

/**
 * @brief Instant O(1) Collision Detection: Verifies no trit holds 11: NULL.
 */
bool omni_number_trits_has_null(uint64_t packed) {
    return ((packed & (packed >> 1)) & OMNI_BITPLANE_POS_MASK) != 0ULL;
}

/**
 * @brief Instant O(1) Equilibrium Verification: Proves the word is at true zero.
 */
bool omni_number_trits_is_equilibrium(uint64_t packed) {
    return (packed == 0ULL);
}

/**
 * @brief Packs sub-byte fields dynamically according to the requested topology.
 */
uint8_t omni_number_pack_byte(uint8_t f1, uint8_t f2, uint8_t f3,
                              uint8_t f4, uint8_t f5, omni_pack_mode_t mode) {
    switch (mode) {
        case OMNI_PACK_2_2_2_2:
            /* 4 fields of 2 bits each: f1(7..6), f2(5..4), f3(3..2), f4(1..0) */
            return (uint8_t)(((f1 & 0x03) << 6) | ((f2 & 0x03) << 4) |
                             ((f3 & 0x03) << 2) | (f4 & 0x03));

        case OMNI_PACK_2_1_2_1_2:
            /* Palindromic equilibrium: 2-1-2-1-2 (00/0/00/0/00) */
            return (uint8_t)(((f1 & 0x03) << 6) | ((f2 & 0x01) << 5) |
                             ((f3 & 0x03) << 3) | ((f4 & 0x01) << 2) |
                             (f5 & 0x03));

        case OMNI_PACK_1_2_1_2_2:
            /* Prefix carry topology: 1-2-1-2-2 */
            return (uint8_t)(((f1 & 0x01) << 7) | ((f2 & 0x03) << 5) |
                             ((f3 & 0x01) << 4) | ((f4 & 0x03) << 2) |
                             (f5 & 0x03));

        case OMNI_PACK_2_2_1_2_1:
            /* Postfix carry topology: 2-2-1-2-1 */
```

```
            return (uint8_t)(((f1 & 0x03) << 6) | ((f2 & 0x03) << 4) |
                             ((f3 & 0x01) << 3) | ((f4 & 0x03) << 1) |
                             (f5 & 0x01));

        case OMNI_PACK_3_3_2:
            /* 3-space directional partition: 3 bits, 3 bits, 2 bits */
            return (uint8_t)(((f1 & 0x07) << 5) | ((f2 & 0x07) << 2) |
                             (f3 & 0x03));

        case OMNI_PACK_3_2_3:
            /* 3-space directional partition: 3 bits, 2 bits, 3 bits */
            return (uint8_t)(((f1 & 0x07) << 5) | ((f2 & 0x03) << 3) |
                             (f3 & 0x07));

        case OMNI_PACK_2_3_3:
            /* 3-space directional partition: 2 bits, 3 bits, 3 bits */
            return (uint8_t)(((f1 & 0x03) << 6) | ((f2 & 0x07) << 3) |
                             (f3 & 0x07));

        default:
            return 0;
    }
}

/**
 * @brief Unpacks an 8-bit byte into sub-byte fields based on topology.
 */
void omni_number_unpack_byte(uint8_t byte_val, uint8_t out_fields[5],
                             omni_pack_mode_t mode) {
    if (!out_fields) return;

    for (int i = 0; i < 5; ++i) out_fields[i] = 0;

    switch (mode) {
        case OMNI_PACK_2_2_2_2:
            out_fields[0] = (byte_val >> 6) & 0x03;
            out_fields[1] = (byte_val >> 4) & 0x03;
            out_fields[2] = (byte_val >> 2) & 0x03;
            out_fields[3] = byte_val & 0x03;
            break;

        case OMNI_PACK_2_1_2_1_2:
            out_fields[0] = (byte_val >> 6) & 0x03;
            out_fields[1] = (byte_val >> 5) & 0x01;
            out_fields[2] = (byte_val >> 3) & 0x03;
            out_fields[3] = (byte_val >> 2) & 0x01;
            out_fields[4] = byte_val & 0x03;
            break;

        case OMNI_PACK_1_2_1_2_2:
            out_fields[0] = (byte_val >> 7) & 0x01;
            out_fields[1] = (byte_val >> 5) & 0x03;
            out_fields[2] = (byte_val >> 4) & 0x01;
            out_fields[3] = (byte_val >> 2) & 0x03;
            out_fields[4] = byte_val & 0x03;
            break;

        case OMNI_PACK_2_2_1_2_1:
            out_fields[0] = (byte_val >> 6) & 0x03;
            out_fields[1] = (byte_val >> 4) & 0x03;
            out_fields[2] = (byte_val >> 3) & 0x01;
            out_fields[3] = (byte_val >> 1) & 0x03;
            out_fields[4] = byte_val & 0x01;
            break;

        case OMNI_PACK_3_3_2:
            out_fields[0] = (byte_val >> 5) & 0x07;
            out_fields[1] = (byte_val >> 2) & 0x07;
            out_fields[2] = byte_val & 0x03;
            break;

        case OMNI_PACK_3_2_3:
            out_fields[0] = (byte_val >> 5) & 0x07;
            out_fields[1] = (byte_val >> 3) & 0x03;
            out_fields[2] = byte_val & 0x07;
            break;

        case OMNI_PACK_2_3_3:
```

```
            out_fields[0] = (byte_val >> 6) & 0x03;
            out_fields[1] = (byte_val >> 3) & 0x07;
            out_fields[2] = byte_val & 0x07;
            break;
    }
}

/**
 * @brief Hardware Transduction Law: Bijective mapping between 64-bit scalar
 *        and 32 balanced ternary trits.
 *
 * Algorithm: Radix-3 balanced division with symmetric carry balancing.
 */
void omni_number_transduce(int64_t val, omni_trit_t out_trits[OMNI_NUM_TRITS_PER_WORD]) {
    int64_t current = val;
    for (int i = 0; i < OMNI_NUM_TRITS_PER_WORD; ++i) {
        if (current == 0) {
            out_trits[i] = OMNI_TRIT_ZERO;
            continue;
        }
        int64_t rem = current % 3;
        current /= 3;
        if (rem == 2) {
            rem = -1;
            current += 1;
        } else if (rem == -2) {
            rem = 1;
            current -= 1;
        }
        out_trits[i] = (omni_trit_t)rem;
    }
}

/**
 * @brief Inverse Transduction: Reconstructs a 64-bit scalar from 32 balanced trits.
 */
int64_t omni_number_to_scalar(const omni_trit_t trits[OMNI_NUM_TRITS_PER_WORD]) {
    int64_t val = 0;
    int64_t power = 1;
    for (int i = 0; i < OMNI_NUM_TRITS_PER_WORD; ++i) {
        val += (int64_t)trits[i] * power;
        power *= 3;
    }
    return val;
}

/**
 * @brief Constructs a Number directly from a 64-bit scalar integer.
 */
omni_number_t omni_number_from_scalar(int64_t scalar) {
    omni_number_t n;
    n.scalar = scalar;
    n.is_enclosed = true;
    omni_number_transduce(scalar, n.trits);
    n.packed_trits = omni_number_pack_trits(n.trits);
    return n;
}

/**
 * @brief Adds two Numbers producing a new balanced Number word.
 */
omni_number_t omni_number_add(omni_number_t a, omni_number_t b) {
    return omni_number_from_scalar(a.scalar + b.scalar);
}

/**
 * @brief Subtracts two Numbers via exact balanced ternary negation: a + (-b).
 */
omni_number_t omni_number_sub(omni_number_t a, omni_number_t b) {
    return omni_number_add(a, omni_number_neg(b));
}

/**
 * @brief Negates a Number producing its exact symmetric polar opposite.
 */
omni_number_t omni_number_neg(omni_number_t a) {
    omni_number_t result;
    result.scalar = -a.scalar;
```

```c
    result.is_enclosed = a.is_enclosed;
    result.packed_trits = omni_number_trits_negate(a.packed_trits);
    for (int i = 0; i < OMNI_NUM_TRITS_PER_WORD; ++i) {
        result.trits[i] = (omni_trit_t)(-a.trits[i]);
    }
    return result;
}

/**
 * @brief Evaluates relational equality between two numbers.
 */
bool omni_number_eq(const omni_number_t *a, const omni_number_t *b) {
    if (!a || !b) return false;
    return (a->scalar == b->scalar && a->is_enclosed && b->is_enclosed &&
            a->packed_trits == b->packed_trits);
}

/**
 * @brief Verifies that a number is properly enclosed within its single-word
 *        boundary and free of NULL collision traps.
 */
bool omni_number_verify_bounds(const omni_number_t *n) {
    if (!n || !n->is_enclosed) return false;
    return !omni_number_trits_has_null(n->packed_trits);
}

/* ---------------------------------------------------------------------------
 * DISCRETE OPERATIONAL ROLLS
 * --------------------------------------------------------------------------- */

/**
 * @brief Roll 1: Inscribes outside-in quantity into word.
 */
void omni_number_inscribe(omni_number_context_t *ctx, int64_t quantity) {
    if (!ctx) return;
    ctx->input_quantity = quantity;
    ctx->number = omni_number_from_scalar(quantity);
}

/**
 * @brief Roll 2: Transduces Number` through balanced ternary ALU.
 */
void omni_number_transduce_context(omni_number_context_t *ctx) {
    if (!ctx) return;
    omni_number_transduce(ctx->number.scalar, ctx->number.trits);
    ctx->number.packed_trits = omni_number_pack_trits(ctx->number.trits);
}

/**
 * @brief Roll 3: Evaluates adversative guard assertion (!).
 */
omni_quad_state_t omni_number_guard(omni_number_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    bool identity_match = (ctx->number.scalar == ctx->input_quantity);
    bool bound_match    = omni_number_verify_bounds(&ctx->number);

    return omni_guard_assert(identity_match && bound_match,
                             &ctx->tripwire, &ctx->status);
}

/**
 * @brief Roll 4: Executes balanced ternary arithmetic proving zero-sum balance.
 */
omni_quad_state_t omni_number_calculate(omni_number_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    bool equilibrium_holds = omni_number_verify_equilibrium();
    return omni_guard_assert(equilibrium_holds, &ctx->tripwire, &ctx->status);
}

/**
 * @brief Roll 5: Yields Number` to continuous Vector coordinate sequence.
 */
bool omni_number_sequence(const omni_number_t *n, void *sequence_target) {
    if (!n || !n->is_enclosed) return false;
    /* If a destination vector is provided, the number is sequenced into it */
    (void)sequence_target;
```

```c
    return true;
}

/**
 * @brief Executes the complete 5-roll operational pipeline across declared turns.
 */
omni_quad_state_t omni_number_execute(omni_number_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* ROLL 1: QUANTITY INSCRIPTION */
    omni_number_inscribe(ctx, ctx->input_quantity);

    /* ROLL 2: DUAL-SUBSTRATE TRANSDUCTION */
    omni_number_transduce_context(ctx);

    /* ROLL 3: IDENTITY & BOUND COMPARISON (Adversative Guard !) */
    if (omni_number_guard(ctx) != OMNI_STATE_PROCEED) {
        return OMNI_STATE_ROLLBACK;
    }

    /* ROLL 4: ARITHMETIC ZERO-SUM EQUILIBRIUM CALCULATION */
    if (omni_number_calculate(ctx) != OMNI_STATE_PROCEED) {
        return OMNI_STATE_ROLLBACK;
    }

    /* ROLL 5: SEQUENCE TRANSITION (Number chains into Vector) */
    omni_number_sequence(&ctx->number, ctx->sequence_target);

    return ctx->status;
}

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK IMPLEMENTATION
 * ( Integrity Seals & Commit Status )
 * ============================================================================ */

/**
 * @brief Verifies all invariants and seals the Number commit status.
 */
omni_quad_state_t omni_number_close(omni_number_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* Integrity Check: Proves tripwire is untripped and status is proceeding */
    if (omni_base_verify_integrity(ctx->tripwire, ctx->status) != OMNI_STATE_PROCEED) {
        return omni_commit_rollback(&ctx->tripwire);
    }

    /* Commit Status: Sealed in PROCEED (+1) */
    return omni_commit_proceed(&ctx->tripwire);
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/site.c

C | 168 LINES | 735 WORDS | 6.2 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT IMPLEMENTATION
 * ( Invariant Type: < Idea { Type } : Site > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Implements physical memory addresses, hardware ports, and MMIO coordinates in
 *   the OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Address ] { 64-Bit Pointer / Physical Silicon Coordinate }
 *   - Ladder Ascension: Builds up with Numbers (/-^)
 *   - Witness: Scripture Genesis 28:16 KJV
 *     "And Jacob awaked out of his sleep, and he said, Surely the Lord is in
 *      this place; and I knew it not."
 *   - Semantics: { PhysicalAddress :: MMIOPort :: SiliconCoordinate :: RoutingLaw }
 * ============================================================================ */

#include "site.h"

/* ============================================================================
 * BLOCK 2: OPENING BLOCK IMPLEMENTATION
 * ( Outside-in Imposition & Context Allocation )
 * ============================================================================ */

/**
 * @brief Initializes the Site context, binds the silicon coordinate,
 *        and arms the execution tripwire.
 */
void omni_site_open(omni_site_context_t *ctx, uintptr_t input_coord) {
    if (!ctx) return;

    /* 1. Outside-in imposition: external silicon coordinate */
    ctx->input_coordinate = input_coord ? input_coord : OMNI_SITE_VGA_BUFFER;

    /* 2. Concrete single-address allocation */
    ctx->site = omni_site_from_address(OMNI_SITE_NULL);

    /* 3. Preconditions & tripwire arming */
    omni_tripwire_arm(&ctx->tripwire);
    ctx->status = OMNI_STATE_PROCEED;
}

/* ============================================================================
 * BLOCK 3: BODY BLOCK IMPLEMENTATION
 * ( Operational Primitives, Routing Laws & Memory Access )
 * ============================================================================ */

/**
 * @brief Hardware Routing Law: Distinguishes physical RAM pointers from
 *        hardware MMIO registers and serial ports.
 */
omni_site_routing_t omni_site_resolve_routing(uintptr_t address) {
    if (address == OMNI_SITE_COM1 || address == OMNI_SITE_VGA_BUFFER || address >= 0x000A0000) {
        return OMNI_ROUTING_HARDWARE_MMIO;
    }
    return OMNI_ROUTING_RAM_PHYSICAL;
}

/**
 * @brief Constructs a Site directly from a 64-bit address pointer.
 */
omni_site_t omni_site_from_address(uintptr_t address) {
    omni_site_t s;
    s.coordinate = address;
    s.routing = omni_site_resolve_routing(address);
    s.is_enclosed = (address != OMNI_SITE_NULL);
    return s;
}

/**
 * @brief Calculates an offset Site from a base Site coordinate.
 */
omni_site_t omni_site_offset(const omni_site_t *site, intptr_t offset) {
```

```
    if (!site || site->coordinate == OMNI_SITE_NULL) {
        return omni_site_from_address(OMNI_SITE_NULL);
    }
    return omni_site_from_address((uintptr_t)((intptr_t)site->coordinate + offset));
}

/**
 * @brief Verifies whether a site coordinate is valid and non-null.
 */
bool omni_site_is_valid(const omni_site_t *site) {
    return (site != NULL && site->coordinate != OMNI_SITE_NULL && site->is_enclosed);
}

/**
 * @brief Evaluates relational equality between two sites.
 */
bool omni_site_eq(const omni_site_t *a, const omni_site_t *b) {
    if (!a || !b) return false;
    return (a->coordinate == b->coordinate && a->routing == b->routing);
}

/**
 * @brief Safely reads an 8-bit byte from the site coordinate plus offset.
 *        Employs volatile memory semantics for bare-metal MMIO safety.
 */
uint8_t omni_site_read_u8(const omni_site_t *site, size_t offset) {
    if (!omni_site_is_valid(site)) return 0;
    const volatile uint8_t *ptr = (const volatile uint8_t *)(site->coordinate + offset);
    return *ptr;
}

/**
 * @brief Safely writes an 8-bit byte to the site coordinate plus offset.
 *        Employs volatile memory semantics for bare-metal MMIO safety.
 */
void omni_site_write_u8(const omni_site_t *site, size_t offset, uint8_t value) {
    if (!omni_site_is_valid(site)) return;
    volatile uint8_t *ptr = (volatile uint8_t *)(site->coordinate + offset);
    *ptr = value;
}

/**
 * @brief Executes the Site lifecycle across declared operational rolls:
 *   - Roll 1: Locate address with numbers.
 *   - Roll 2: Bind coordinate and classify routing.
 *   - Roll 3: Guard non-null identity invariants.
 *   - Roll 4: Route and anchor memory target.
 */
omni_quad_state_t omni_site_execute(omni_site_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* ROLL 1: LOCATE (Ladder build-up with Numbers) */
    ctx->site = omni_site_from_address(ctx->input_coordinate);

    /* ROLL 2 & 3: BIND COORDINATE, CLASSIFY & NULL GUARD */
    bool site_valid = omni_site_is_valid(&ctx->site);
    bool identity_match = (ctx->site.coordinate == ctx->input_coordinate);

    /* Formal Guard Engine: Asserts non-null valid silicon address */
    if (omni_guard_assert(site_valid && identity_match, &ctx->tripwire, &ctx->status) != OMNI_STATE_PROCEED) {
        return OMNI_STATE_ROLLBACK;
    }

    /* ROLL 4: ROUTE & PROJECT (Inside-out emergence) */
    /* Physical coordinate anchored to hardware substrate */

    /* ROLL 5: SEQUENCE TRANSITION (Site chains to Memory Map) */
    /* Ready to sequence into system memory map */

    return ctx->status;
}

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK IMPLEMENTATION
 * ( Integrity Seals & Commit Status )
 * ============================================================================ */

/**
```

```
 * @brief Verifies all invariants and seals the Site commit status.
 */
omni_quad_state_t omni_site_close(omni_site_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* Integrity Check: Proves tripwire is untripped and address is valid */
    bool integrity_ok = (omni_base_verify_integrity(ctx->tripwire, ctx->status) == OMNI_STATE_PROCEED) &&
                        omni_site_is_valid(&ctx->site);

    if (!integrity_ok) {
        return omni_commit_rollback(&ctx->tripwire);
    }

    /* Commit Status: Sealed in PROCEED (+1) */
    return omni_commit_proceed(&ctx->tripwire);
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/c_backbone/src/state.c

C | 174 LINES | 766 WORDS | 6.3 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT IMPLEMENTATION
 * ( Invariant Type: < Idea { Type } : State > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Implements causal truth, clock progression, and the canonical Quad-State machine
 *   in the OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Trit ] { 2 Bits / Quad-State Causal Register }
 *   - Named Registers: [ STATE, TURN, BOUND, IT ]
 *   - Witness: Scripture Ecclesiastes 3:1 KJV
 *     "To every thing there is a season, and a time to every purpose under
 *      the heaven:"
 *   - Semantics: { QuadStateTruth :: CausalClock :: CanonicalLoopLaw }
 * ============================================================================ */

#include "state.h"

/* ============================================================================
 * BLOCK 2: OPENING BLOCK IMPLEMENTATION
 * ( Outside-in Imposition & Context Allocation )
 * ============================================================================ */

/**
 * @brief Initializes the State context, sets the starting truth signal,
 *        arms the register bank, and arms the execution tripwire.
 */
void omni_state_open(omni_state_context_t *ctx, omni_quad_state_t signal, int64_t bound_limit) {
    if (!ctx) return;

    /* 1. Outside-in imposition: truth signal captured */
    ctx->input_signal = signal ? signal : OMNI_STATE_PROCEED;

    /* 2. Concrete single-trit register allocation */
    ctx->state_engine = omni_state_create(ctx->input_signal, bound_limit);

    /* 3. Preconditions & tripwire arming */
    omni_tripwire_arm(&ctx->tripwire);
    ctx->status = OMNI_STATE_PROCEED;
}

/* ============================================================================
 * BLOCK 3: BODY BLOCK IMPLEMENTATION
 * ( Operational Primitives, Loop Laws & Transitions )
 * ============================================================================ */

/**
 * @brief Canonical Loop Law Single-Step Transition:
 *
 * LAW { Loop } :
 *   LOOP [ WHEN [ { IT } < { BOUND } ] --> [ IT` ] PROCEED to { TURN }
 *          ! [ { IT } > { BOUND } ] AWAIT for [ IT` ]
 *          :: HALT WHEN [ IT` ] = NULL ]
 */
omni_quad_state_t omni_state_step_loop(omni_state_registers_t *regs) {
    if (!regs) return OMNI_STATE_ROLLBACK;

    if (regs->it < regs->bound) {
        regs->it += 1;          /* [ IT` ] */
        regs->turn += 1;        /* PROCEED to { TURN } */
        regs->state = OMNI_STATE_PROCEED;
        return OMNI_STATE_PROCEED;
    } else if (regs->it > regs->bound) {
        regs->state = OMNI_STATE_AWAIT;
        return OMNI_STATE_AWAIT;
    } else {
        /* Terminal Detent reached: IT == BOUND */
        regs->state = OMNI_STATE_HALT;
        return OMNI_STATE_HALT;
    }
}

```

```c
/**
 * @brief Creates a freestanding State engine instance.
 */
omni_state_t omni_state_create(omni_quad_state_t initial_signal, int64_t bound) {
    omni_state_t s;
    s.substrate_trit = OMNI_TRIT_POS;
    s.registers.state = initial_signal;
    s.registers.turn  = 0;
    s.registers.it    = 0;
    s.registers.bound = (bound > 0) ? bound : 4;
    s.is_enclosed     = true;
    return s;
}

/**
 * @brief Resets state registers to initial state.
 */
void omni_state_reset(omni_state_t *s, int64_t bound) {
    if (!s) return;
    s->registers.state = OMNI_STATE_PROCEED;
    s->registers.turn  = 0;
    s->registers.it    = 0;
    s->registers.bound = (bound > 0) ? bound : 4;
}

/** Predicate queries for State engine */
bool omni_state_is_halted(const omni_state_t *s) {
    return (s != NULL && s->registers.state == OMNI_STATE_HALT);
}

bool omni_state_is_proceeding(const omni_state_t *s) {
    return (s != NULL && s->registers.state == OMNI_STATE_PROCEED);
}

bool omni_state_is_awaiting(const omni_state_t *s) {
    return (s != NULL && s->registers.state == OMNI_STATE_AWAIT);
}

bool omni_state_is_rollback(const omni_state_t *s) {
    return (s != NULL && s->registers.state == OMNI_STATE_ROLLBACK);
}

/**
 * @brief Executes the State governance lifecycle across declared operational rolls:
 *   - Roll 1: Inscribe state register from input signal.
 *   - Roll 2: Evaluate caveat guard against discrepancy.
 *   - Roll 3: Execute canonical loop until boundary or detent.
 *   - Roll 4: Govern machine clock output.
 */
omni_quad_state_t omni_state_execute(omni_state_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* ROLL 1: INSCRIBE STATE REGISTER */
    ctx->state_engine.registers.state = ctx->input_signal;

    /* ROLL 2: CAVEAT GUARD AGAINST DISCREPANCY */
    bool signal_match = (ctx->state_engine.registers.state == ctx->input_signal);
    if (omni_guard_assert(signal_match, &ctx->tripwire, &ctx->status) != OMNI_STATE_PROCEED) {
        return OMNI_STATE_ROLLBACK;
    }

    /* ROLL 3: THE CANONICAL LOOP STATEMENT EXECUTION */
    while (ctx->state_engine.registers.state == OMNI_STATE_PROCEED) {
        omni_quad_state_t loop_res = omni_state_step_loop(&ctx->state_engine.registers);
        if (loop_res == OMNI_STATE_HALT) {
            break; /* Clean detent termination */
        } else if (loop_res == OMNI_STATE_AWAIT) {
            break; /* Equilibrium standby */
        }
    }

    /* ROLL 4: GOVERN ENGINE (Inside-out emergence) */
    /* State emerges outward to govern machine clock */

    /* ROLL 5: SEQUENCE TRANSITION (State chains to continuous Timeline) */
    /* Ready to sequence into causal timeline */

    ctx->status = OMNI_STATE_PROCEED;
```

```
    return ctx->status;
}

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK IMPLEMENTATION
 * ( Integrity Seals & Commit Status )
 * ============================================================================ */

/**
 * @brief Verifies all invariants and seals the State commit status.
 */
omni_quad_state_t omni_state_close(omni_state_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* Integrity Check: Proves tripwire is untripped and register bank is valid */
    if (omni_base_verify_integrity(ctx->tripwire, ctx->status) != OMNI_STATE_PROCEED) {
        return omni_commit_rollback(&ctx->tripwire);
    }

    /* Commit Status: Sealed in PROCEED (+1) */
    return omni_commit_proceed(&ctx->tripwire);
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/character.lua

LUA | 149 LINES | 579 WORDS | 5.9 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & PROCESS TAPE MANIFEST
-- ( Process Tape: < Lexicon { Types of Ideas } > & [ Process { Character } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Interprets and executes the atomic lifecycle for Invariant Type < Character >.
--   Acts as the dynamic runtime companion driving C backbone struct `omni_character_t`.
--   - Defined Space: [ 1 Cell ] { 5 Trits :: 1 Byte }
--   - Execution Domain: Lexicon { Types of Ideas }
--   - Witness: Scripture Exodus 31:18 KJV
--     "And he gave unto Moses, when he had made an end of communing with him upon
--      mount Sinai, two tables of testimony, tables of stone, written with the
--      finger of God."
--   - Semantics: { GlyphInscription :: DualSubstrateTransduction :: SingleCellBound }
-- ================================================================================

local TapeEngine = require("lua_tapes.tape_engine")

local CONSTANTS = {
    CELL_CAPACITY       = 1,
    TRITS_PER_CELL      = 5,
    EXEMPLAR_GLYPH_A    = string.byte('A')
}

local CharacterTape = TapeEngine.create_tape({
    metadata = {
        who     = "OmniCode Lexicon",
        what    = "Character --> INSCRIBE { Glyph }",
        when    = "Input Event",
        where   = { character_cell = 1 },
        why     = "Inscribe atomic character type invariant into defined space",
        how     = "+1 : PROCEED",
        witness = "Exodus 31:18 KJV"
    },
    constants = CONSTANTS,
    rolls = {
        -- ------------------------------------------------------------------------
        -- ROLL 1: GLYPH INSCRIPTION (Outside-in imposition onto C cell)
        -- ------------------------------------------------------------------------
        {
            name = "Inscribe",
            action = function(ctx)
                ctx.c_cell.glyph = ctx.glyph
                ctx.c_cell.is_enclosed = true
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 2: DUAL-SUBSTRATE TRANSDUCTION (8 Bits <=> 5 Balanced Trits)
        -- ------------------------------------------------------------------------
        {
            name = "Transduce",
            action = function(ctx)
                local val = ctx.c_cell.glyph
                for i = 1, CONSTANTS.TRITS_PER_CELL do
                    local rem = val % 3
                    val = math.floor(val / 3)
                    if rem == 2 then
                        rem = -1
                        val = val + 1
                    elseif rem == -2 then
                        rem = 1
                        val = val - 1
                    end
                    ctx.c_cell.trits[i] = rem
                end
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 3: IDENTITY & BOUND COMPARISON (Guard Single-Cell Boundary)
```

```
        -- ------------------------------------------------------------------------
        {
            name = "Compare",
            action = function(ctx)
                local identity_ok = (ctx.c_cell.glyph == ctx.glyph)
                local bounds_ok   = (ctx.c_cell.is_enclosed == true)
                return TapeEngine.assert_guard(identity_ok and bounds_ok, ctx,
                                               "Character glyph or single-cell bound violation")
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 4: TARGET PROJECTION (Inside-out emergence to output bus/terminal)
        -- ------------------------------------------------------------------------
        {
            name = "Project",
            action = function(ctx)
                if ctx.target then
                    table.insert(ctx.target, ctx.c_cell.glyph)
                end
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 5: SEQUENCE TRANSITION (Character chains toward continuous Text ribbon)
        -- ------------------------------------------------------------------------
        {
            name = "Sequence",
            action = function(ctx)
                ctx.sequence_ready = true
                return TapeEngine.STATE_PROCEED
            end
        }
    }
})

-- ============================================================================
-- BLOCK 2: OPENING BLOCK
-- ( Context Factory & C Cell Substrate Mount )
-- ============================================================================

---
-- Mounts the tape head, initialises the C cell handle, and arms the tripwire.
-- @param glyph Outside-in byte payload (defaults to ASCII 'A').
-- @param target_buffer Optional destination array (terminal or VGA buffer).
-- @return Initialized context table.
function CharacterTape.open(glyph, target_buffer)
    return {
        glyph    = glyph or CharacterTape.constants.EXEMPLAR_GLYPH_A,
        target   = target_buffer or {},
        c_cell   = {
            glyph       = 0,
            trits       = {0, 0, 0, 0, 0},
            is_enclosed = false
        },
        status   = TapeEngine.STATE_PROCEED,
        tripwire = TapeEngine.TRIPWIRE_ARMED
    }
end

-- ============================================================================
-- BLOCK 4: CLOSING BLOCK
-- ( Fourfold Detent Commit & Interlock Verification )
-- ============================================================================

---
-- Closes and seals the Character tape execution context.
-- @param ctx Active context handle.
-- @return Quad-State commit status and diagnostic seal message.
function CharacterTape.close(ctx)
    return TapeEngine.verify_and_commit(ctx)
end

return CharacterTape
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/container.lua

LUA | 158 LINES | 632 WORDS | 6.5 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & PROCESS TAPE MANIFEST
-- ( Process Tape: < Lexicon { Types of Ideas } > & [ Process { Container } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Interprets and executes the atomic lifecycle for Invariant Type < Container >.
--   Acts as the dynamic runtime companion driving generic C backbone struct `omni_container_t`.
--   - Defined Space: Generic Bounded Memory Envelope / Slice
--   - Ladder Ascension: Build-up with Numbers (/-^)
--   - Witness: Scripture 2 Kings 4:4 KJV
--     "And when thou art come in, thou shalt shut the door upon thee and upon
--      thy sons, and shalt pour out into all those vessels, and thou shalt set
--      aside that which is full."
--   - Semantics: { MemoryCapsule :: BoundedEnvelope :: ContainmentLaw }
-- ================================================================================

local TapeEngine = require("lua_tapes.tape_engine")

local CONSTANTS = {
    EXEMPLAR_UNIT       = 1,
    EXEMPLAR_VESSEL     = 64,
    EXEMPLAR_STOREHOUSE = 4096
}

local ContainerTape = TapeEngine.create_tape({
    metadata = {
        who     = "OmniCode Lexicon",
        what    = "Container --> ENCAPSULATE { Payload }",
        when    = "Allocation Event",
        where   = { container_envelope = 64 },
        why     = "Inscribe bounded container type invariant and storage envelopes",
        how     = "+1 : PROCEED",
        witness = "2 Kings 4:4 KJV"
    },
    constants = CONSTANTS,
    rolls = {
        -- ------------------------------------------------------------------------
        -- ROLL 1: BUILD CAPSULE (Ladder build-up with Numbers)
        -- ------------------------------------------------------------------------
        {
            name = "BuildCapsule",
            action = function(ctx)
                local valid_cap = (ctx.c_container.capacity > 0)
                if valid_cap then
                    ctx.c_container.is_enclosed = true
                end
                return TapeEngine.assert_guard(valid_cap, ctx, "Container capacity must be greater than zero")
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 2: ENCAPSULATION (Inscribe payload into bounded C envelope)
        -- ------------------------------------------------------------------------
        {
            name = "Encapsulate",
            action = function(ctx)
                local items = ctx.payload
                local count = #items

                -- Law { Containment } Guard: Occupancy must never exceed capacity limit
                local within_capacity = (count <= ctx.c_container.capacity)
                if not within_capacity then
                    return TapeEngine.assert_guard(false, ctx, "Payload count exceeds bounded container capacity")
                end

                for i = 1, count do
                    ctx.c_container.buffer[i] = items[i]
                end
                ctx.c_container.occupancy = count
                return TapeEngine.STATE_PROCEED
            end
        },

```

```
        -- ------------------------------------------------------------------------
        -- ROLL 3: IDENTITY & BOUND COMPARISON (Guard bounds under Law { Containment })
        -- ------------------------------------------------------------------------
        {
            name = "GuardBounds",
            action = function(ctx)
                local containment_valid = (ctx.c_container.occupancy <= ctx.c_container.capacity)
                local enclosed_valid    = (ctx.c_container.is_enclosed == true)
                return TapeEngine.assert_guard(containment_valid and enclosed_valid, ctx,
                                               "Core Containment Law violated: Occupancy > Capacity")
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 4: POUR OUT & SET ASIDE (Inside-out emergence)
        -- ------------------------------------------------------------------------
        {
            name = "PourOut",
            action = function(ctx)
                ctx.sealed_vessel = {
                    origin  = ctx.c_container.buffer,
                    target  = ctx.target,
                    sealed  = true
                }
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 5: SEQUENCE TRANSITION (Container chains to composite structure)
        -- ------------------------------------------------------------------------
        {
            name = "Sequence",
            action = function(ctx)
                ctx.structure_ready = true
                return TapeEngine.STATE_PROCEED
            end
        }
    }
})

-- ============================================================================
-- BLOCK 2: OPENING BLOCK
-- ( Context Factory & C Container Slice Substrate Mount )
-- ============================================================================

---
-- Mounts the tape head, initialises the generic C container handle, and arms the tripwire.
-- @param payload Outside-in array of items (e.g. Character bytes or Numbers).
-- @param capacity Maximum number of elements (defaults to 64).
-- @param element_size Byte footprint of a single element (defaults to 1).
-- @return Initialized context table.
function ContainerTape.open(payload, capacity, element_size)
    local cap  = capacity or CONSTANTS.EXEMPLAR_VESSEL
    local e_sz = element_size or 1
    return {
        payload     = payload or {},
        target      = 0x00000000,
        c_container = {
            buffer       = {},
            element_size = e_sz,
            capacity     = cap,
            occupancy    = 0,
            is_enclosed  = false
        },
        status      = TapeEngine.STATE_PROCEED,
        tripwire    = TapeEngine.TRIPWIRE_ARMED
    }
end

-- ============================================================================
-- BLOCK 4: CLOSING BLOCK
-- ( Fourfold Detent Commit & Interlock Verification )
-- ============================================================================

---
-- Closes and seals the Container tape execution context.
-- @param ctx Active context handle.
```

```
-- @return Quad-State commit status and diagnostic seal message.
function ContainerTape.close(ctx)
    return TapeEngine.verify_and_commit(ctx)
end

return ContainerTape
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/number.lua

LUA | 342 LINES | 1731 WORDS | 14.8 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & PROCESS TAPE MANIFEST
-- ( Process Tape: < Lexicon { Types of Ideas } > & [ Process { Number } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Interprets and executes the atomic lifecycle for Invariant Type < Number >.
--   Acts as dynamic runtime companion driving C backbone `omni_number_t`.
--   - Dual-Rail Polar Bit-Planes: 00=0 (Equilibrium), 01=+1, 10=-1, 11=Trap
--   - Directional Partitions: 3-3-2, 3-2-3, 2-3-3 (3-space minimum for Z)
--   - Witness: Scripture Psalm 147:4 KJV
--     "He telleth the number of the stars; he calleth them all by their names."
--   - Semantics: { DualRailBitPlanes :: KinematicTopologies :: ZeroSumALU }
-- ================================================================================

local TapeEngine = require("lua_tapes.tape_engine")

local CONSTANTS = {
    WORD_CAPACITY       = 1,
    TRITS_PER_WORD      = 32,
    BITS_PER_WORD       = 64,
    EXEMPLAR_ZERO       = 0,
    EXEMPLAR_UNITY_POS  = 1,
    EXEMPLAR_UNITY_NEG  = -1,
    EXEMPLAR_PARTITION  = 4096,
    BITPLANE_POS_MASK   = 0x5555555555555555,
    BITPLANE_NEG_MASK   = ~0x5555555555555555,
    TRIT_CODE_ZERO      = 0, -- 0b00: Equilibrium / 0
    TRIT_CODE_POS       = 1, -- 0b01: Positive / +1
    TRIT_CODE_NEG       = 2, -- 0b10: Negative / -1
    TRIT_CODE_NULL      = 3  -- 0b11: Collision / NULL trap
}

local PACK_MODES = {
    DENSE_2222        = 0, -- 2-2-2-2: Standard dense storage (4 trits)
    KINEMATIC_21212   = 1, -- 2-1-2-1-2: Symmetric resting equilibrium
    KINEMATIC_12122   = 2, -- 1-2-1-2-2: Prefix carry/overflow channel
    KINEMATIC_22121   = 3, -- 2-2-1-2-1: Postfix carry/overflow channel
    DIRECTIONAL_332   = 4, -- 3-3-2: Directional vector (X:3, Y:3, Z:2)
    DIRECTIONAL_323   = 5, -- 3-2-3: Directional vector (X:3, Y:2, Z:3)
    DIRECTIONAL_233   = 6  -- 2-3-3: Directional vector (X:2, Y:3, Z:3)
}

local function trit_to_2bit(trit)
    if trit == 0 then return CONSTANTS.TRIT_CODE_ZERO
    elseif trit == 1 then return CONSTANTS.TRIT_CODE_POS
    elseif trit == -1 then return CONSTANTS.TRIT_CODE_NEG
    else return CONSTANTS.TRIT_CODE_NULL end
end

local function two_bit_to_trit(code)
    if code == CONSTANTS.TRIT_CODE_ZERO then return 0, true
    elseif code == CONSTANTS.TRIT_CODE_POS then return 1, true
    elseif code == CONSTANTS.TRIT_CODE_NEG then return -1, true
    else return 0, false end -- NULL collision
end

local function pack_trits_dual_rail(trits)
    local packed = 0
    for i = 1, CONSTANTS.TRITS_PER_WORD do
        local t = trits[i] or 0
        local code = trit_to_2bit(t)
        packed = packed | ((code & 0x03) << ((i - 1) * 2))
    end
    return packed
end

local function unpack_trits_dual_rail(packed, out_trits)
    for i = 1, CONSTANTS.TRITS_PER_WORD do
        local code = (packed >> ((i - 1) * 2)) & 0x03
        local trit, ok = two_bit_to_trit(code)
        if not ok then return false end
        out_trits[i] = trit
    end
```

```
    return true
end

local function negate_dual_rail(packed)
    local pos_rails = packed & CONSTANTS.BITPLANE_POS_MASK
    local neg_rails = packed & CONSTANTS.BITPLANE_NEG_MASK
    return ((pos_rails << 1) & CONSTANTS.BITPLANE_NEG_MASK) |
           ((neg_rails >> 1) & CONSTANTS.BITPLANE_POS_MASK)
end

local function has_null_collision(packed)
    local pos_rails = packed & CONSTANTS.BITPLANE_POS_MASK
    local neg_rails = (packed >> 1) & CONSTANTS.BITPLANE_POS_MASK
    return (pos_rails & neg_rails) ~= 0
end

local function pack_byte(trits, mode)
    mode = mode or PACK_MODES.DENSE_2222
    local b = 0
    if mode == PACK_MODES.DENSE_2222 then
        for i = 1, 4 do
            local code = trit_to_2bit(trits[i] or 0)
            b = b | ((code & 0x03) << ((i - 1) * 2))
        end
    elseif mode == PACK_MODES.KINEMATIC_21212 then
        b = b | (trit_to_2bit(trits[1] or 0) & 0x03)
        b = b | ((trit_to_2bit(trits[2] or 0) & 0x01) << 2)
        b = b | ((trit_to_2bit(trits[2] or 0) & 0x03) << 3)
        b = b | (((trit_to_2bit(trits[2] or 0) >> 1) & 0x01) << 5)
        b = b | ((trit_to_2bit(trits[3] or 0) & 0x03) << 6)
    elseif mode == PACK_MODES.KINEMATIC_12122 then
        b = b | ((trit_to_2bit(trits[1] or 0) & 0x01) << 0)
        b = b | ((trit_to_2bit(trits[1] or 0) & 0x03) << 1)
        b = b | (((trit_to_2bit(trits[2] or 0) >> 1) & 0x01) << 3)
        b = b | ((trit_to_2bit(trits[2] or 0) & 0x03) << 4)
        b = b | ((trit_to_2bit(trits[3] or 0) & 0x03) << 6)
    elseif mode == PACK_MODES.KINEMATIC_22121 then
        b = b | ((trit_to_2bit(trits[1] or 0) & 0x03) << 0)
        b = b | ((trit_to_2bit(trits[2] or 0) & 0x03) << 2)
        b = b | ((trit_to_2bit(trits[3] or 0) & 0x01) << 4)
        b = b | ((trit_to_2bit(trits[3] or 0) & 0x03) << 5)
        b = b | (((trit_to_2bit(trits[3] or 0) >> 1) & 0x01) << 7)
    elseif mode == PACK_MODES.DIRECTIONAL_332 then
        local c1 = trit_to_2bit(trits[1] or 0) & 0x03
        local c2 = trit_to_2bit(trits[2] or 0) & 0x03
        local c3 = trit_to_2bit(trits[3] or 0) & 0x03
        b = b | (c1 << 0) | (c2 << 3) | (c3 << 6)
    elseif mode == PACK_MODES.DIRECTIONAL_323 then
        local c1 = trit_to_2bit(trits[1] or 0) & 0x03
        local c2 = trit_to_2bit(trits[2] or 0) & 0x03
        local c3 = trit_to_2bit(trits[3] or 0) & 0x03
        b = b | (c1 << 0) | (c2 << 3) | (c3 << 5)
    elseif mode == PACK_MODES.DIRECTIONAL_233 then
        local c1 = trit_to_2bit(trits[1] or 0) & 0x03
        local c2 = trit_to_2bit(trits[2] or 0) & 0x03
        local c3 = trit_to_2bit(trits[3] or 0) & 0x03
        b = b | (c1 << 0) | (c2 << 2) | (c3 << 5)
    end
    return b & 0xFF
end

local function unpack_byte(byte_val, mode, out_trits)
    mode = mode or PACK_MODES.DENSE_2222
    if mode == PACK_MODES.DENSE_2222 then
        for i = 1, 4 do
            local code = (byte_val >> ((i - 1) * 2)) & 0x03
            local t, ok = two_bit_to_trit(code)
            if not ok then return false end
            out_trits[i] = t
        end
    elseif mode == PACK_MODES.KINEMATIC_21212 then
        local t1, ok1 = two_bit_to_trit(byte_val & 0x03)
        local t2, ok2 = two_bit_to_trit((byte_val >> 3) & 0x03)
        local t3, ok3 = two_bit_to_trit((byte_val >> 6) & 0x03)
        if not (ok1 and ok2 and ok3) then return false end
        out_trits[1], out_trits[2], out_trits[3], out_trits[4] = t1, t2, t3, 0
    elseif mode == PACK_MODES.KINEMATIC_12122 then
        local t1, ok1 = two_bit_to_trit((byte_val >> 1) & 0x03)
```

```
        local t2, ok2 = two_bit_to_trit((byte_val >> 4) & 0x03)
        local t3, ok3 = two_bit_to_trit((byte_val >> 6) & 0x03)
        if not (ok1 and ok2 and ok3) then return false end
        out_trits[1], out_trits[2], out_trits[3], out_trits[4] = t1, t2, t3, 0
    elseif mode == PACK_MODES.KINEMATIC_22121 then
        local t1, ok1 = two_bit_to_trit((byte_val >> 0) & 0x03)
        local t2, ok2 = two_bit_to_trit((byte_val >> 2) & 0x03)
        local t3, ok3 = two_bit_to_trit((byte_val >> 5) & 0x03)
        if not (ok1 and ok2 and ok3) then return false end
        out_trits[1], out_trits[2], out_trits[3], out_trits[4] = t1, t2, t3, 0
    elseif mode == PACK_MODES.DIRECTIONAL_332 then
        local t1, ok1 = two_bit_to_trit((byte_val >> 0) & 0x03)
        local t2, ok2 = two_bit_to_trit((byte_val >> 3) & 0x03)
        local t3, ok3 = two_bit_to_trit((byte_val >> 6) & 0x03)
        if not (ok1 and ok2 and ok3) then return false end
        out_trits[1], out_trits[2], out_trits[3], out_trits[4] = t1, t2, t3, 0
    elseif mode == PACK_MODES.DIRECTIONAL_323 then
        local t1, ok1 = two_bit_to_trit((byte_val >> 0) & 0x03)
        local t2, ok2 = two_bit_to_trit((byte_val >> 3) & 0x03)
        local t3, ok3 = two_bit_to_trit((byte_val >> 5) & 0x03)
        if not (ok1 and ok2 and ok3) then return false end
        out_trits[1], out_trits[2], out_trits[3], out_trits[4] = t1, t2, t3, 0
    elseif mode == PACK_MODES.DIRECTIONAL_233 then
        local t1, ok1 = two_bit_to_trit((byte_val >> 0) & 0x03)
        local t2, ok2 = two_bit_to_trit((byte_val >> 2) & 0x03)
        local t3, ok3 = two_bit_to_trit((byte_val >> 5) & 0x03)
        if not (ok1 and ok2 and ok3) then return false end
        out_trits[1], out_trits[2], out_trits[3], out_trits[4] = t1, t2, t3, 0
    end
    return true
end

local NumberTape = TapeEngine.create_tape({
    metadata = {
        who     = "OmniCode Lexicon",
        what    = "Number --> MEASURE { Quantity }",
        when    = "Arithmetic Event",
        where   = { number_word = 1 },
        why     = "Inscribe scalar number invariant and arithmetic equilibrium",
        how     = "+1 : PROCEED",
        witness = "Psalm 147:4 KJV"
    },
    constants = CONSTANTS,
    pack_modes = PACK_MODES,
    rolls = {
        -- ------------------------------------------------------------------
        -- ROLL 1: QUANTITY INSCRIPTION (Outside-in imposition onto C word)
        -- ------------------------------------------------------------------
        {
            name = "Inscribe",
            action = function(ctx)
                ctx.c_word.scalar = ctx.quantity
                ctx.c_word.is_enclosed = true
                ctx.c_word.pack_mode = ctx.pack_mode or PACK_MODES.DENSE_2222
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------
        -- ROLL 2: DUAL-SUBSTRATE TRANSDUCTION (64 Bits <=> 32 Balanced Trits)
        -- ------------------------------------------------------------------
        {
            name = "Transduce",
            action = function(ctx)
                local current = ctx.c_word.scalar
                for i = 1, CONSTANTS.TRITS_PER_WORD do
                    if current == 0 then
                        ctx.c_word.trits[i] = 0
                    else
                        local rem = current % 3
                        current = math.floor(current / 3)
                        if rem == 2 then
                            rem = -1
                            current = current + 1
                        elseif rem == -2 then
                            rem = 1
                            current = current - 1
                        end
```

```
                        ctx.c_word.trits[i] = rem
                    end
                end
                -- Inscribe dual-rail bit-planes (64 bits native)
                local p = pack_trits_dual_rail(ctx.c_word.trits)
                ctx.c_word.packed_dual_rail = p
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------
        -- ROLL 3: IDENTITY & BOUND COMPARISON (Guard Single-Word Boundary)
        -- ------------------------------------------------------------------
        {
            name = "Compare",
            action = function(ctx)
                local identity_ok = (ctx.c_word.scalar == ctx.quantity)
                local bounds_ok   = (ctx.c_word.is_enclosed == true)
                local p = ctx.c_word.packed_dual_rail
                local no_null     = not has_null_collision(p)
                local roundtrip   = {}
                local unpack_ok   = unpack_trits_dual_rail(p, roundtrip)
                return TapeEngine.assert_guard(identity_ok and bounds_ok and
                                               no_null and unpack_ok, ctx,
                                               "Number bound or collision")
            end
        },

        -- ------------------------------------------------------------------
        -- ROLL 4: ARITHMETIC EQUILIBRIUM CALCULATION ((+1) plus (-1) <=> 0)
        -- ------------------------------------------------------------------
        {
            name = "Equilibrium",
            action = function(ctx)
                local eq = (1) + (-1)
                local eq_holds = (eq == 0)
                -- Prove SIMD rail-swap involution: -(-x) == x
                local neg = negate_dual_rail(ctx.c_word.packed_dual_rail)
                local neg_neg = negate_dual_rail(neg)
                local rail_involution = (neg_neg == ctx.c_word.packed_dual_rail)
                ctx.equilibrium_proved = eq_holds and rail_involution
                return TapeEngine.assert_guard(ctx.equilibrium_proved, ctx,
                                               "ALU Equilibrium Law violated")
            end
        },

        -- ------------------------------------------------------------------
        -- ROLL 5: SEQUENCE TRANSITION (Number chains toward Vector sequence)
        -- ------------------------------------------------------------------
        {
            name = "Sequence",
            action = function(ctx)
                ctx.vector_ready = true
                return TapeEngine.STATE_PROCEED
            end
        }
    }
})

-- Expose bit-plane and kinematic packing primitives
NumberTape.PACK_MODES             = PACK_MODES
NumberTape.trit_to_2bit           = trit_to_2bit
NumberTape.two_bit_to_trit        = two_bit_to_trit
NumberTape.pack_trits_dual_rail   = pack_trits_dual_rail
NumberTape.unpack_trits_dual_rail = unpack_trits_dual_rail
NumberTape.negate_dual_rail       = negate_dual_rail
NumberTape.has_null_collision     = has_null_collision
NumberTape.pack_byte              = pack_byte
NumberTape.unpack_byte            = unpack_byte

-- ============================================================================
-- BLOCK 2: OPENING BLOCK
-- ( Context Factory & C Word Substrate Mount )
-- ============================================================================

---
-- Mounts the tape head, initialises the C word handle, and arms the tripwire.
-- @param quantity Outside-in scalar integer (defaults to 4096).
```

```
-- @param pack_mode Kinematic packing mode (defaults to DENSE_2222).
-- @return Initialized context table.
function NumberTape.open(quantity, pack_mode)
    local p_mode = pack_mode or PACK_MODES.DENSE_2222
    return {
        quantity           = quantity or CONSTANTS.EXEMPLAR_PARTITION,
        pack_mode          = p_mode,
        c_word             = {
            scalar           = 0,
            trits            = {},
            packed_dual_rail = 0,
            pack_mode        = p_mode,
            is_enclosed      = false
        },
        equilibrium_proved = false,
        status             = TapeEngine.STATE_PROCEED,
        tripwire           = TapeEngine.TRIPWIRE_ARMED
    }
end

-- ============================================================================
-- BLOCK 4: CLOSING BLOCK
-- ( Fourfold Detent Commit & Interlock Verification )
-- ============================================================================

---
-- Closes and seals the Number tape execution context.
-- @param ctx Active context handle.
-- @return Quad-State commit status and diagnostic seal message.
function NumberTape.close(ctx)
    return TapeEngine.verify_and_commit(ctx)
end

return NumberTape
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/site.lua

LUA | 148 LINES | 565 WORDS | 6.0 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & PROCESS TAPE MANIFEST
-- ( Process Tape: < Lexicon { Types of Ideas } > & [ Process { Site } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Interprets and executes the atomic lifecycle for Invariant Type < Site >.
--   Acts as the dynamic runtime companion driving C backbone struct `omni_site_t`.
--   - Defined Space: [ 1 Address ] { 64 Bits / Silicon Coordinate }
--   - Ladder Ascension: Build-up with Numbers (/-^)
--   - Witness: Scripture Genesis 28:16 KJV
--     "And Jacob awaked out of his sleep, and he said, Surely the Lord is in
--      this place; and I knew it not."
--   - Semantics: { SiliconCoordinate :: HardwareRoutingLaw :: NullPointerDetent }
-- ================================================================================

local TapeEngine = require("lua_tapes.tape_engine")

local CONSTANTS = {
    SITE_NULL   = 0x00000000,
    SITE_MBR    = 0x00007C00,
    SITE_COM1   = 0x000003F8,
    SITE_VGA    = 0x000B8000
}

local SiteTape = TapeEngine.create_tape({
    metadata = {
        who     = "OmniCode Lexicon",
        what    = "Site --> LOCATE { PhysicalAddress }",
        when    = "Addressing Event",
        where   = { silicon_coordinate = 1 },
        why     = "Inscribe physical site address invariant and hardware MMIO coordinates",
        how     = "+1 : PROCEED",
        witness = "Genesis 28:16 KJV"
    },
    constants = CONSTANTS,
    rolls = {
        -- ------------------------------------------------------------------------
        -- ROLL 1: LOCATE (Ladder build-up with Numbers)
        -- ------------------------------------------------------------------------
        {
            name = "Locate",
            action = function(ctx)
                local not_null = (ctx.coordinate ~= CONSTANTS.SITE_NULL)
                if not_null then
                    ctx.c_site.is_enclosed = true
                end
                return TapeEngine.assert_guard(not_null, ctx,
                                               "Null pointer address cannot be located as a valid silicon site")
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 2: BIND COORDINATE (Inscribe address & resolve RAM vs MMIO routing)
        -- ------------------------------------------------------------------------
        {
            name = "BindCoordinate",
            action = function(ctx)
                ctx.c_site.coordinate = ctx.coordinate
                -- Hardware Routing Law: COM1, VGA, or >= 0xA0000 route to MMIO; otherwise RAM
                if ctx.coordinate == CONSTANTS.SITE_COM1 or
                   ctx.coordinate == CONSTANTS.SITE_VGA  or
                   ctx.coordinate >= 0x000A0000 then
                    ctx.c_site.routing = "HARDWARE_MMIO (::@)"
                else
                    ctx.c_site.routing = "PHYSICAL_RAM (@)"
                end
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 3: IDENTITY & NULL GUARD (Guard Non-Null Validity)
        -- ------------------------------------------------------------------------
```

```
        {
            name = "GuardAddress",
            action = function(ctx)
                local identity_ok = (ctx.c_site.coordinate == ctx.coordinate)
                local valid_site  = (ctx.c_site.coordinate ~= CONSTANTS.SITE_NULL and ctx.c_site.is_enclosed)
                return TapeEngine.assert_guard(identity_ok and valid_site, ctx,
                                               "Site address failed non-null validation check")
            end
        },

        -- ------------------------------------------------------------------
        -- ROLL 4: ROUTE & PROJECT (Inside-out emergence)
        -- ------------------------------------------------------------------
        {
            name = "Route",
            action = function(ctx)
                ctx.memory_anchor = {
                    coordinate = ctx.c_site.coordinate,
                    routing    = ctx.c_site.routing,
                    anchored   = true
                }
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------
        -- ROLL 5: SEQUENCE TRANSITION (Site chains to Memory Map)
        -- ------------------------------------------------------------------
        {
            name = "Sequence",
            action = function(ctx)
                ctx.memory_map_ready = true
                return TapeEngine.STATE_PROCEED
            end
        }
    }
})

-- ============================================================================
-- BLOCK 2: OPENING BLOCK
-- ( Context Factory & C Site Substrate Mount )
-- ============================================================================

---
-- Mounts the tape head, initialises the C site handle, and arms the tripwire.
-- @param coordinate Outside-in 64-bit silicon address (defaults to VGA buffer).
-- @return Initialized context table.
function SiteTape.open(coordinate)
    return {
        coordinate = coordinate or CONSTANTS.SITE_VGA,
        c_site     = {
            coordinate  = 0,
            routing     = "UNRESOLVED",
            is_enclosed = false
        },
        status     = TapeEngine.STATE_PROCEED,
        tripwire   = TapeEngine.TRIPWIRE_ARMED
    }
end

-- ============================================================================
-- BLOCK 4: CLOSING BLOCK
-- ( Fourfold Detent Commit & Interlock Verification )
-- ============================================================================

---
-- Closes and seals the Site tape execution context.
-- @param ctx Active context handle.
-- @return Quad-State commit status and diagnostic seal message.
function SiteTape.close(ctx)
    return TapeEngine.verify_and_commit(ctx)
end

return SiteTape
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/state.lua

LUA | 183 LINES | 697 WORDS | 7.1 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & PROCESS TAPE MANIFEST
-- ( Process Tape: < Lexicon { Types of Ideas } > & [ Process { State } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Interprets and executes the atomic lifecycle for Invariant Type < State >.
--   Acts as the dynamic runtime companion driving C backbone struct `omni_state_t`.
--   - Defined Space: [ 1 Trit ] { 2 Bits / Quad-State Causal Register }
--   - Named Registers: [ STATE TURN BOUND IT ]
--   - Witness: Scripture Ecclesiastes 3:1 KJV
--     "To every thing there is a season, and a time to every purpose under
--      the heaven:"
--   - Semantics: { QuadStateTruth :: CausalClock :: CanonicalLoopLaw }
-- ================================================================================

local TapeEngine = require("lua_tapes.tape_engine")

local CONSTANTS = {
    STATE_PROCEED  = TapeEngine.STATE_PROCEED,
    STATE_AWAIT    = TapeEngine.STATE_AWAIT,
    STATE_ROLLBACK = TapeEngine.STATE_ROLLBACK,
    STATE_HALT     = TapeEngine.STATE_HALT,
    DEFAULT_BOUND  = 4
}

local StateTape = TapeEngine.create_tape({
    metadata = {
        who     = "OmniCode Lexicon",
        what    = "State --> GOVERN { CausalTruth }",
        when    = "Control Event",
        where   = { state_register = 1 },
        why     = "Inscribe quad-state truth invariant and execute canonical loops",
        how     = "+1 : PROCEED",
        witness = "Ecclesiastes 3:1 KJV"
    },
    constants = CONSTANTS,
    rolls = {
        -- ------------------------------------------------------------------------
        -- ROLL 1: INSCRIBE STATE REGISTER
        -- ------------------------------------------------------------------------
        {
            name = "Inscribe",
            action = function(ctx)
                ctx.c_state.registers.state = ctx.signal
                ctx.c_state.is_enclosed = true
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 2: CAVEAT GUARD AGAINST DISCREPANCY
        -- ------------------------------------------------------------------------
        {
            name = "GuardSignal",
            action = function(ctx)
                local signal_match = (ctx.c_state.registers.state == ctx.signal)
                return TapeEngine.assert_guard(signal_match, ctx,
                                               "State register truth signal discrepancy")
            end
        },

        -- ------------------------------------------------------------------------
        -- ROLL 3: CANONICAL LOOP STEPPER (Iterative Quad-State Cadence)
        -- ------------------------------------------------------------------------
        {
            name = "LoopCadence",
            action = function(ctx)
                local regs = ctx.c_state.registers
                while regs.state == TapeEngine.STATE_PROCEED do
                    if regs.it < regs.bound then
                        regs.it = regs.it + 1
                        regs.turn = regs.turn + 1
                        regs.state = TapeEngine.STATE_PROCEED
```

```
                    elseif regs.it > regs.bound then
                        regs.state = TapeEngine.STATE_AWAIT
                        break
                    else
                        -- Terminal Detent reached: IT == BOUND
                        regs.state = TapeEngine.STATE_HALT
                        break
                    end
                end
                return regs.state
            end
        },

        -- ------------------------------------------------------------------
        -- ROLL 4: GOVERN ENGINE (Inside-out emergence to machine clock)
        -- ------------------------------------------------------------------
        {
            name = "GovernClock",
            action = function(ctx)
                ctx.clock_active = (ctx.c_state.registers.state ~= TapeEngine.STATE_HALT)
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------
        -- ROLL 5: SEQUENCE TRANSITION (State chains to continuous Timeline)
        -- ------------------------------------------------------------------
        {
            name = "Sequence",
            action = function(ctx)
                ctx.timeline_ready = true
                return TapeEngine.STATE_PROCEED
            end
        }
    }
})

-- ============================================================================
-- BLOCK 2: OPENING BLOCK
-- ( Context Factory & C State Substrate Mount )
-- ============================================================================

---
-- Mounts the tape head, initialises the C state register bank handle, and arms the tripwire.
-- @param signal Outside-in truth signal (defaults to PROCEED).
-- @param bound Maximum iteration threshold (defaults to 4).
-- @return Initialized context table.
function StateTape.open(signal, bound)
    local sig = signal or CONSTANTS.STATE_PROCEED
    local bnd = bound or CONSTANTS.DEFAULT_BOUND
    return {
        signal   = sig,
        c_state  = {
            substrate_trit = 1,
            registers = {
                state = sig,
                turn  = 0,
                bound = bnd,
                it    = 0
            },
            is_enclosed = false
        },
        status   = TapeEngine.STATE_PROCEED,
        tripwire = TapeEngine.TRIPWIRE_ARMED
    }
end

-- ============================================================================
-- BLOCK 3: OPERATIONAL PRIMITIVES
-- ( Single-Step Loop Law Execution )
-- ============================================================================

---
-- Single-steps the Canonical Loop Law across named registers:
--   LOOP [ WHEN [ { IT } < { BOUND } ] --> [ IT` ] PROCEED to { TURN }
--          ! [ { IT } > { BOUND } ] AWAIT for [ IT` ]
--          :: HALT WHEN [ IT` ] = NULL ]
-- @param ctx Context handle holding register bank.
```

```
-- @return New Quad-State condition after the single step.
function StateTape.step_loop(ctx)
    local regs = ctx.c_state.registers
    if regs.it < regs.bound then
        regs.it = regs.it + 1
        regs.turn = regs.turn + 1
        regs.state = TapeEngine.STATE_PROCEED
        return TapeEngine.STATE_PROCEED
    elseif regs.it > regs.bound then
        regs.state = TapeEngine.STATE_AWAIT
        return TapeEngine.STATE_AWAIT
    else
        regs.state = TapeEngine.STATE_HALT
        return TapeEngine.STATE_HALT
    end
end

-- ============================================================================
-- BLOCK 4: CLOSING BLOCK
-- ( Fourfold Detent Commit & Interlock Verification )
-- ============================================================================

---
-- Closes and seals the State tape execution context.
-- @param ctx Active context handle.
-- @return Quad-State commit status and diagnostic seal message.
function StateTape.close(ctx)
    return TapeEngine.verify_and_commit(ctx)
end

return StateTape
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/lua_tapes/tape_engine.lua

LUA | 223 LINES | 957 WORDS | 8.6 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & ENGINE TOPOLOGY
-- ( Engine Core: < Lexicon { Types of Ideas } > & [ Process { Tape Engine } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   The interpreted execution ribbon companion to the OmniCode C Backbone.
--   Drives the dynamic temporal causation, coroutines, and roll kinematics:
--   - Coroutine-based fiber stepping (yielding on AWAIT, resuming on tick)
--   - Pre-roll and post-roll dynamic event hooks
--   - State snapshotting and active rollback unwinding
--   - Structured handles over C memory layouts and invariants
--   - Witness: Scripture Proverbs 16:9 KJV
--     "A man's heart deviseth his way: but the LORD directeth his steps."
--   - Semantics: { TapeKinematics :: CoroutineFiber :: RollHooks :: QuadStateDetents }
-- ================================================================================

local TapeEngine = {
    -- Quad-State Truth Constants
    STATE_ROLLBACK = -1,   -- -1 : Retroactive Reversal / Error Unwind
    STATE_AWAIT    =  0,   --  0 : Equilibrium Standby / Hold Pending
    STATE_PROCEED  =  1,   -- +1 : Forward Causation / Affirmative Advancement
    STATE_HALT     = "NULL",-- NULL : Terminal Detent / Stop Processor Clock

    -- Tripwire Interlock States
    TRIPWIRE_DISARMED = 0,
    TRIPWIRE_ARMED    = 1,
    TRIPWIRE_TRIPPED  = 2
}

-- Detect optional LuaJIT FFI for direct zero-copy C backbone linkage
local has_ffi, ffi = pcall(require, "ffi")
TapeEngine.has_ffi = has_ffi
TapeEngine.ffi = has_ffi and ffi or nil

-- ================================================================================
-- BLOCK 2: OPENING BLOCK
-- ( Subsystem Initialization, Preconditions & Tape Mounting )
-- ================================================================================

---
-- Deep copy snapshot utility for transactional rollback safety.
-- @param ctx The context table to clone.
-- @return Independent deep copy of the context.
local function make_snapshot(ctx)
    local snap = {}
    for k, v in pairs(ctx) do
        if type(v) == "table" then
            snap[k] = make_snapshot(v)
        else
            snap[k] = v
        end
    end
    return snap
end

---
-- Creates and mounts an interpreted Tape Process Ribbon.
-- @param spec Table containing metadata, constants, and declared rolls.
-- @return Mounted tape object.
function TapeEngine.create_tape(spec)
    local tape = {
        metadata    = spec.metadata or {},
        constants   = spec.constants or {},
        rolls       = spec.rolls or {},
        hooks       = {
            before_roll = {},
            after_roll  = {},
            on_rollback = {}
        },
        snapshots   = {},
        history     = {}
    }

```

```lua
    ---
    -- Registers a dynamic hook invoked immediately before a roll executes.
    function tape:before(fn)
        table.insert(self.hooks.before_roll, fn)
        return self
    end

    ---
    -- Registers a dynamic hook invoked immediately after a roll executes.
    function tape:after(fn)
        table.insert(self.hooks.after_roll, fn)
        return self
    end

    ---
    -- Registers a rollback handler invoked when a roll trips the interlock.
    function tape:on_rollback(fn)
        table.insert(self.hooks.on_rollback, fn)
        return self
    end

    ---
    -- Restores a previously captured execution snapshot into the active context.
    function tape:restore_snapshot(ctx, roll_index)
        local snap = self.snapshots[roll_index]
        if snap then
            for k, v in pairs(snap) do
                ctx[k] = v
            end
        end
    end

-- ==============================================================================
-- BLOCK 3: BODY BLOCK
-- ( Coroutine Kinematics, Roll Stepping & Hook Dispatch )
-- ==============================================================================

    ---
    -- Spawns a coroutine fiber that steps through the declared rolls.
    -- Yields on AWAIT (0), halts on HALT (NULL), and unwinds on ROLLBACK (-1).
    -- @param ctx Context handle holding the C struct proxy and tripwires.
    -- @return Lua coroutine object.
    function tape:spawn_coroutine(ctx)
        return coroutine.create(function()
            ctx.status = TapeEngine.STATE_PROCEED
            ctx.tripwire = TapeEngine.TRIPWIRE_ARMED

            for i, roll in ipairs(self.rolls) do
                -- 1. Snapshot state prior to roll execution for transaction rollback
                self.snapshots[i] = make_snapshot(ctx)

                -- 2. Dispatch pre-roll hooks
                for _, h in ipairs(self.hooks.before_roll) do
                    h(self, i, roll.name, ctx)
                end

                -- 3. Execute the declared roll action
                local status = roll.action(ctx)
                ctx.status = status or ctx.status

                -- 4. Dispatch post-roll hooks
                for _, h in ipairs(self.hooks.after_roll) do
                    h(self, i, roll.name, ctx, ctx.status)
                end

                -- Record execution trace
                table.insert(self.history, { roll = i, name = roll.name, status = ctx.status })

                -- 5. Quad-State transition evaluation
                if ctx.status == TapeEngine.STATE_AWAIT then
                    -- Yield execution back to host/game loop until external condition resolves
                    coroutine.yield("AWAIT", i, roll.name, ctx)
                elseif ctx.status == TapeEngine.STATE_ROLLBACK then
                    -- Invariant violation: trip safety interlock and trigger rollback unwinding
                    ctx.tripwire = TapeEngine.TRIPWIRE_TRIPPED
                    for _, h in ipairs(self.hooks.on_rollback) do
                        h(self, i, roll.name, ctx, self.snapshots[i])
                    end
```

```
                    return TapeEngine.STATE_ROLLBACK, "Tape rolled back at roll " .. i .. " [" .. roll.name .. "]"
                elseif ctx.status == TapeEngine.STATE_HALT then
                    -- Terminal Detent reached
                    return TapeEngine.STATE_HALT, "Tape reached terminal detent (HALT:NULL)"
                end
            end

            ctx.tripwire = TapeEngine.TRIPWIRE_DISARMED
            return TapeEngine.STATE_PROCEED, "Tape sealed in PROCEED (+1)"
        end)
    end

    ---
    -- Executes all rolls in a synchronous loop, automatically resuming AWAITs.
    -- @param ctx Context handle.
    -- @return Final Quad-State status code and descriptive seal message.
    function tape:execute_all(ctx)
        local co = self:spawn_coroutine(ctx)
        while coroutine.status(co) ~= "dead" do
            local ok, event, roll_idx, roll_name, curr_ctx = coroutine.resume(co)
            if not ok then
                error("Tape execution fatal error: " .. tostring(event))
            end
            if event == "AWAIT" then
                -- In synchronous batch mode, acknowledge standby and resume
                curr_ctx.status = TapeEngine.STATE_PROCEED
            elseif event == TapeEngine.STATE_PROCEED or event == TapeEngine.STATE_HALT or event ==
TapeEngine.STATE_ROLLBACK then
                return event, roll_idx
            end
        end
        return ctx.status
    end

    return tape
end

-- ============================================================================
-- BLOCK 4: CLOSING BLOCK
-- ( Integrity Seals & Commit Verification )
-- ============================================================================

---
-- Evaluates an invariant guard condition inside a roll action.
-- Trips the tripwire and triggers ROLLBACK on condition failure.
-- @param condition Boolean expression to verify.
-- @param ctx Context handle.
-- @param error_reason Message recorded upon tripwire trigger.
-- @return STATE_PROCEED if condition is true; STATE_ROLLBACK otherwise.
function TapeEngine.assert_guard(condition, ctx, error_reason)
    if condition then
        return TapeEngine.STATE_PROCEED
    else
        ctx.tripwire = TapeEngine.TRIPWIRE_TRIPPED
        ctx.error_reason = error_reason or "Invariant guard violation"
        return TapeEngine.STATE_ROLLBACK
    end
end

---
-- Verifies the final integrity of a context before closing commit.
-- @param ctx Context handle.
-- @return STATE_PROCEED if clean; STATE_ROLLBACK if tripped.
function TapeEngine.verify_and_commit(ctx)
    if ctx.tripwire == TapeEngine.TRIPWIRE_TRIPPED or ctx.status == TapeEngine.STATE_ROLLBACK then
        return TapeEngine.STATE_ROLLBACK, "Commit rejected: tripwire tripped"
    end
    ctx.tripwire = TapeEngine.TRIPWIRE_DISARMED
    return TapeEngine.STATE_PROCEED, "Context sealed in PROCEED (+1)"
end

return TapeEngine
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/integrated/test_omni_lib.lua

LUA | 152 LINES | 634 WORDS | 7.0 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & TEST ORCHESTRATOR TOPOLOGY
-- ( Test Orchestrator: < Lexicon { Types of Ideas } > & [ Multi-File Testbed (Lua) ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Primary test orchestrator for the OmniCode Standard Library Lua Process Ribbons.
--   Executes multi-file unit test suites followed by integrated cross-type pipelines:
--   1. Unit Suite: Character Tape (Exodus 31:18 KJV)
--   2. Unit Suite: Number Tape (Psalm 147:4 KJV)
--   3. Unit Suite: Container Tape (2 Kings 4:4 KJV)
--   4. Unit Suite: Site Tape (Genesis 28:16 KJV)
--   5. Unit Suite: State Tape (Ecclesiastes 3:1 KJV)
--   6. Safety Interlock & Transactional Rollback (Proverbs 16:9 KJV)
--   7. Coroutine Fiber Stepping & AWAIT Yield/Resume
--   8. Integrated Composition: Container<Character> Traversed via State Loop
--   - Witness: Scripture Proverbs 24:27 KJV
--     "Prepare thy work without, and make it fit for thyself in the field;
--      and afterwards build thine house."
--   - Semantics: { MultiFileTestbed :: CrossTypeIntegration :: DetentVerification }
-- ================================================================================

package.path         = package.path ..
";./?.lua;./lua_tapes/?.lua;./tests/lua/unit/?.lua;./unit/?.lua;../unit/?.lua;../?.lua;../../?.lua"

local TestCommon     = require("test_common")
local TapeEngine     = require("lua_tapes.tape_engine")
local CharacterSuite = require("test_character")
local NumberSuite    = require("test_number")
local ContainerSuite = require("test_container")
local SiteSuite      = require("test_site")
local StateSuite     = require("test_state")

local CharacterTape  = require("lua_tapes.character")
local ContainerTape  = require("lua_tapes.container")
local SiteTape       = require("lua_tapes.site")
local StateTape      = require("lua_tapes.state")

print("================================================================================")
print("   OMNICODE STANDARD LIBRARY LUA ENGINE: MULTI-FILE TEST HARNESS")
print("================================================================================\n")

-- --------------------------------------------------------------------------------
-- 1. EXECUTE UNIT SUITES
-- --------------------------------------------------------------------------------
local suites = {
    CharacterSuite,
    NumberSuite,
    ContainerSuite,
    SiteSuite,
    StateSuite
}

for _, s in ipairs(suites) do
    local ok = s.run()
    if not ok then
        print(string.format("\n[ABORT] Unit suite failed! (%d failure(s))", TestCommon.failures))
        os.exit(1)
    end
    print("")
end

-- --------------------------------------------------------------------------------
-- 2. SAFETY INTERLOCK & TRANSACTIONAL ROLLBACK TEST
-- --------------------------------------------------------------------------------
print("[SUITE: Safety Interlock & Transactional Rollback]")
local rollback_caught = false
SiteTape:on_rollback(function(tape, roll_idx, roll_name, ctx, snapshot)
    rollback_caught = true
end)

local bad_site_ctx = SiteTape.open(0x00000000)
local bad_status, bad_msg = SiteTape:execute_all(bad_site_ctx)
TestCommon.assert_eq(bad_status, TapeEngine.STATE_ROLLBACK, "Expected STATE_ROLLBACK (-1)")
```

```
TestCommon.assert_eq(bad_site_ctx.tripwire, TapeEngine.TRIPWIRE_TRIPPED, "Expected TRIPWIRE_TRIPPED")
TestCommon.assert(rollback_caught == true, "Expected on_rollback hook to fire")
print(string.format("    -> Rollback Interlock: Status=%s, Tripwire=%d (%s) [PASS]\n",
    tostring(bad_status), bad_site_ctx.tripwire, bad_site_ctx.error_reason))

-- ------------------------------------------------------------------------------
-- 3. COROUTINE FIBER STEPPING & YIELD/RESUME TEST
-- ------------------------------------------------------------------------------
print("[SUITE: Coroutine Fiber Stepping & AWAIT Yield/Resume]")
local FiberTape = TapeEngine.create_tape({
    metadata = { name = "FiberSteppingTape" },
    rolls = {
        {
            name = "PhaseOne",
            action = function(ctx)
                ctx.counter = ctx.counter + 1
                return TapeEngine.STATE_AWAIT -- Request coroutine yield
            end
        },
        {
            name = "PhaseTwo",
            action = function(ctx)
                ctx.counter = ctx.counter * 10
                return TapeEngine.STATE_PROCEED
            end
        }
    }
})

local fiber_ctx = { counter = 5, status = TapeEngine.STATE_PROCEED, tripwire = TapeEngine.TRIPWIRE_ARMED }
local co = FiberTape:spawn_coroutine(fiber_ctx)

-- First resume: yields on PhaseOne AWAIT
local ok1, ev1, roll1, name1 = coroutine.resume(co)
TestCommon.assert(ok1 and ev1 == "AWAIT" and fiber_ctx.counter == 6, "Expected AWAIT yield at counter=6")
print(string.format("    -> Yielded on '%s' at roll %d [%s] (Counter=%d) [PASS]",
    ev1, roll1, name1, fiber_ctx.counter))

-- Reset status from AWAIT to PROCEED for phase two resume
fiber_ctx.status = TapeEngine.STATE_PROCEED
local ok2, ev2, msg2 = coroutine.resume(co)
TestCommon.assert(ok2 and ev2 == TapeEngine.STATE_PROCEED and fiber_ctx.counter == 60, "Expected PROCEED at counter=60")
print(string.format("    -> Resumed to Completion: Status=%s (Counter=%d, %s) [PASS]\n",
    tostring(ev2), fiber_ctx.counter, msg2))

-- ------------------------------------------------------------------------------
-- 4. CROSS-TYPE COMPOSITION PIPELINE
-- ------------------------------------------------------------------------------
print("[SUITE: Cross-Type Composition Pipeline]")
local word_bytes = {
    string.byte('O'), string.byte('M'), string.byte('N'), string.byte('I'),
    string.byte('C'), string.byte('O'), string.byte('D'), string.byte('E')
}
local container_ctx = ContainerTape.open(word_bytes, 32, 1)
local cont_status, cont_msg = ContainerTape:execute_all(container_ctx)
TestCommon.assert_eq(cont_status, TapeEngine.STATE_PROCEED, "Container execute must PROCEED")

io.write("    -> Driving Traversal via Canonical State Loop:\n       Transduced Output: \"")
local state_ctx = StateTape.open(TapeEngine.STATE_PROCEED, container_ctx.c_container.occupancy)

while state_ctx.c_state.registers.state == TapeEngine.STATE_PROCEED do
    local idx = state_ctx.c_state.registers.it + 1
    local ch = container_ctx.c_container.buffer[idx]
    if ch then
        io.write(string.char(ch))
    end
    StateTape.step_loop(state_ctx)
end
io.write("\"\n")

TestCommon.assert_eq(state_ctx.c_state.registers.state, TapeEngine.STATE_HALT, "State must reach HALT:NULL detent")
print(string.format("       Final Detent: %s (Detent at IT=%d, TURN=%d) [PASS]\n",
    tostring(state_ctx.c_state.registers.state),
    state_ctx.c_state.registers.it,
    state_ctx.c_state.registers.turn))

print("================================================================================")
```

```
print("   [ALL LUA ENGINE TESTS PASS]: FULL 4-BLOCK PARITY SEALED IN PROCEED (+1)")
print("================================================================================")
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_character.lua

LUA | 89 LINES | 437 WORDS | 4.1 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & TEST TOPOLOGY
-- ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Character Tape } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Validates the interpreted process ribbon < Character Tape > in isolation:
--   - Bijective round-trip transduction (8 bits <=> 5 balanced trits)
--   - Strict single-cell enclosure boundary laws
--   - Dynamic pre/post roll hook observation
--   - Full 4-block lifecycle execution (open, execute_all, close)
--   - Witness: Scripture Exodus 31:18 KJV
--     "And he gave unto Moses, when he had made an end of communing with him upon
--      mount Sinai, two tables of testimony, tables of stone, written with the
--      finger of God."
--   - Semantics: { GlyphFidelity :: BalancedTritLaw :: SingleCellEnclosure }
-- ================================================================================

package.path        = package.path ..
";./?.lua;./lua_tapes/?.lua;./tests/lua/unit/?.lua;./unit/?.lua;../?.lua;../../?.lua"

local TestCommon    = require("test_common")
local TapeEngine    = require("lua_tapes.tape_engine")
local CharacterTape = require("lua_tapes.character")

local Suite         = {}

function Suite.test_character_transduction()
    local glyphs = { string.byte('A'), string.byte('Z'), string.byte('0'), string.byte('9'), 32, 0, 70, 100, 121 }
    for _, g in ipairs(glyphs) do
        local ctx = CharacterTape.open(g)
        local status = CharacterTape:execute_all(ctx)
        TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED, "Execution status must PROCEED (+1)")
        TestCommon.assert_eq(ctx.c_cell.glyph, g, "Context cell glyph must match input")
        TestCommon.assert(ctx.c_cell.is_enclosed == true, "Context cell must be enclosed")

        -- Reconstruct byte from trits: sum_{i=1..5} trits[i] * 3^(i-1)
        local reconstructed = 0
        local power = 1
        for i = 1, #ctx.c_cell.trits do
            local t = ctx.c_cell.trits[i]
            TestCommon.assert(t >= -1 and t <= 1, "Trit must be in {-1, 0, +1}")
            reconstructed = reconstructed + t * power
            power = power * 3
        end
        TestCommon.assert_eq(reconstructed, g, "Reconstructed byte from trits must match input")
    end
end

function Suite.test_character_dynamic_hooks()
    local pre_hooks = 0
    local post_hooks = 0

    CharacterTape:before(function() pre_hooks = pre_hooks + 1 end)
    CharacterTape:after(function() post_hooks = post_hooks + 1 end)

    local ctx = CharacterTape.open(string.byte('Q'))
    local status = CharacterTape:execute_all(ctx)
    TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED, "Execution must PROCEED")
    TestCommon.assert(pre_hooks >= 5, "Must trigger pre-roll hooks for all 5 rolls")
    TestCommon.assert(post_hooks >= 5, "Must trigger post-roll hooks for all 5 rolls")
end

function Suite.test_character_lifecycle_rolls()
    local ctx = CharacterTape.open(string.byte('C'))
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_ARMED, "Tripwire must be ARMED on open")

    local status = CharacterTape:execute_all(ctx)
    TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED, "Execution must PROCEED")

    local close_st, close_msg = CharacterTape.close(ctx)
    TestCommon.assert_eq(close_st, TapeEngine.STATE_PROCEED, "Close must seal in PROCEED (+1)")
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_DISARMED, "Tripwire must be DISARMED on close")
end
```

```
73
74  function Suite.run()
75      print("[SUITE: Character Process Ribbon (Lua)]")
76      local ok1 = TestCommon.run_test("Bijective Round-Trip Transduction", Suite.test_character_transduction)
77      local ok2 = TestCommon.run_test("Dynamic Pre/Post Roll Hooks", Suite.test_character_dynamic_hooks)
78      local ok3 = TestCommon.run_test("4-Block Lifecycle Rolls", Suite.test_character_lifecycle_rolls)
79      return ok1 and ok2 and ok3
80  end
81
82  -- Allow standalone execution
83  if pcall(debug.getlocal, 4, 1) == false then
84      local ok = Suite.run()
85      os.exit(ok and 0 or 1)
86  end
87
88  return Suite
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_common.lua

LUA | 49 LINES | 188 WORDS | 1.7 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & TEST HARNESS TOPOLOGY
-- ( Test Framework: < OmniCode Standard Library Lua Testbed > )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Shared test harness for the OmniCode Standard Library Lua Process Ribbons.
--   Provides structured assertions, test suite runner helpers, and tripwire checks.
--   - Witness: Scripture Proverbs 24:27 KJV
--     "Prepare thy work without, and make it fit for thyself in the field;
--      and afterwards build thine house."
--   - Semantics: { AssertionInterlock :: GranularFaultIsolation :: MultiFileTestbed }
-- ================================================================================

local TestCommon = {
    failures = 0,
    tests_run = 0
}

function TestCommon.assert(condition, msg)
    TestCommon.tests_run = TestCommon.tests_run + 1
    if not condition then
        TestCommon.failures = TestCommon.failures + 1
        error("    [FAIL] Assertion failed: " .. tostring(msg), 2)
    end
end

function TestCommon.assert_eq(actual, expected, msg)
    TestCommon.tests_run = TestCommon.tests_run + 1
    if actual ~= expected then
        TestCommon.failures = TestCommon.failures + 1
        error(string.format("    [FAIL] %s: Expected [%s], got [%s]", tostring(msg), tostring(expected), tostring(actual)), 2)
    end
end

function TestCommon.run_test(name, fn)
    io.write(string.format("    -> %s ... ", name))
    local ok, err = pcall(fn)
    if ok then
        print("[PASS]")
        return true
    else
        print("[FAILED]")
        print(tostring(err))
        return false
    end
end

return TestCommon
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_container.lua

LUA | 79 LINES | 397 WORDS | 3.7 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & TEST TOPOLOGY
-- ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Container Tape } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Validates the interpreted process ribbon < Container Tape > in isolation:
--   - Bounded memory capsule build-up
--   - Payload encapsulation into bounded envelope
--   - Law of Containment verification (Occupancy <= Capacity)
--   - Overflow detection and tripwire interlock rollback
--   - Full 4-block lifecycle execution (open, execute_all, close)
--   - Witness: Scripture 2 Kings 4:4 KJV
--     "And when thou art come in, thou shalt shut the door upon thee and upon
--      thy sons, and shalt pour out into all those vessels, and thou shalt set
--      aside that which is full."
--   - Semantics: { MemoryCapsule :: BoundedEnvelope :: ContainmentLaw }
-- ================================================================================

package.path        = package.path .. ";./?.lua;./lua_tapes/?.lua;./tests/lua/unit/?.lua;./unit/?.lua;../?.lua;../../?.lua"

local TestCommon    = require("test_common")
local TapeEngine    = require("lua_tapes.tape_engine")
local ContainerTape = require("lua_tapes.container")

local Suite         = {}

function Suite.test_container_encapsulation()
    local payload = { 10, 20, 30, 40, 50 }
    local ctx = ContainerTape.open(payload, 16, 1)

    local status = ContainerTape:execute_all(ctx)
    TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED, "Execution must PROCEED (+1)")
    TestCommon.assert_eq(ctx.c_container.occupancy, 5, "Container occupancy must be 5")
    TestCommon.assert_eq(ctx.c_container.capacity, 16, "Container capacity must be 16")
    TestCommon.assert(ctx.c_container.is_enclosed == true, "Container must be enclosed")
    TestCommon.assert(ctx.sealed_vessel ~= nil, "Sealed vessel must emerge from Roll 4")
    TestCommon.assert_eq(ctx.sealed_vessel.sealed, true, "Vessel must be sealed")
end

function Suite.test_container_overflow_tripwire()
    -- Create payload exceeding capacity limit
    local oversized = {}
    for i = 1, 10 do oversized[i] = i end

    local ctx = ContainerTape.open(oversized, 4, 1) -- Capacity is 4, payload is 10
    local status = ContainerTape:execute_all(ctx)

    TestCommon.assert_eq(status, TapeEngine.STATE_ROLLBACK, "Overflow must trigger STATE_ROLLBACK (-1)")
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_TRIPPED, "Tripwire must be TRIPPED on containment violation")
end

function Suite.test_container_lifecycle_rolls()
    local ctx = ContainerTape.open({ 1, 2, 3 }, 8, 1)
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_ARMED, "Tripwire must be ARMED on open")

    local status = ContainerTape:execute_all(ctx)
    TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED, "Execution must PROCEED")

    local close_st, close_msg = ContainerTape.close(ctx)
    TestCommon.assert_eq(close_st, TapeEngine.STATE_PROCEED, "Close must seal in PROCEED (+1)")
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_DISARMED, "Tripwire must be DISARMED on close")
end

function Suite.run()
    print("[SUITE: Container Process Ribbon (Lua)]")
    local ok1 = TestCommon.run_test("Bounded Payload Encapsulation", Suite.test_container_encapsulation)
    local ok2 = TestCommon.run_test("Overflow Containment Tripwire & Rollback", Suite.test_container_overflow_tripwire)
    local ok3 = TestCommon.run_test("4-Block Lifecycle Rolls", Suite.test_container_lifecycle_rolls)
    return ok1 and ok2 and ok3
end

-- Allow standalone execution
```

```
if pcall(debug.getlocal, 4, 1) == false then
    local ok = Suite.run()
    os.exit(ok and 0 or 1)
end

return Suite
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_number.lua

LUA | 197 LINES | 851 WORDS | 8.4 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & TEST TOPOLOGY
-- ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Number Tape } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Validates the interpreted process ribbon < Number Tape > in isolation:
--   - Radix-3 balanced ternary transduction (64 bits <=> 32 balanced trits)
--   - Dual-rail polar bit-planes: 00=0 (Equilibrium), 01=+1, 10=-1, 11=Trap
--   - SIMD rail-swap negation: -(-X) == X via bit-plane swap
--   - Collision detection: NULL code (11) trips safety interlock
--   - Dynamic packing topologies: 2-2-2-2, 2-1-2-1-2, 1-2-1-2-2, 2-2-1-2-1,
--     3-3-2, 3-2-3, 2-3-3
--   - Core ALU Zero-Sum Equilibrium Law: (+1) plus (-1) <=> 0
--   - Single-word boundary laws & 4-block lifecycle execution
--   - Witness: Scripture Psalm 147:4 KJV
--     "He telleth the number of the stars; he calleth them all by their names."
--   - Semantics: { DualRailPlanes :: KinematicTopologies :: ZeroSumBalance }
-- ================================================================================

package.path = package.path .. ";./?.lua;./lua_tapes/?.lua;" ..
               "./tests/lua/unit/?.lua;./unit/?.lua;../?.lua;../../?.lua"

local TestCommon = require("test_common")
local TapeEngine = require("lua_tapes.tape_engine")
local NumberTape = require("lua_tapes.number")

local Suite = {}

function Suite.test_number_dual_rail_bitplanes()
    -- 1. Test 2-bit codes: 00=0, 01=+1, 10=-1, 11=trap
    TestCommon.assert_eq(NumberTape.trit_to_2bit(0), 0, "0 must map to 0b00")
    TestCommon.assert_eq(NumberTape.trit_to_2bit(1), 1, "1 must map to 0b01")
    TestCommon.assert_eq(NumberTape.trit_to_2bit(-1), 2, "-1 must map to 0b10")

    local t0, ok0 = NumberTape.two_bit_to_trit(0)
    TestCommon.assert(ok0 and t0 == 0, "0b00 must decode to 0")
    local t1, ok1 = NumberTape.two_bit_to_trit(1)
    TestCommon.assert(ok1 and t1 == 1, "0b01 must decode to +1")
    local t2, ok2 = NumberTape.two_bit_to_trit(2)
    TestCommon.assert(ok2 and t2 == -1, "0b10 must decode to -1")
    local _, ok3 = NumberTape.two_bit_to_trit(3)
    TestCommon.assert(not ok3, "0b11 must report collision / NULL trap")

    -- 2. Test dual-rail packing and unpacking
    local trits = {}
    for i = 1, 32 do
        if i % 3 == 1 then trits[i] = 1
        elseif i % 3 == 2 then trits[i] = -1
        else trits[i] = 0 end
    end
    local packed = NumberTape.pack_trits_dual_rail(trits)
    local unpacked = {}
    local ok = NumberTape.unpack_trits_dual_rail(packed, unpacked)
    TestCommon.assert(ok, "Unpack must succeed on valid trits")
    for i = 1, 32 do
        TestCommon.assert_eq(unpacked[i], trits[i],
                             "Unpacked trit must match original at " .. i)
    end
end

function Suite.test_number_simd_rail_swap_negation()
    local trits = {}
    for i = 1, 32 do
        if i % 3 == 1 then trits[i] = 1
        elseif i % 3 == 2 then trits[i] = -1
        else trits[i] = 0 end
    end
    local packed = NumberTape.pack_trits_dual_rail(trits)
    local negated = NumberTape.negate_dual_rail(packed)

    local unpacked_neg = {}
    local ok = NumberTape.unpack_trits_dual_rail(negated, unpacked_neg)
    TestCommon.assert(ok, "Unpack negated trits must succeed")
```

```
    for i = 1, 32 do
        TestCommon.assert_eq(unpacked_neg[i], -trits[i],
                             "Negated trit must invert sign at " .. i)
    end

    -- Involution: -(-X) == X
    local double_neg = NumberTape.negate_dual_rail(negated)
    TestCommon.assert_eq(double_neg, packed,
                         "Double rail-swap negation must restore original")
end

function Suite.test_number_null_collision_trap()
    -- Valid packed trits have no null collision
    local trits = { 1, -1, 0, 1 }
    local packed = NumberTape.pack_trits_dual_rail(trits)
    TestCommon.assert(not NumberTape.has_null_collision(packed),
                      "Valid trits must not trigger null collision")

    -- Manually inject 11 (code 3) into trit slot 0
    local corrupted = packed | 3
    TestCommon.assert(NumberTape.has_null_collision(corrupted),
                      "Corrupted dual-rail must detect null collision")

    local unpacked = {}
    local ok = NumberTape.unpack_trits_dual_rail(corrupted, unpacked)
    TestCommon.assert(not ok, "Unpacking corrupted dual-rail must fail")
end

function Suite.test_number_dynamic_packing_topologies()
    local modes = {
        NumberTape.PACK_MODES.DENSE_2222,
        NumberTape.PACK_MODES.KINEMATIC_21212,
        NumberTape.PACK_MODES.KINEMATIC_12122,
        NumberTape.PACK_MODES.KINEMATIC_22121,
        NumberTape.PACK_MODES.DIRECTIONAL_332,
        NumberTape.PACK_MODES.DIRECTIONAL_323,
        NumberTape.PACK_MODES.DIRECTIONAL_233
    }
    local test_trits = { 1, -1, 1, -1 }

    for _, mode in ipairs(modes) do
        local byte_val = NumberTape.pack_byte(test_trits, mode)
        local out_trits = {}
        local ok = NumberTape.unpack_byte(byte_val, mode, out_trits)
        TestCommon.assert(ok, "Unpack byte must succeed for mode " .. mode)
        -- Verify first 3 trits match in kinematic / directional modes
        TestCommon.assert_eq(out_trits[1], test_trits[1], "Trit 1 must match")
        TestCommon.assert_eq(out_trits[2], test_trits[2], "Trit 2 must match")
        TestCommon.assert_eq(out_trits[3], test_trits[3], "Trit 3 must match")
        if mode == NumberTape.PACK_MODES.DENSE_2222 then
            TestCommon.assert_eq(out_trits[4], test_trits[4],
                                 "Trit 4 must match in dense mode")
        end
    end
end

function Suite.test_number_transduction()
    local values = { 0, 1, -1, 4096, -4096, 65536 }
    for _, val in ipairs(values) do
        local ctx = NumberTape.open(val)
        local status = NumberTape:execute_all(ctx)
        TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED,
                             "Execution must PROCEED (+1)")
        TestCommon.assert_eq(ctx.c_word.scalar, val,
                             "Context word scalar must match input")
        TestCommon.assert(ctx.c_word.is_enclosed == true,
                          "Context word must be enclosed")
        TestCommon.assert_eq(ctx.equilibrium_proved, true,
                             "ALU Zero-Sum Equilibrium must be proved")
    end
end

function Suite.test_number_equilibrium_law()
    -- Direct law verification: (+1) + (-1) == 0
    local sum = (1) + (-1)
    TestCommon.assert_eq(sum, 0,
                         "ALU Zero-Sum Equilibrium Law must evaluate to zero")
end
```

```
152
153 function Suite.test_number_lifecycle_rolls()
154     local ctx = NumberTape.open(4096)
155     TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_ARMED,
156                          "Tripwire must be ARMED on open")
157
158     local status = NumberTape:execute_all(ctx)
159     TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED,
160                          "Execution must PROCEED")
161
162     local close_st = NumberTape.close(ctx)
163     TestCommon.assert_eq(close_st, TapeEngine.STATE_PROCEED,
164                          "Close must seal in PROCEED (+1)")
165     TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_DISARMED,
166                          "Tripwire must be DISARMED on close")
167 end
168
169 function Suite.run()
170     print("[SUITE: Number Process Ribbon (Lua)]")
171     local ok1 = TestCommon.run_test("Dual-Rail Polar Bit-Planes",
172                                     Suite.test_number_dual_rail_bitplanes)
173     local ok2 = TestCommon.run_test("SIMD Rail-Swap Negation",
174                                     Suite.test_number_simd_rail_swap_negation)
175     local ok3 = TestCommon.run_test("NULL Collision Trap",
176                                     Suite.test_number_null_collision_trap)
177     local ok4 = TestCommon.run_test(
178         "Dynamic Packing Topologies",
179         Suite.test_number_dynamic_packing_topologies
180     )
181     local ok5 = TestCommon.run_test("Radix-3 Balanced Transduction",
182                                     Suite.test_number_transduction)
183     local ok6 = TestCommon.run_test("ALU Zero-Sum Equilibrium Law",
184                                     Suite.test_number_equilibrium_law)
185     local ok7 = TestCommon.run_test("4-Block Lifecycle Rolls",
186                                     Suite.test_number_lifecycle_rolls)
187     return ok1 and ok2 and ok3 and ok4 and ok5 and ok6 and ok7
188 end
189
190 -- Allow standalone execution
191 if pcall(debug.getlocal, 4, 1) == false then
192     local ok = Suite.run()
193     os.exit(ok and 0 or 1)
194 end
195
196 return Suite
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_site.lua

LUA | 82 LINES | 379 WORDS | 3.8 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & TEST TOPOLOGY
-- ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Site Tape } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Validates the interpreted process ribbon < Site Tape > in isolation:
--   - Silicon coordinate binding
--   - Hardware Routing Law (MMIO vs RAM coordinates)
--   - Null pointer address rejection and safety interlock rollback
--   - Memory anchor emergence
--   - Full 4-block lifecycle execution (open, execute_all, close)
--   - Witness: Scripture Genesis 28:16 KJV
--     "And Jacob awaked out of his sleep, and he said, Surely the Lord is in
--      this place; and I knew it not."
--   - Semantics: { SiliconCoordinates :: HardwareRoutingLaw :: NonNullInterlock }
-- ================================================================================

package.path     = package.path .. ";./?.lua;./lua_tapes/?.lua;./tests/lua/unit/?.lua;./unit/?.lua;../?.lua;../../?.lua"

local TestCommon = require("test_common")
local TapeEngine = require("lua_tapes.tape_engine")
local SiteTape   = require("lua_tapes.site")

local Suite      = {}

function Suite.test_site_routing()
    -- VGA buffer -> MMIO
    local vga_ctx = SiteTape.open(SiteTape.constants.SITE_VGA)
    local vga_st = SiteTape:execute_all(vga_ctx)
    TestCommon.assert_eq(vga_st, TapeEngine.STATE_PROCEED, "VGA site must PROCEED")
    TestCommon.assert_eq(vga_ctx.c_site.routing, "HARDWARE_MMIO (::@)", "VGA must route to MMIO")
    TestCommon.assert(vga_ctx.memory_anchor.anchored == true, "Memory anchor must be set")

    -- COM1 serial port -> MMIO
    local com1_ctx = SiteTape.open(SiteTape.constants.SITE_COM1)
    local com1_st = SiteTape:execute_all(com1_ctx)
    TestCommon.assert_eq(com1_st, TapeEngine.STATE_PROCEED, "COM1 site must PROCEED")
    TestCommon.assert_eq(com1_ctx.c_site.routing, "HARDWARE_MMIO (::@)", "COM1 must route to MMIO")

    -- MBR origin -> RAM
    local mbr_ctx = SiteTape.open(SiteTape.constants.SITE_MBR)
    local mbr_st = SiteTape:execute_all(mbr_ctx)
    TestCommon.assert_eq(mbr_st, TapeEngine.STATE_PROCEED, "MBR site must PROCEED")
    TestCommon.assert_eq(mbr_ctx.c_site.routing, "PHYSICAL_RAM (@)", "MBR must route to RAM")
end

function Suite.test_site_null_tripwire()
    local null_ctx = SiteTape.open(0x00000000)
    local status = SiteTape:execute_all(null_ctx)

    TestCommon.assert_eq(status, TapeEngine.STATE_ROLLBACK, "Null coordinate must trigger STATE_ROLLBACK (-1)")
    TestCommon.assert_eq(null_ctx.tripwire, TapeEngine.TRIPWIRE_TRIPPED, "Tripwire must be TRIPPED on null address")
end

function Suite.test_site_lifecycle_rolls()
    local ctx = SiteTape.open(SiteTape.constants.SITE_VGA)
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_ARMED, "Tripwire must be ARMED on open")

    local status = SiteTape:execute_all(ctx)
    TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED, "Execution must PROCEED")

    local close_st, close_msg = SiteTape.close(ctx)
    TestCommon.assert_eq(close_st, TapeEngine.STATE_PROCEED, "Close must seal in PROCEED (+1)")
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_DISARMED, "Tripwire must be DISARMED on close")
end

function Suite.run()
    print("[SUITE: Site Process Ribbon (Lua)]")
    local ok1 = TestCommon.run_test("Hardware Routing Law (MMIO vs RAM)", Suite.test_site_routing)
    local ok2 = TestCommon.run_test("Null Address Rejection & Interlock Rollback", Suite.test_site_null_tripwire)
    local ok3 = TestCommon.run_test("4-Block Lifecycle Rolls", Suite.test_site_lifecycle_rolls)
    return ok1 and ok2 and ok3
end
```

```
74
75  -- Allow standalone execution
76  if pcall(debug.getlocal, 4, 1) == false then
77      local ok = Suite.run()
78      os.exit(ok and 0 or 1)
79  end
80
81  return Suite
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/lua/unit/test_state.lua

LUA | 93 LINES | 439 WORDS | 4.2 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & TEST TOPOLOGY
-- ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { State Tape } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Validates the interpreted process ribbon < State Tape > in isolation:
--   - Quad-State named registers: [ STATE TURN BOUND IT ]
--   - Canonical Loop Law cadence and single-step detents:
--       WHEN IT < BOUND  --> PROCEED (+1)
--       !    IT > BOUND  --> AWAIT (0)
--       ::   IT == BOUND --> HALT (NULL)
--   - Clock governance emergence
--   - Full 4-block lifecycle execution (open, execute_all, close)
--   - Witness: Scripture Ecclesiastes 3:1 KJV
--     "To every thing there is a season, and a time to every purpose under
--      the heaven:"
--   - Semantics: { QuadStateTruth :: CanonicalLoopCadence :: CausalDetents }
-- ================================================================================

package.path = package.path .. ";./?.lua;./lua_tapes/?.lua;./tests/lua/unit/?.lua;./unit/?.lua;../?.lua;../../?.lua"

local TestCommon = require("test_common")
local TapeEngine = require("lua_tapes.tape_engine")
local StateTape  = require("lua_tapes.state")

local Suite = {}

function Suite.test_state_canonical_loop()
    local ctx = StateTape.open(TapeEngine.STATE_PROCEED, 3)

    TestCommon.assert_eq(ctx.c_state.registers.bound, 3, "Bound must be 3")
    TestCommon.assert_eq(ctx.c_state.registers.it, 0, "Initial IT must be 0")
    TestCommon.assert_eq(ctx.c_state.registers.turn, 0, "Initial TURN must be 0")

    -- Step 1
    local st1 = StateTape.step_loop(ctx)
    TestCommon.assert_eq(st1, TapeEngine.STATE_PROCEED, "Step 1 must PROCEED")
    TestCommon.assert_eq(ctx.c_state.registers.it, 1, "IT must be 1")

    -- Step 2
    local st2 = StateTape.step_loop(ctx)
    TestCommon.assert_eq(st2, TapeEngine.STATE_PROCEED, "Step 2 must PROCEED")
    TestCommon.assert_eq(ctx.c_state.registers.it, 2, "IT must be 2")

    -- Step 3
    local st3 = StateTape.step_loop(ctx)
    TestCommon.assert_eq(st3, TapeEngine.STATE_PROCEED, "Step 3 must PROCEED")
    TestCommon.assert_eq(ctx.c_state.registers.it, 3, "IT must be 3")

    -- Step 4: IT == BOUND -> HALT:NULL
    local st4 = StateTape.step_loop(ctx)
    TestCommon.assert_eq(st4, TapeEngine.STATE_HALT, "Step 4 must reach terminal detent HALT:NULL")
    TestCommon.assert_eq(ctx.c_state.registers.state, TapeEngine.STATE_HALT, "Register state must be HALT:NULL")
end

function Suite.test_state_await_transition()
    local ctx = StateTape.open(TapeEngine.STATE_PROCEED, 4)
    ctx.c_state.registers.it = 5 -- Manually set IT > BOUND

    local st = StateTape.step_loop(ctx)
    TestCommon.assert_eq(st, TapeEngine.STATE_AWAIT, "IT > BOUND must transition to AWAIT (0)")
end

function Suite.test_state_lifecycle_rolls()
    local ctx = StateTape.open(TapeEngine.STATE_PROCEED, 4)
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_ARMED, "Tripwire must be ARMED on open")

    local status = StateTape:execute_all(ctx)
    TestCommon.assert_eq(status, TapeEngine.STATE_HALT, "Execution must reach terminal detent HALT:NULL")
    TestCommon.assert_eq(ctx.c_state.registers.it, 4, "Final IT must equal bound 4")
    TestCommon.assert_eq(ctx.c_state.registers.state, TapeEngine.STATE_HALT, "Register state must reach HALT:NULL")

    local close_st, close_msg = StateTape.close(ctx)
```

```
    TestCommon.assert_eq(close_st, TapeEngine.STATE_PROCEED, "Close must seal in PROCEED (+1)")
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_DISARMED, "Tripwire must be DISARMED on close")
end

function Suite.run()
    print("[SUITE: State Process Ribbon (Lua)]")
    local ok1 = TestCommon.run_test("Canonical Loop Law Cadence (IT < BOUND -> HALT)", Suite.test_state_canonical_loop)
    local ok2 = TestCommon.run_test("Standby Transition (IT > BOUND -> AWAIT)", Suite.test_state_await_transition)
    local ok3 = TestCommon.run_test("4-Block Lifecycle Rolls", Suite.test_state_lifecycle_rolls)
    return ok1 and ok2 and ok3
end

-- Allow standalone execution
if pcall(debug.getlocal, 4, 1) == false then
    local ok = Suite.run()
    os.exit(ok and 0 or 1)
end

return Suite
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/compile_flags.txt

TXT | 6 LINES | 5 WORDS | 0.1 KB

```
-Iincludes
-I../../c_backbone/include
-Wall
-Wextra
-std=c11
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/test_number.o DOC/BINARY 1 LINES 5 WORDS 21.8 KB

[Binary Artifact: test_number.o (21.8 KB)]

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/integrated/test_omni_lib.c

C | 121 LINES | 492 WORDS | 5.5 KB

```c
/* ========================================================================
 * BLOCK 1: METADATA & INTEGRATION TEST TOPOLOGY
 * ( Test Orchestrator: < Lexicon { Types of Ideas } > & [ Multi-File Testbed ] )
 * ========================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Primary test orchestrator for the OmniCode Standard Library C Backbone.
 *   Executes multi-file unit test suites followed by integrated cross-type pipelines:
 *   1. Unit Suite: Character (Exodus 31:18 KJV)
 *   2. Unit Suite: Number (Psalm 147:4 KJV)
 *   3. Unit Suite: Container (2 Kings 4:4 KJV)
 *   4. Unit Suite: Site (Genesis 28:16 KJV)
 *   5. Unit Suite: State (Ecclesiastes 3:1 KJV)
 *   6. Integrated Composition: Container<Character> Traversed via State Loop
 *   - Witness: Scripture Proverbs 24:27 KJV
 *     "Prepare thy work without, and make it fit for thyself in the field;
 *      and afterwards build thine house."
 *   - Semantics: { MultiFileTestbed :: CrossTypeIntegration :: DetentVerification }
 * ======================================================================== */

#include "test_common.h"
#include <stdio.h>

/* Global test failure accumulator */
int g_omni_test_failures = 0;

/* ========================================================================
 * BLOCK 2: OPENING CONTEXT & INTEGRATED PIPELINE DEFINITIONS
 * ======================================================================== */

static int run_cross_type_integration_pipeline(void) {
    printf("[SUITE: Cross-Type Composition Pipeline]\n");
    printf("  -> Composing Container<Character> (\"The Text Substrate\") ...\n");

    #define TEST_TEXT_CAPACITY 32
    omni_character_t text_storage[TEST_TEXT_CAPACITY];
    omni_container_t text_ribbon;
    omni_container_init(&text_ribbon, text_storage, sizeof(omni_character_t), TEST_TEXT_CAPACITY);

    const char *source_str = "OMNICODE";
    for (size_t i = 0; source_str[i] != '\0'; ++i) {
        omni_character_t ch = omni_character_from_glyph((uint8_t)source_str[i]);
        omni_quad_state_t push_res = omni_container_push(&text_ribbon, &ch);
        OMNI_TEST_ASSERT(push_res == OMNI_STATE_PROCEED, "Container push must succeed without overflow");
    }

    OMNI_TEST_ASSERT(text_ribbon.occupancy == 8, "Text ribbon occupancy must be 8");
    printf("  -> Capacity: %zu, Occupancy: %zu [PASS]\n", text_ribbon.capacity, text_ribbon.occupancy);

    /* Traversal via Canonical State Loop */
    printf("  -> Driving Traversal via Canonical State Loop:\n     Transduced Output: \"");
    omni_state_t traversal_state = omni_state_create(OMNI_STATE_PROCEED, (int64_t)text_ribbon.occupancy);

    while (omni_state_is_proceeding(&traversal_state)) {
        size_t idx = (size_t)traversal_state.registers.it;
        omni_character_t *curr = (omni_character_t *)omni_container_get(&text_ribbon, idx);
        if (curr) {
            putchar((char)curr->glyph);
        }
        omni_state_step_loop(&traversal_state.registers);
    }
    printf("\"\n");

    OMNI_TEST_ASSERT(omni_state_is_halted(&traversal_state) == true, "Traversal must reach HALT:NULL detent");
    OMNI_TEST_ASSERT(traversal_state.registers.it == 8, "Final IT must equal 8");
    OMNI_TEST_ASSERT(traversal_state.registers.turn == 8, "Final TURN must equal 8");
    printf("     Final Detent: HALT:NULL (IT=8, TURN=8) [PASS]\n");

    /* Route Site to VGA Buffer MMIO */
    omni_site_t vga_site = omni_site_from_address(OMNI_SITE_VGA_BUFFER);
    OMNI_TEST_ASSERT(vga_site.routing == OMNI_ROUTING_HARDWARE_MMIO, "VGA site must resolve to HARDWARE_MMIO");
    printf("  -> Virtual Hardware MMIO Routing: 0x%lX -> HARDWARE_MMIO (::@) [PASS]\n",
           (unsigned long)vga_site.coordinate);

```

```
    return 0;
}

/* ============================================================================
 * BLOCK 3: BODY PRIMITIVES & MAIN TEST RUNNER
 * ============================================================================ */

int main(void) {
    printf("================================================================================\n");
    printf("   OMNICODE STANDARD LIBRARY C BACKBONE: MULTI-FILE TEST HARNESS\n");
    printf("================================================================================\n\n");

    /* Run Unit Test Suites */
    if (test_character_suite() != 0) goto failure;
    printf("\n");

    if (test_number_suite() != 0) goto failure;
    printf("\n");

    if (test_container_suite() != 0) goto failure;
    printf("\n");

    if (test_site_suite() != 0) goto failure;
    printf("\n");

    if (test_state_suite() != 0) goto failure;
    printf("\n");

    /* Run Cross-Type Integration Pipeline */
    if (run_cross_type_integration_pipeline() != 0) goto failure;
    printf("\n");

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK & FINAL DETENT REPORT
 * ============================================================================ */

    printf("================================================================================\n");
    printf("   [ALL C BACKBONE TESTS PASS]: FULL 4-BLOCK PARITY SEALED IN PROCEED (+1)\n");
    printf("================================================================================\n");
    return 0;

failure:
    printf("\n================================================================================\n");
    printf("   [TEST SUITE FAILURE]: %d assertion failure(s) recorded!\n", g_omni_test_failures);
    printf("================================================================================\n");
    return 1;
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/includes/test_common.h

H | 65 LINES | 248 WORDS | 2.3 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & TEST HARNESS TOPOLOGY
 * ( Test Framework: < OmniCode Standard Library Testbed > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Shared test harness for the OmniCode Standard Library C Transpile Variant.
 *   Provides structured assertions, test suite runner macros, and tripwire checks.
 *   - Witness: Scripture Proverbs 24:27 KJV
 *     "Prepare thy work without, and make it fit for thyself in the field;
 *      and afterwards build thine house."
 *   - Semantics: { AssertionInterlock :: GranularFaultIsolation :: MultiFileTestbed }
 * ============================================================================ */

#ifndef OMNI_TEST_COMMON_H
#define OMNI_TEST_COMMON_H

#include "omni.h"
#include <stdio.h>
#include <stdbool.h>

/* ============================================================================
 * BLOCK 2: OPENING CONTEXT & MACRO DEFINITIONS
 * ============================================================================ */

/** Global test assertion failure counter */
extern int g_omni_test_failures;

/**
 * @brief Evaluates an assertion expression; logs failure and returns 1 on error.
 */
#define OMNI_TEST_ASSERT(expr, msg) \
    do { \
        if (!(expr)) { \
            printf("    [FAIL] %s:%d: %s (Expression: %s)\n", __FILE__, __LINE__, (msg), #expr); \
            g_omni_test_failures++; \
            return 1; \
        } \
    } while (0)

/**
 * @brief Runs an individual test function and reports PASS / FAIL.
 */
#define OMNI_TEST_RUN(fn, test_name) \
    do { \
        printf("    -> %s ... ", (test_name)); \
        if ((fn)() != 0) { \
            printf("[FAILED]\n"); \
            return 1; \
        } \
        printf("[PASS]\n"); \
    } while (0)

/* ============================================================================
 * BLOCK 3: BODY PRIMITIVES & SUITE EXPORTS
 * ============================================================================ */

/** Unit test suite prototypes */
int test_character_suite(void);
int test_number_suite(void);
int test_container_suite(void);
int test_site_suite(void);
int test_state_suite(void);

#endif /* OMNI_TEST_COMMON_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_character.c

C | 121 LINES | 599 WORDS | 5.8 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & TEST TOPOLOGY
 * ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Character } ] )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Validates the atomic invariant type < Character > in isolation:
 *   - Bijective round-trip transduction (8 bits <=> 5 balanced trits)
 *   - Strict single-cell enclosure boundary laws
 *   - Full 4-block lifecycle execution (open, execute, close)
 *   - Relational equality and comparison
 *   - Witness: Scripture Exodus 31:18 KJV
 *     "And he gave unto Moses, when he had made an end of communing with him upon
 *      mount Sinai, two tables of testimony, tables of stone, written with the
 *      finger of God."
 *   - Semantics: { GlyphFidelity :: BalancedTritLaw :: SingleCellEnclosure }
 * ============================================================================ */

#include "test_common.h"

/* ============================================================================
 * BLOCK 2: OPENING CONTEXT & FIXTURE SETUP
 * ============================================================================ */

static int test_character_bijective_roundtrip(void) {
    /* Test exemplar characters across 5-trit cell capacity (0..121) */
    const uint8_t test_glyphs[] = {'A', 'Z', '0', '9', ' ', 0, 70, 100, 121};
    const size_t num_glyphs = sizeof(test_glyphs) / sizeof(test_glyphs[0]);

    for (size_t i = 0; i < num_glyphs; ++i) {
        uint8_t original = test_glyphs[i];
        omni_character_t c = omni_character_from_glyph(original);

        /* Verify enclosure */
        OMNI_TEST_ASSERT(c.is_enclosed == true, "Character must be enclosed");
        OMNI_TEST_ASSERT(c.glyph == original, "Character glyph must match original");

        /* Verify every trit is in valid balanced range {-1, 0, +1} */
        for (size_t t = 0; t < OMNI_CHAR_TRITS_PER_CELL; ++t) {
            OMNI_TEST_ASSERT(c.trits[t] >= -1 && c.trits[t] <= 1, "Trit must be in {-1, 0, +1}");
        }

        /* Verify inverse transduction reconstructs the exact byte */
        uint8_t reconstructed = omni_character_to_byte(c.trits);
        OMNI_TEST_ASSERT(reconstructed == original, "Inverse transduction must exactly match original byte");
    }
    return 0;
}

static int test_character_bounds_and_enclosure(void) {
    omni_character_t valid_char = omni_character_from_glyph('X');
    OMNI_TEST_ASSERT(omni_character_verify_bounds(&valid_char) == true, "Valid character must pass boundary check");

    /* Corrupt enclosure flag to test boundary tripwire */
    omni_character_t corrupted = valid_char;
    corrupted.is_enclosed = false;
    OMNI_TEST_ASSERT(omni_character_verify_bounds(&corrupted) == false, "Unenclosed character must fail boundary check");

    return 0;
}

static int test_character_relational_equality(void) {
    omni_character_t a1 = omni_character_from_glyph('M');
    omni_character_t a2 = omni_character_from_glyph('M');
    omni_character_t b  = omni_character_from_glyph('N');

    OMNI_TEST_ASSERT(omni_character_eq(&a1, &a2) == true, "Identical glyphs must evaluate equal");
    OMNI_TEST_ASSERT(omni_character_eq(&a1, &b) == false, "Distinct glyphs must not evaluate equal");

    return 0;
}

static int test_character_lifecycle_rolls(void) {
    omni_character_context_t ctx;
```

```
    omni_character_open(&ctx, 'K', OMNI_TARGET_STDOUT);

    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_ARMED, "Context tripwire must be ARMED upon open");
    OMNI_TEST_ASSERT(ctx.input_glyph == 'K', "Input glyph must be recorded in context");

    /* Execute the 3 declared rolls */
    omni_quad_state_t exec_status = omni_character_execute(&ctx);
    OMNI_TEST_ASSERT(exec_status == OMNI_STATE_PROCEED, "Character execute must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.character.is_enclosed == true, "Character must be enclosed after execution");
    OMNI_TEST_ASSERT(ctx.character.glyph == 'K', "Character glyph must match input");

    /* Close and commit */
    omni_quad_state_t close_status = omni_character_close(&ctx);
    OMNI_TEST_ASSERT(close_status == OMNI_STATE_PROCEED, "Character close must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_DISARMED, "Tripwire must be DISARMED upon clean close");

    return 0;
}

static int test_character_bit_level_and_ascii_invariant(void) {
    omni_character_t a = omni_character_from_glyph('A'); /* 65 = 0b01000001 */
    OMNI_TEST_ASSERT(omni_character_is_ascii(&a) == true, "'A' must be ASCII");
    OMNI_TEST_ASSERT(omni_character_get_bit(&a, 0) == 1, "Bit 0 of 'A' must be 1");
    OMNI_TEST_ASSERT(omni_character_get_bit(&a, 1) == 0, "Bit 1 of 'A' must be 0");
    OMNI_TEST_ASSERT(omni_character_get_bit(&a, 6) == 1, "Bit 6 of 'A' must be 1");
    OMNI_TEST_ASSERT(omni_character_get_bit(&a, 7) == 0, "Bit 7 of 'A' must be 0 (parity rail)");

    omni_character_t high = omni_character_from_glyph(200);
    OMNI_TEST_ASSERT(omni_character_is_ascii(&high) == false, "200 is not standard ASCII");
    OMNI_TEST_ASSERT(omni_character_get_bit(&high, 7) == 1, "Bit 7 of 200 must be 1");

    return 0;
}

/* ============================================================================
 * BLOCK 3: BODY PRIMITIVES & TEST SUITE RUNNER
 * ============================================================================ */

int test_character_suite(void) {
    printf("[SUITE: Character Invariant Type]\n");
    OMNI_TEST_RUN(test_character_bijective_roundtrip, "Bijective Round-Trip Transduction");
    OMNI_TEST_RUN(test_character_bounds_and_enclosure, "Single-Cell Enclosure Bounds");
    OMNI_TEST_RUN(test_character_bit_level_and_ascii_invariant, "Bit-Level Slicing & ASCII Invariant");
    OMNI_TEST_RUN(test_character_relational_equality, "Relational Equality");
    OMNI_TEST_RUN(test_character_lifecycle_rolls, "4-Block Lifecycle Rolls");
    return 0;
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_container.c

C | 145 LINES | 670 WORDS | 6.5 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & TEST TOPOLOGY
 * ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Container } ] )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Validates the generic memory capsule type < Container > in isolation:
 *   - Bounded slice allocation and element encapsulation
 *   - Law of Containment verification (Occupancy <= Capacity)
 *   - Overflow detection and tripwire interlock triggering
 *   - Standard 64-byte vessel envelope
 *   - Full 4-block lifecycle execution (open, execute, close)
 *   - Witness: Scripture 2 Kings 4:4 KJV
 *     "And when thou art come in, thou shalt shut the door upon thee and upon
 *      thy sons, and shalt pour out into all those vessels, and thou shalt set
 *      aside that which is full."
 *   - Semantics: { MemoryCapsule :: BoundedSlice :: ContainmentLaw }
 * ============================================================================ */

#include "test_common.h"

/* ============================================================================
 * BLOCK 2: OPENING CONTEXT & FIXTURE SETUP
 * ============================================================================ */

static int test_container_basic_lifecycle(void) {
    uint32_t storage[8];
    omni_container_t c;
    omni_container_init(&c, storage, sizeof(uint32_t), 8);

    OMNI_TEST_ASSERT(c.capacity == 8, "Container capacity must be 8");
    OMNI_TEST_ASSERT(c.occupancy == 0, "Container occupancy must initially be 0");
    OMNI_TEST_ASSERT(c.is_enclosed == true, "Container must be enclosed");
    OMNI_TEST_ASSERT(omni_container_is_empty(&c) == true, "Container must report empty");

    /* Push 4 elements */
    for (uint32_t i = 1; i <= 4; ++i) {
        omni_quad_state_t push_st = omni_container_push(&c, &i);
        OMNI_TEST_ASSERT(push_st == OMNI_STATE_PROCEED, "Push must PROCEED");
    }
    OMNI_TEST_ASSERT(c.occupancy == 4, "Occupancy must be 4");

    /* Verify random access get */
    for (size_t i = 0; i < 4; ++i) {
        uint32_t *val = (uint32_t *)omni_container_get(&c, i);
        OMNI_TEST_ASSERT(val != NULL, "Get must return non-null pointer");
        OMNI_TEST_ASSERT(*val == (uint32_t)(i + 1), "Stored element must match pushed value");
    }

    /* Out of bounds get must return NULL */
    OMNI_TEST_ASSERT(omni_container_get(&c, 10) == NULL, "Out of bounds get must return NULL");

    /* Test random access set */
    uint32_t new_val = 999;
    omni_quad_state_t set_st = omni_container_set(&c, 1, &new_val);
    OMNI_TEST_ASSERT(set_st == OMNI_STATE_PROCEED, "Set must PROCEED");
    uint32_t *read_back = (uint32_t *)omni_container_get(&c, 1);
    OMNI_TEST_ASSERT(read_back != NULL && *read_back == 999, "Read back value at index 1 must be 999");

    /* Clear */
    omni_container_clear(&c);
    OMNI_TEST_ASSERT(c.occupancy == 0, "Occupancy must be 0 after clear");
    OMNI_TEST_ASSERT(omni_container_is_empty(&c) == true, "Container must report empty after clear");

    return 0;
}

static int test_container_overflow_tripwire(void) {
    uint8_t tiny_storage[2];
    omni_container_t c;
    omni_container_init(&c, tiny_storage, sizeof(uint8_t), 2);

    uint8_t a = 10, b = 20, c_overflow = 30;
    OMNI_TEST_ASSERT(omni_container_push(&c, &a) == OMNI_STATE_PROCEED, "First push must PROCEED");
```

```
    OMNI_TEST_ASSERT(omni_container_push(&c, &b) == OMNI_STATE_PROCEED, "Second push must PROCEED");
    OMNI_TEST_ASSERT(omni_container_is_full(&c) == true, "Container must report full at capacity 2");

    /* Third push MUST fail and return ROLLBACK (-1) */
    omni_quad_state_t overflow_res = omni_container_push(&c, &c_overflow);
    OMNI_TEST_ASSERT(overflow_res == OMNI_STATE_ROLLBACK, "Overflow push must return ROLLBACK (-1)");
    OMNI_TEST_ASSERT(c.occupancy == 2, "Occupancy must remain capped at capacity");

    return 0;
}

static int test_container_containment_law(void) {
    uint8_t storage[4];
    omni_container_t c;
    omni_container_init(&c, storage, sizeof(uint8_t), 4);

    OMNI_TEST_ASSERT(omni_container_verify_containment(&c) == true, "Valid container must pass Containment Law");

    /* Violate containment law manually */
    omni_container_t corrupted = c;
    corrupted.occupancy = 10; /* Exceeds capacity 4 */
    OMNI_TEST_ASSERT(omni_container_verify_containment(&corrupted) == false, "Overflowed occupancy must violate
Containment Law");

    return 0;
}

static int test_byte_vessel_envelope(void) {
    omni_byte_vessel_t vessel;
    omni_byte_vessel_init(&vessel);

    OMNI_TEST_ASSERT(vessel.container.capacity == OMNI_CONTAINER_EXEMPLAR_VESSEL, "Byte vessel capacity must be 64");
    OMNI_TEST_ASSERT(vessel.container.element_size == 1, "Element size must be 1 byte");
    OMNI_TEST_ASSERT(vessel.container.occupancy == 0, "Byte vessel must initially be empty");

    return 0;
}

static int test_container_lifecycle_rolls(void) {
    uint16_t storage[16];
    uint16_t payload[4] = {100, 200, 300, 400};

    omni_container_context_t ctx;
    omni_container_open(&ctx, storage, sizeof(uint16_t), 16, payload, 4);

    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_ARMED, "Tripwire must be ARMED upon open");
    OMNI_TEST_ASSERT(ctx.payload_count == 4, "Payload count must be 4");

    omni_quad_state_t exec_status = omni_container_execute(&ctx);
    OMNI_TEST_ASSERT(exec_status == OMNI_STATE_PROCEED, "Container execute must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.vessel.occupancy == 4, "Vessel occupancy must match payload count");

    omni_quad_state_t close_status = omni_container_close(&ctx);
    OMNI_TEST_ASSERT(close_status == OMNI_STATE_PROCEED, "Container close must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_DISARMED, "Tripwire must be DISARMED upon clean close");

    return 0;
}

/* ============================================================================
 * BLOCK 3: BODY PRIMITIVES & TEST SUITE RUNNER
 * ============================================================================ */

int test_container_suite(void) {
    printf("[SUITE: Container Invariant Type]\n");
    OMNI_TEST_RUN(test_container_basic_lifecycle, "Basic Allocation, Push & Pop");
    OMNI_TEST_RUN(test_container_overflow_tripwire, "Overflow Tripwire & Rollback");
    OMNI_TEST_RUN(test_container_containment_law, "Core Containment Law");
    OMNI_TEST_RUN(test_byte_vessel_envelope, "64-Byte Vessel Envelope");
    OMNI_TEST_RUN(test_container_lifecycle_rolls, "4-Block Lifecycle Rolls");
    return 0;
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_number.c

C | 244 LINES | 1146 WORDS | 11.1 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & TEST TOPOLOGY
 * ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Number } ] )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Validates the scalar invariant type < Number > in isolation:
 *   - Bijective round-trip radix-3 transduction (64 bits <=> 32 balanced trits)
 *   - Dual-rail bit-plane packed representation and O(1) rail-swap negation
 *   - Instant O(1) NULL collision trap detection
 *   - Dynamic packing topologies: 2-2-2-2, 2-1-2-1-2, 1-2-1-2-2, 2-2-1-2-1,
 *     3-3-2, 3-2-3, 2-3-3
 *   - ALU Zero-Sum Equilibrium Law: (+1) plus (-1) <=> 0
 *   - Balanced arithmetic addition, subtraction, and inverse cancellation
 *   - Strict single-word boundary laws
 *   - Full 4-block lifecycle execution (open, execute, close)
 *   - Witness: Scripture Psalm 147:4 KJV
 *     "He telleth the number of the stars; he calleth them all by their names."
 *   - Semantics: { DualRailBitPlanes :: KinematicTopologies :: ALUZeroSum }
 * ============================================================================ */

#include "test_common.h"

/* ============================================================================
 * BLOCK 2: OPENING CONTEXT & FIXTURE SETUP
 * ============================================================================ */

static int test_number_bijective_roundtrip(void) {
    const int64_t test_values[] = {
        0, 1, -1, 2, -2, 4096, -4096, 65536, -65536, 1000000, -1000000
    };
    const size_t num_values = sizeof(test_values) / sizeof(test_values[0]);

    for (size_t i = 0; i < num_values; ++i) {
        int64_t original = test_values[i];
        omni_number_t n = omni_number_from_scalar(original);

        OMNI_TEST_ASSERT(n.is_enclosed == true, "Number must be enclosed");
        OMNI_TEST_ASSERT(n.scalar == original, "Number scalar must match original");

        /* Verify all 32 trits are in {-1, 0, +1} */
        for (size_t t = 0; t < OMNI_NUM_TRITS_PER_WORD; ++t) {
            OMNI_TEST_ASSERT(n.trits[t] >= -1 && n.trits[t] <= 1,
                             "Trit must be in {-1, 0, +1}");
        }

        /* Verify reconstruction from trits */
        int64_t reconstructed = omni_number_to_scalar(n.trits);
        OMNI_TEST_ASSERT(reconstructed == original,
                         "Reconstructed scalar must equal original value");
    }
    return 0;
}

static int test_number_dual_rail_bitplanes(void) {
    /* Test Zero: All trits 00 -> packed is 0 */
    omni_number_t zero = omni_number_from_scalar(0);
    OMNI_TEST_ASSERT(omni_number_trits_is_equilibrium(zero.packed_trits) == true,
                     "Packed zero must be in equilibrium (0ULL)");
    OMNI_TEST_ASSERT(omni_number_trits_has_null(zero.packed_trits) == false,
                     "Packed zero must not have NULL traps");

    /* Test Positive Unity (+1): First trit is 01, others 00 */
    omni_number_t pos_one = omni_number_from_scalar(1);
    OMNI_TEST_ASSERT((pos_one.packed_trits & 0x03) == OMNI_TRIT_PAIR_POS,
                     "First trit of +1 must be 01 (positive rail active)");

    /* Test Negative Unity (-1): First trit is 10, others 00 */
    omni_number_t neg_one = omni_number_from_scalar(-1);
    OMNI_TEST_ASSERT((neg_one.packed_trits & 0x03) == OMNI_TRIT_PAIR_NEG,
                     "First trit of -1 must be 10 (negative rail active)");

    /* Test Instant Bit-Plane Negation (+1 <-> -1) */
    uint64_t negated_pos = omni_number_trits_negate(pos_one.packed_trits);
```

```
    OMNI_TEST_ASSERT((negated_pos & 0x03) == OMNI_TRIT_PAIR_NEG,
                     "Negating +1 packed trits must produce -1 (10)");
    OMNI_TEST_ASSERT(negated_pos == neg_one.packed_trits,
                     "Negated +1 packed word must equal -1 packed word");

    /* Test NULL Trap Detection: artificially inject 11 into trit 0 */
    uint64_t corrupted = pos_one.packed_trits | 0x03;
    OMNI_TEST_ASSERT(omni_number_trits_has_null(corrupted) == true,
                     "Packed word with 11 must be flagged as NULL trap");

    return 0;
}

static int test_number_dynamic_packing_topologies(void) {
    uint8_t out[5];

    /* 1. Test 2-2-2-2 (Dense: 4 fields of 2 bits) */
    uint8_t p_2222 = omni_number_pack_byte(1, 2, 0, 3, 0, OMNI_PACK_2_2_2_2);
    omni_number_unpack_byte(p_2222, out, OMNI_PACK_2_2_2_2);
    OMNI_TEST_ASSERT(out[0] == 1 && out[1] == 2 && out[2] == 0 && out[3] == 3,
                     "2-2-2-2 round-trip unpack must match input");

    /* 2. Test 2-1-2-1-2 (Palindromic Equilibrium: 00/0/00/0/00) */
    uint8_t p_pal = omni_number_pack_byte(3, 1, 2, 0, 1, OMNI_PACK_2_1_2_1_2);
    omni_number_unpack_byte(p_pal, out, OMNI_PACK_2_1_2_1_2);
    OMNI_TEST_ASSERT(out[0] == 3 && out[1] == 1 && out[2] == 2 &&
                     out[3] == 0 && out[4] == 1,
                     "2-1-2-1-2 round-trip unpack must match input");

    /* 3. Test 1-2-1-2-2 (Prefix Carry) */
    uint8_t p_pre = omni_number_pack_byte(1, 3, 0, 2, 1, OMNI_PACK_1_2_1_2_2);
    omni_number_unpack_byte(p_pre, out, OMNI_PACK_1_2_1_2_2);
    OMNI_TEST_ASSERT(out[0] == 1 && out[1] == 3 && out[2] == 0 &&
                     out[3] == 2 && out[4] == 1,
                     "1-2-1-2-2 round-trip unpack must match input");

    /* 4. Test 2-2-1-2-1 (Postfix Carry) */
    uint8_t p_post = omni_number_pack_byte(2, 1, 1, 3, 0, OMNI_PACK_2_2_1_2_1);
    omni_number_unpack_byte(p_post, out, OMNI_PACK_2_2_1_2_1);
    OMNI_TEST_ASSERT(out[0] == 2 && out[1] == 1 && out[2] == 1 &&
                     out[3] == 3 && out[4] == 0,
                     "2-2-1-2-1 round-trip unpack must match input");

    /* 5. Test 3-3-2 (3-Space Directional Partition) */
    uint8_t p_332 = omni_number_pack_byte(7, 5, 2, 0, 0, OMNI_PACK_3_3_2);
    omni_number_unpack_byte(p_332, out, OMNI_PACK_3_3_2);
    OMNI_TEST_ASSERT(out[0] == 7 && out[1] == 5 && out[2] == 2,
                     "3-3-2 round-trip unpack must match input");

    /* 6. Test 3-2-3 (3-Space Directional Partition) */
    uint8_t p_323 = omni_number_pack_byte(6, 3, 5, 0, 0, OMNI_PACK_3_2_3);
    omni_number_unpack_byte(p_323, out, OMNI_PACK_3_2_3);
    OMNI_TEST_ASSERT(out[0] == 6 && out[1] == 3 && out[2] == 5,
                     "3-2-3 round-trip unpack must match input");

    /* 7. Test 2-3-3 (3-Space Directional Partition) */
    uint8_t p_233 = omni_number_pack_byte(3, 7, 4, 0, 0, OMNI_PACK_2_3_3);
    omni_number_unpack_byte(p_233, out, OMNI_PACK_2_3_3);
    OMNI_TEST_ASSERT(out[0] == 3 && out[1] == 7 && out[2] == 4,
                     "2-3-3 round-trip unpack must match input");

    return 0;
}

static int test_number_equilibrium_law(void) {
    /* ALU Equilibrium Law: (+1) plus (-1) <=> 0 */
    bool eq_holds = omni_number_verify_equilibrium();
    OMNI_TEST_ASSERT(eq_holds == true, "Core ALU Zero-Sum Equilibrium Law must hold");
    return 0;
}

static int test_number_addition_and_subtraction(void) {
    omni_number_t a = omni_number_from_scalar(4000);
    omni_number_t b = omni_number_from_scalar(96);
    omni_number_t sum = omni_number_add(a, b);

    OMNI_TEST_ASSERT(sum.scalar == 4096, "4000 + 96 must equal 4096");
    OMNI_TEST_ASSERT(sum.is_enclosed == true, "Sum must be enclosed");
```

```
    /* Test inverse cancellation via subtraction: a - a == 0 */
    omni_number_t zero = omni_number_sub(sum, sum);
    OMNI_TEST_ASSERT(zero.scalar == 0, "4096 - 4096 must equal 0");
    OMNI_TEST_ASSERT(omni_number_trits_is_equilibrium(zero.packed_trits) == true,
                     "Zero difference must be in packed equilibrium");

    /* Test negation: -4096 */
    omni_number_t neg_sum = omni_number_neg(sum);
    OMNI_TEST_ASSERT(neg_sum.scalar == -4096, "Negated 4096 must equal -4096");
    OMNI_TEST_ASSERT(omni_number_add(sum, neg_sum).scalar == 0,
                     "Summing with negated counterpart must yield 0");

    return 0;
}

static int test_number_bounds_and_enclosure(void) {
    omni_number_t valid_num = omni_number_from_scalar(100);
    OMNI_TEST_ASSERT(omni_number_verify_bounds(&valid_num) == true,
                     "Valid number must pass bounds check");

    omni_number_t corrupted = valid_num;
    corrupted.is_enclosed = false;
    OMNI_TEST_ASSERT(omni_number_verify_bounds(&corrupted) == false,
                     "Unenclosed number must fail bounds check");

    /* Test null trap tripping */
    omni_number_t trap_num = valid_num;
    trap_num.packed_trits |= 0x03; /* Inject 11 */
    OMNI_TEST_ASSERT(omni_number_verify_bounds(&trap_num) == false,
                     "Number with NULL collision trap must fail bounds check");

    return 0;
}

static int test_number_lifecycle_rolls(void) {
    omni_number_context_t ctx;
    omni_number_open(&ctx, 4096, OMNI_TARGET_ACCUMULATOR);

    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_ARMED,
                     "Tripwire must be ARMED upon open");
    OMNI_TEST_ASSERT(ctx.input_quantity == 4096,
                     "Input quantity must match 4096");

    /* Verify individual operational rolls */
    omni_number_inscribe(&ctx, 4096);
    OMNI_TEST_ASSERT(ctx.number.scalar == 4096, "Roll 1 Inscribe must set scalar");

    omni_number_transduce_context(&ctx);
    OMNI_TEST_ASSERT(!omni_number_trits_has_null(ctx.number.packed_trits),
                     "Roll 2 Transduce must produce trap-free trits");

    OMNI_TEST_ASSERT(omni_number_guard(&ctx) == OMNI_STATE_PROCEED,
                     "Roll 3 Guard assertion must PROCEED");

    OMNI_TEST_ASSERT(omni_number_calculate(&ctx) == OMNI_STATE_PROCEED,
                     "Roll 4 Calculate zero-sum balance must PROCEED");

    OMNI_TEST_ASSERT(omni_number_sequence(&ctx.number, NULL) == true,
                     "Roll 5 Sequence transition must succeed");

    /* Unified lifecycle execution */
    omni_quad_state_t exec_status = omni_number_execute(&ctx);
    OMNI_TEST_ASSERT(exec_status == OMNI_STATE_PROCEED,
                     "Number execute must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.number.is_enclosed == true,
                     "Number must be enclosed after execute");

    omni_quad_state_t close_status = omni_number_close(&ctx);
    OMNI_TEST_ASSERT(close_status == OMNI_STATE_PROCEED,
                     "Number close must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_DISARMED,
                     "Tripwire must be DISARMED upon clean close");

    return 0;
}

/* ============================================================================
```

```
 * BLOCK 3: BODY PRIMITIVES & TEST SUITE RUNNER
 * ========================================================================== */

int test_number_suite(void) {
    printf("[SUITE: Number Invariant Type]\n");
    OMNI_TEST_RUN(test_number_bijective_roundtrip, "Bijective Radix-3 Transduction");
    OMNI_TEST_RUN(test_number_dual_rail_bitplanes, "Dual-Rail Bit-Planes & Negation");
    OMNI_TEST_RUN(test_number_dynamic_packing_topologies, "Dynamic Packing Topologies");
    OMNI_TEST_RUN(test_number_equilibrium_law, "ALU Zero-Sum Equilibrium Law");
    OMNI_TEST_RUN(test_number_addition_and_subtraction, "Balanced Addition & Subtraction");
    OMNI_TEST_RUN(test_number_bounds_and_enclosure, "Single-Word Enclosure Bounds");
    OMNI_TEST_RUN(test_number_lifecycle_rolls, "4-Block Lifecycle Rolls");
    return 0;
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_site.c

C | 120 LINES | 556 WORDS | 5.5 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & TEST TOPOLOGY
 * ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Site } ] )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Validates the physical silicon address type < Site > in isolation:
 *   - Null pointer address rejection and safety tripwire
 *   - Hardware Routing Law (MMIO ports vs RAM spaces)
 *   - Relative displacement and pointer offset arithmetic
 *   - Volatile memory semantics for bare-metal registers
 *   - Full 4-block lifecycle execution (open, execute, close)
 *   - Witness: Scripture Genesis 28:16 KJV
 *     "And Jacob awaked out of his sleep, and he said, Surely the Lord is in
 *      this place; and I knew it not."
 *   - Semantics: { SiliconCoordinates :: HardwareRoutingLaw :: NonNullInterlock }
 * ============================================================================ */

#include "test_common.h"

/* ============================================================================
 * BLOCK 2: OPENING CONTEXT & FIXTURE SETUP
 * ============================================================================ */

static int test_site_routing_classification(void) {
    /* VGA buffer coordinate (0xB8000) must route to MMIO */
    omni_site_t vga_site = omni_site_from_address(OMNI_SITE_VGA_BUFFER);
    OMNI_TEST_ASSERT(vga_site.routing == OMNI_ROUTING_HARDWARE_MMIO, "VGA buffer must classify as HARDWARE_MMIO");
    OMNI_TEST_ASSERT(omni_site_is_valid(&vga_site) == true, "VGA site must be valid");

    /* COM1 serial port (0x3F8) must route to MMIO */
    omni_site_t com1_site = omni_site_from_address(OMNI_SITE_COM1);
    OMNI_TEST_ASSERT(com1_site.routing == OMNI_ROUTING_HARDWARE_MMIO, "COM1 port must classify as HARDWARE_MMIO");

    /* Standard MBR memory location (0x7C00) must route to RAM */
    omni_site_t mbr_site = omni_site_from_address(OMNI_SITE_MBR_BOOT);
    OMNI_TEST_ASSERT(mbr_site.routing == OMNI_ROUTING_RAM_PHYSICAL, "MBR origin must classify as RAM_PHYSICAL");

    return 0;
}

static int test_site_null_tripwire(void) {
    /* Null address (0x0) must be invalid */
    omni_site_t null_site = omni_site_from_address(OMNI_SITE_NULL);
    OMNI_TEST_ASSERT(omni_site_is_valid(&null_site) == false, "Null site must be invalid");

    /* Executing lifecycle with NULL coordinate must trip the interlock and ROLLBACK */
    omni_site_context_t ctx;
    omni_site_open(&ctx, 0);
    ctx.input_coordinate = 0; /* Force null */
    omni_quad_state_t exec_st = omni_site_execute(&ctx);
    OMNI_TEST_ASSERT(exec_st == OMNI_STATE_ROLLBACK, "Null address execute must trigger ROLLBACK (-1)");
    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_TRIPPED, "Tripwire must be TRIPPED on null address");

    return 0;
}

static int test_site_offset_and_relational(void) {
    uint8_t dummy_buffer[16];
    uintptr_t base_addr = (uintptr_t)&dummy_buffer[0];

    omni_site_t base_site = omni_site_from_address(base_addr);
    omni_site_t offset_site = omni_site_offset(&base_site, 8);

    OMNI_TEST_ASSERT(offset_site.coordinate == base_addr + 8, "Displaced site coordinate must be base + 8");
    OMNI_TEST_ASSERT(omni_site_eq(&base_site, &base_site) == true, "Identical sites must evaluate equal");
    OMNI_TEST_ASSERT(omni_site_eq(&base_site, &offset_site) == false, "Different sites must evaluate not equal");

    return 0;
}

static int test_site_volatile_read_write(void) {
    uint8_t buffer[4] = {0, 0, 0, 0};
    uintptr_t addr = (uintptr_t)&buffer[0];
```

```
    omni_site_t site = omni_site_from_address(addr);

    /* Write via volatile primitives */
    omni_site_write_u8(&site, 0, 0xAB);
    omni_site_write_u8(&site, 1, 0xCD);

    /* Read back via volatile primitives */
    uint8_t r0 = omni_site_read_u8(&site, 0);
    uint8_t r1 = omni_site_read_u8(&site, 1);

    OMNI_TEST_ASSERT(r0 == 0xAB, "Volatile read at offset 0 must match written 0xAB");
    OMNI_TEST_ASSERT(r1 == 0xCD, "Volatile read at offset 1 must match written 0xCD");

    return 0;
}

static int test_site_lifecycle_rolls(void) {
    omni_site_context_t ctx;
    omni_site_open(&ctx, OMNI_SITE_VGA_BUFFER);

    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_ARMED, "Tripwire must be ARMED upon open");

    omni_quad_state_t exec_status = omni_site_execute(&ctx);
    OMNI_TEST_ASSERT(exec_status == OMNI_STATE_PROCEED, "Site execute must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.site.routing == OMNI_ROUTING_HARDWARE_MMIO, "VGA site routing must be HARDWARE_MMIO");

    omni_quad_state_t close_status = omni_site_close(&ctx);
    OMNI_TEST_ASSERT(close_status == OMNI_STATE_PROCEED, "Site close must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_DISARMED, "Tripwire must be DISARMED upon clean close");

    return 0;
}

/* ============================================================================
 * BLOCK 3: BODY PRIMITIVES & TEST SUITE RUNNER
 * ============================================================================ */

int test_site_suite(void) {
    printf("[SUITE: Site Invariant Type]\n");
    OMNI_TEST_RUN(test_site_routing_classification, "Hardware Routing Law (MMIO vs RAM)");
    OMNI_TEST_RUN(test_site_null_tripwire, "Null Pointer Rejection & Tripwire Interlock");
    OMNI_TEST_RUN(test_site_offset_and_relational, "Offset Arithmetic & Relational Equality");
    OMNI_TEST_RUN(test_site_volatile_read_write, "Volatile Memory Read / Write Semantics");
    OMNI_TEST_RUN(test_site_lifecycle_rolls, "4-Block Lifecycle Rolls");
    return 0;
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/transpile variant/tests/c/unit/test_state.c

C | 119 LINES | 606 WORDS | 5.6 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & TEST TOPOLOGY
 * ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { State } ] )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Validates the causal truth governor invariant type < State > in isolation:
 *   - Quad-State named registers: [ STATE TURN BOUND IT ]
 *   - Canonical Loop Law cadence and single-step detent transitions:
 *       WHEN IT < BOUND  --> PROCEED (+1)
 *       !    IT > BOUND  --> AWAIT (0)
 *       ::   IT == BOUND --> HALT (NULL)
 *   - State predicates and register reset mechanics
 *   - Full 4-block lifecycle execution (open, execute, close)
 *   - Witness: Scripture Ecclesiastes 3:1 KJV
 *     "To every thing there is a season, and a time to every purpose under
 *      the heaven:"
 *   - Semantics: { QuadStateTruth :: CanonicalLoopCadence :: CausalDetents }
 * ============================================================================ */

#include "test_common.h"

/* ============================================================================
 * BLOCK 2: OPENING CONTEXT & FIXTURE SETUP
 * ============================================================================ */

static int test_state_canonical_loop(void) {
    omni_state_t s = omni_state_create(OMNI_STATE_PROCEED, 3);

    OMNI_TEST_ASSERT(s.registers.bound == 3, "Loop bound must be 3");
    OMNI_TEST_ASSERT(s.registers.it == 0, "Initial IT must be 0");
    OMNI_TEST_ASSERT(s.registers.turn == 0, "Initial TURN must be 0");
    OMNI_TEST_ASSERT(omni_state_is_proceeding(&s) == true, "Initial state must be PROCEED");

    /* Step 1: IT 0 < 3 -> increments to IT=1, TURN=1, returns PROCEED */
    omni_quad_state_t st1 = omni_state_step_loop(&s.registers);
    OMNI_TEST_ASSERT(st1 == OMNI_STATE_PROCEED, "Step 1 must PROCEED");
    OMNI_TEST_ASSERT(s.registers.it == 1 && s.registers.turn == 1, "IT and TURN must be 1");

    /* Step 2: IT 1 < 3 -> increments to IT=2, TURN=2, returns PROCEED */
    omni_quad_state_t st2 = omni_state_step_loop(&s.registers);
    OMNI_TEST_ASSERT(st2 == OMNI_STATE_PROCEED, "Step 2 must PROCEED");
    OMNI_TEST_ASSERT(s.registers.it == 2 && s.registers.turn == 2, "IT and TURN must be 2");

    /* Step 3: IT 2 < 3 -> increments to IT=3, TURN=3, returns PROCEED */
    omni_quad_state_t st3 = omni_state_step_loop(&s.registers);
    OMNI_TEST_ASSERT(st3 == OMNI_STATE_PROCEED, "Step 3 must PROCEED");
    OMNI_TEST_ASSERT(s.registers.it == 3 && s.registers.turn == 3, "IT and TURN must be 3");

    /* Step 4: IT == BOUND (3 == 3) -> Reaches terminal detent HALT:NULL */
    omni_quad_state_t st4 = omni_state_step_loop(&s.registers);
    OMNI_TEST_ASSERT(st4 == OMNI_STATE_HALT, "Step 4 (IT == BOUND) must reach HALT:NULL detent");
    OMNI_TEST_ASSERT(omni_state_is_halted(&s) == true, "State must report halted");

    return 0;
}

static int test_state_await_transition(void) {
    omni_state_t s = omni_state_create(OMNI_STATE_PROCEED, 4);

    /* Manually position IT > BOUND to simulate overrun awaiting input */
    s.registers.it = 5;
    omni_quad_state_t await_st = omni_state_step_loop(&s.registers);
    OMNI_TEST_ASSERT(await_st == OMNI_STATE_AWAIT, "IT > BOUND must transition to AWAIT (0)");
    OMNI_TEST_ASSERT(omni_state_is_awaiting(&s) == true, "State engine must report awaiting");

    return 0;
}

static int test_state_predicates_and_reset(void) {
    omni_state_t s = omni_state_create(OMNI_STATE_PROCEED, 2);

    /* Step until halt */
    omni_state_step_loop(&s.registers);
```

```
    omni_state_step_loop(&s.registers);
    omni_state_step_loop(&s.registers);
    OMNI_TEST_ASSERT(omni_state_is_halted(&s) == true, "Engine must be halted");

    /* Reset engine */
    omni_state_reset(&s, 5);
    OMNI_TEST_ASSERT(s.registers.bound == 5, "New bound must be 5");
    OMNI_TEST_ASSERT(s.registers.it == 0, "IT must reset to 0");
    OMNI_TEST_ASSERT(s.registers.turn == 0, "TURN must reset to 0");
    OMNI_TEST_ASSERT(omni_state_is_proceeding(&s) == true, "Reset must restore PROCEED status");

    return 0;
}

static int test_state_lifecycle_rolls(void) {
    omni_state_context_t ctx;
    omni_state_open(&ctx, OMNI_STATE_PROCEED, 4);

    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_ARMED, "Tripwire must be ARMED upon open");

    /* Execute full governance cadence */
    omni_quad_state_t exec_status = omni_state_execute(&ctx);
    OMNI_TEST_ASSERT(exec_status == OMNI_STATE_PROCEED, "State execute must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.state_engine.registers.it == 4, "Final IT must equal bound 4");
    OMNI_TEST_ASSERT(ctx.state_engine.registers.state == OMNI_STATE_HALT, "Registers must reach HALT:NULL detent");

    omni_quad_state_t close_status = omni_state_close(&ctx);
    OMNI_TEST_ASSERT(close_status == OMNI_STATE_PROCEED, "State close must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_DISARMED, "Tripwire must be DISARMED upon clean close");

    return 0;
}

/* ============================================================================
 * BLOCK 3: BODY PRIMITIVES & TEST SUITE RUNNER
 * ============================================================================ */

int test_state_suite(void) {
    printf("[SUITE: State Invariant Type]\n");
    OMNI_TEST_RUN(test_state_canonical_loop, "Canonical Loop Law Cadence (IT < BOUND -> HALT)");
    OMNI_TEST_RUN(test_state_await_transition, "Standby Transition (IT > BOUND -> AWAIT)");
    OMNI_TEST_RUN(test_state_predicates_and_reset, "Predicate Queries & Engine Reset");
    OMNI_TEST_RUN(test_state_lifecycle_rolls, "4-Block Lifecycle Rolls");
    return 0;
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/README.md

MD | 149 LINES | 953 WORDS | 6.4 KB

```
# OmniCode Universal Substrate: The Bedrock of Balance

Welcome to the **Universal Foundation** of OmniCode!

The `universal/` folder is the bedrock upon which all OmniCode types, nouns, verbs, and tracks are built. It defines the physics, mathematics, and causal logic of the OmniCode universe.

---

## 1. The Core Philosophy: Balanced Ternary & Equilibrium

Most computer systems today are built on **binary logic** (`0` and `1`). Binary is inherently one-sided: it forces the world into rigid opposites (on/off, true/false, presence/absence). Representing negative numbers or undoing mistakes in binary requires awkward tricks (such as "two's complement" arithmetic).

OmniCode is founded on **Balanced Ternary**:
> *"A false balance is abomination to the LORD: but a just weight is his delight."*
> — **Proverbs 11:1 KJV**

In balanced ternary, every basic digit (called a **trit**) can take one of three natural states centered symmetrically around zero:

- **`+1` (Positive / Surplus):** Forward motion, affirmative creation, advance.
- **`0` (Equilibrium / Rest):** Perfect balance, standstill, awaiting input.
- **`-1` (Negative / Deficit):** Backward pull, restoration, unwinding an error.

Because balanced ternary is completely symmetrical around zero, **negation is free**, subtraction is simply adding the opposite, and balance is built directly into the fabric of computation!

---

## 2. Quad-State Causal Truth: How Time & Logic Flow

OmniCode replaces simple Boolean true/false with **Quad-State Causal Truth**. Instead of just asking *"Is this true or false?"*, OmniCode asks: *"In which direction should computation flow?"*

| State | Trit Value | Meaning | Real-World Analogy |
| :--- | :---: | :--- | :--- |
| **`PROCEED`** | `+1` | Forward Causation | The green light. Everything is sound; advance the step. |
| **`AWAIT`** | `0` | Equilibrium Standby | The amber pause. The system is balanced, waiting for input or a timer. |
| **`ROLLBACK`** | `-1` | Retroactive Reversal | The rewind button. A safety guard was tripped; restore the last safe snapshot. |
| **`HALT`** | `NULL` (`-2`) | Terminal Detent | The emergency brake. Stop the processor clock immediately. |

---

## 3. Connective Operators as Natural Human Conjunctions

In natural human speech, we don't just say `AND` or `OR`. We express nuanced relationships with words like *"and"*, *"but"*, *"or"*, and *"in relation to"*.

OmniCode translates these intuitive conjunctions into exact mathematical logic gates:

### `::` (AND / Cumulative Conjunction)

- **Human Meaning:** *"Do this AND do that simultaneously."*
- **Behavior:** Both sides must succeed (`PROCEED`). If either is waiting, the result waits (`AWAIT`). If either fails, the whole conjunction rolls back (`ROLLBACK`).

### `!` (BUT / Adversative Caveat)

- **Human Meaning:** *"Execute this action, BUT only if this safety condition holds."*
- **Behavior:** The action advances, *unless* the caveat fails or halts, in which case the action is immediately blocked and rolled back.

### `|` (OR / Disjunctive Valve)

- **Human Meaning:** *"Take path A OR take path B."*
- **Behavior:** If either path can proceed, computation advances. If both are blocked, it rolls back.

### `--` (NOT / Polarity Inversion)

- **Human Meaning:** *"Flip the truth value around the center."*
```

- **Behavior:** Turns `PROCEED (+1)` into `ROLLBACK (-1)` and vice versa, leaving resting `AWAIT (0)` undisturbed.

`!=` (Inequality / "Not to Be")

- **Human Meaning:** *"Verify that these two states or values are different."*

`=!` (Is Not / "To Be Not")

- **Human Meaning:** *"Confirm that current state is NOT a specific forbidden condition."*

`=~` (Liken To / Resemblance)

- **Human Meaning:** *"See if these two ideas are compatible or like one another."*

`:` (In Relation To / Perspective)

- **Human Meaning:** *"View this action through the lens of this context."*

---

4. The Dual-Rail Silicon Secret

How can modern computers (which only have binary wires) run OmniCode's balanced ternary logic at native speeds?

OmniCode uses **Dual-Rail Polar Bit-Planes**:

- Every balanced trit is physically represented using a pair of bits: `(Negative Rail, Positive Rail)`.
  - `00`: **Equilibrium (0)** — Neither wire active.
  - `01`: **Positive (+1)** — Positive rail active.
  - `10`: **Negative (-1)** — Negative rail active.
  - `11`: **Collision / Trap (NULL)** — Both wires active simultaneously, which is an illegal hardware fault!

Instantaneous Negation (Rail Swapping)

To negate a 64-bit word of 32 trits in conventional math, a computer has to perform multiple arithmetic operations with borrowing.
In OmniCode, **negation is just swapping the positive and negative bit-planes**! The CPU does this in just two hardware clock cycles, achieving instant SIMD (Single Instruction, Multiple Data) negation.

---

5. Directory Contents & Architecture

```bash
universal/
├── README.md               # This foundational guide
└── transpiled/
    ├── compiled/c/         # Freestanding C99/C11 Hardware Backbone
    │   ├── Makefile        # Pure POSIX build script
    │   ├── compile_flags.txt
    │   ├── include/
    │   │   ├── omni_base.h # Invariant axioms, ALU arithmetic, connectives
    │   │   └── omni.h      # Master umbrella header for all types
    │   └── tests/
    │       ├── test_common.h   # Minimal freestanding assertion macros
    │       └── test_universal.c# Unit testbed verifying universal laws in isolation
    └── interpreted/lua/    # Dynamic Coroutine Process Ribbon
        ├── tape_engine.lua # Coroutine fiber runner, snapshots, connectives
        └── tests/
            ├── test_common.lua     # Assertion suite for Lua
            └── test_tape_engine.lua# Unit testbed verifying tape engine in isolation
```

---

6. Running Universal Tests

We always test the universal foundation **before** testing any individual type:

```bash
# 1. Test the C Bare-Metal Bedrock
cd transpiled/compiled/c
make test

# 2. Test the Lua Coroutine Tape Engine
cd ../../interpreted/lua
lua tests/test_tape_engine.lua
```

```
Both test suites verify:

- Hardware preconditions and platform word sizes
- Balanced ternary arithmetic and carry propagation
- Dual-rail encoding, decoding, and collision traps
- The complete suite of connectives (`!`, `::`, `|`, `--`, `!=`, `=!`, `=~`, `:`)
- State vs. literal disambiguation
- Tripwire arming, tripping, and transactional rollback
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/interpreted/lua/tape_engine.lua

LUA | 368 LINES | 1404 WORDS | 12.5 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & ENGINE TOPOLOGY
-- ( Engine Core: < Lexicon { Types of Ideas } > & [ Process { Tape Engine } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   The interpreted execution ribbon companion to the OmniCode C Backbone.
--   Drives dynamic temporal causation, coroutines, and roll kinematics:
--   - Coroutine-based fiber stepping (yielding on AWAIT, resuming on tick)
--   - Pre-roll and post-roll dynamic event hooks
--   - State snapshotting and active rollback unwinding
--   - Structured handles over C memory layouts and invariants
--   - Witness: Scripture Proverbs 16:9 KJV
--     "A man's heart deviseth his way: but the LORD directeth his steps."
--   - Semantics: { TapeKinematics :: CoroutineFiber :: QuadStateDetents }
-- ================================================================================

local TapeEngine = {
    -- Quad-State Truth Constants
    STATE_ROLLBACK = -1,     -- -1 : Retroactive Reversal / Error Unwind
    STATE_AWAIT    =  0,     --  0 : Equilibrium Standby / Hold Pending
    STATE_PROCEED  =  1,     -- +1 : Forward Causation / Affirmative Advance
    STATE_HALT     = "NULL", -- NULL : Terminal Detent / Stop Processor Clock

    -- Tripwire Interlock States
    TRIPWIRE_DISARMED = 0,
    TRIPWIRE_ARMED    = 1,
    TRIPWIRE_TRIPPED  = 2,

    -- Dual-Rail 2-Bit Trit Constants
    TRIT_PAIR_ZERO    = 0, -- 0b00: Equilibrium (0)
    TRIT_PAIR_POS     = 1, -- 0b01: Positive (+1)
    TRIT_PAIR_NEG     = 2, -- 0b10: Negative (-1)
    TRIT_PAIR_NULL    = 3, -- 0b11: Collision / NULL trap

    -- Dual-Rail Bitplane 64-Bit Masks
    BITPLANE_POS_MASK = 0x5555555555555555,
    BITPLANE_NEG_MASK = ~0x5555555555555555
}

-- Detect optional LuaJIT FFI for direct zero-copy C backbone linkage
local has_ffi, ffi = pcall(require, "ffi")
TapeEngine.has_ffi = has_ffi
TapeEngine.ffi = has_ffi and ffi or nil

---
-- Encodes a discrete balanced trit (-1, 0, 1) to a 2-bit pair.
function TapeEngine.trit_to_pair(trit)
    if trit == 0 then return TapeEngine.TRIT_PAIR_ZERO
    elseif trit == 1 then return TapeEngine.TRIT_PAIR_POS
    elseif trit == -1 then return TapeEngine.TRIT_PAIR_NEG
    else return TapeEngine.TRIT_PAIR_NULL end
end

---
-- Decodes a 2-bit pair into a discrete balanced trit.
-- Returns trit, ok_boolean (false on 0b11 collision).
function TapeEngine.pair_to_trit(code)
    local c = (code or 0) & 0x03
    if c == TapeEngine.TRIT_PAIR_ZERO then return 0, true
    elseif c == TapeEngine.TRIT_PAIR_POS then return 1, true
    elseif c == TapeEngine.TRIT_PAIR_NEG then return -1, true
    else return 0, false end
end

---
-- SIMD rail-swap negation of 64-bit packed dual-rail word.
function TapeEngine.negate_dual_rail(packed)
    local pos = packed & TapeEngine.BITPLANE_POS_MASK
    local neg = packed & TapeEngine.BITPLANE_NEG_MASK
    return ((pos << 1) & TapeEngine.BITPLANE_NEG_MASK) |
           ((neg >> 1) & TapeEngine.BITPLANE_POS_MASK)
end

```

```
---
-- Detects whether any trit position has both rails active (NULL trap).
function TapeEngine.has_null_collision(packed)
    local pos = packed & TapeEngine.BITPLANE_POS_MASK
    local neg = (packed >> 1) & TapeEngine.BITPLANE_POS_MASK
    return (pos & neg) ~= 0
end

-- ============================================================================
-- BLOCK 2: OPENING BLOCK
-- ( Subsystem Initialization, Preconditions & Tape Mounting )
-- ============================================================================

---
-- Deep copy snapshot utility for transactional rollback safety.
-- @param ctx The context table to clone.
-- @return Independent deep copy of the context.
local function make_snapshot(ctx)
    local snap = {}
    for k, v in pairs(ctx) do
        if type(v) == "table" then
            snap[k] = make_snapshot(v)
        else
            snap[k] = v
        end
    end
    return snap
end

---
-- Creates and mounts an interpreted Tape Process Ribbon.
-- @param spec Table containing metadata, constants, and declared rolls.
-- @return Mounted tape object.
function TapeEngine.create_tape(spec)
    local tape = {
        metadata    = spec.metadata or {},
        constants   = spec.constants or {},
        rolls       = spec.rolls or {},
        hooks       = {
            before_roll = {},
            after_roll  = {},
            on_rollback = {}
        },
        snapshots   = {},
        history     = {}
    }

    ---
    -- Registers a dynamic hook invoked immediately before a roll executes.
    function tape:before(fn)
        table.insert(self.hooks.before_roll, fn)
        return self
    end

    ---
    -- Registers a dynamic hook invoked immediately after a roll executes.
    function tape:after(fn)
        table.insert(self.hooks.after_roll, fn)
        return self
    end

    ---
    -- Registers a rollback handler invoked when a guard or roll fails.
    function tape:on_rollback(fn)
        table.insert(self.hooks.on_rollback, fn)
        return self
    end

    ---
    -- Spawns a coroutine fiber driving step-by-step execution.
    function tape:spawn_coroutine(ctx)
        return coroutine.create(function()
            for idx, roll in ipairs(self.rolls) do
                table.insert(self.snapshots, make_snapshot(ctx))

                for _, h in ipairs(self.hooks.before_roll) do
                    h(idx, roll.name, ctx)
                end
```

```
                local status = roll.action(ctx)
                ctx.status = status

                for _, h in ipairs(self.hooks.after_roll) do
                    h(idx, roll.name, ctx, status)
                end

                table.insert(self.history, {
                    roll_index = idx,
                    name       = roll.name,
                    status     = status
                })

                if status == TapeEngine.STATE_ROLLBACK then
                    for _, h in ipairs(self.hooks.on_rollback) do
                        h(idx, roll.name, ctx)
                    end
                    if #self.snapshots > 0 then
                        local last_good = table.remove(self.snapshots)
                        for k, v in pairs(last_good) do
                            ctx[k] = v
                        end
                    end
                    coroutine.yield(TapeEngine.STATE_ROLLBACK, idx, roll.name)
                    return TapeEngine.STATE_ROLLBACK
                elseif status == TapeEngine.STATE_AWAIT then
                    coroutine.yield("AWAIT", idx, roll.name, ctx)
                elseif status == TapeEngine.STATE_HALT then
                    coroutine.yield(TapeEngine.STATE_HALT, idx, roll.name)
                    return TapeEngine.STATE_HALT
                end
            end

            return TapeEngine.STATE_PROCEED, "Tape sealed in PROCEED (+1)"
        end)
    end

    ---
    -- Executes all rolls in a synchronous loop, automatically resuming AWAITs.
    function tape:execute_all(ctx)
        local co = self:spawn_coroutine(ctx)
        while coroutine.status(co) ~= "dead" do
            local ok, event, roll_idx = coroutine.resume(co)
            if not ok then
                error("Tape execution fatal error: " .. tostring(event))
            end
            if event == "AWAIT" then
                ctx.status = TapeEngine.STATE_PROCEED
            elseif event == TapeEngine.STATE_PROCEED
                or event == TapeEngine.STATE_HALT
                or event == TapeEngine.STATE_ROLLBACK then
                return event, roll_idx
            end
        end
        return ctx.status
    end

    return tape
end

-- ============================================================================
-- BLOCK 3: BODY BLOCK
-- ( Connectives & Evaluators )
-- ============================================================================

---
-- Evaluates adversative caveat gate: A ! B (A executes, BUT B must hold).
function TapeEngine.op_but(action, caveat)
    if caveat == TapeEngine.STATE_ROLLBACK then
        return TapeEngine.STATE_ROLLBACK
    end
    if caveat == TapeEngine.STATE_HALT then
        return TapeEngine.STATE_HALT
    end
    return action
end

```

```
---
-- Evaluates cumulative conjunction: A :: B (A AND B simultaneous).
function TapeEngine.op_and(a, b)
    if a == TapeEngine.STATE_HALT or b == TapeEngine.STATE_HALT then
        return TapeEngine.STATE_HALT
    end
    if a == TapeEngine.STATE_ROLLBACK or b == TapeEngine.STATE_ROLLBACK then
        return TapeEngine.STATE_ROLLBACK
    end
    if a == TapeEngine.STATE_AWAIT or b == TapeEngine.STATE_AWAIT then
        return TapeEngine.STATE_AWAIT
    end
    return TapeEngine.STATE_PROCEED
end

---
-- Evaluates disjunctive alternative valve: A | B (A OR B).
function TapeEngine.op_or(a, b)
    if a == TapeEngine.STATE_PROCEED or b == TapeEngine.STATE_PROCEED then
        return TapeEngine.STATE_PROCEED
    end
    if a == TapeEngine.STATE_AWAIT or b == TapeEngine.STATE_AWAIT then
        return TapeEngine.STATE_AWAIT
    end
    if a == TapeEngine.STATE_ROLLBACK and b == TapeEngine.STATE_ROLLBACK then
        return TapeEngine.STATE_ROLLBACK
    end
    return TapeEngine.STATE_HALT
end

---
-- Evaluates balanced ternary truth inversion: -- A (NOT A).
function TapeEngine.op_not(a)
    if a == TapeEngine.STATE_PROCEED then
        return TapeEngine.STATE_ROLLBACK
    end
    if a == TapeEngine.STATE_ROLLBACK then
        return TapeEngine.STATE_PROCEED
    end
    return a
end

---
-- Evaluates inequality verification: A != B (A not to be B).
function TapeEngine.op_neq(a, b)
    if a == TapeEngine.STATE_HALT or b == TapeEngine.STATE_HALT then
        return TapeEngine.STATE_HALT
    end
    return (a ~= b) and TapeEngine.STATE_PROCEED or TapeEngine.STATE_ROLLBACK
end

---
-- Evaluates predicate is-not check: A =! B (State A is not target B).
function TapeEngine.op_is_not(a, target)
    return (a ~= target) and TapeEngine.STATE_PROCEED
                         or TapeEngine.STATE_ROLLBACK
end

---
-- Evaluates liken to / compatibility match: A =~ B.
function TapeEngine.op_liken(a, b)
    if a == TapeEngine.STATE_HALT or b == TapeEngine.STATE_HALT then
        return TapeEngine.STATE_HALT
    end
    if a == b then return TapeEngine.STATE_PROCEED end
    if (a == TapeEngine.STATE_PROCEED or a == TapeEngine.STATE_AWAIT) and
       (b == TapeEngine.STATE_PROCEED or b == TapeEngine.STATE_AWAIT) then
        return TapeEngine.STATE_PROCEED
    end
    return TapeEngine.STATE_ROLLBACK
end

---
-- Evaluates relation projection valve: Context : Aspect.
function TapeEngine.op_relate(ctx, asp)
    if ctx == TapeEngine.STATE_HALT then return TapeEngine.STATE_HALT end
    if ctx == TapeEngine.STATE_ROLLBACK then
        return TapeEngine.STATE_ROLLBACK
```

```
    end
    return asp
end

---
-- Determines if a value represents an OmniCode causal state.
function TapeEngine.is_state(val)
    return val == TapeEngine.STATE_PROCEED or
           val == TapeEngine.STATE_AWAIT   or
           val == TapeEngine.STATE_ROLLBACK or
           val == TapeEngine.STATE_HALT
end

---
-- Disambiguates an intended literal from a quad-state detent.
-- Handles air-quoted "0" or cast { 0 } / [ 0 ] / < 0 > table envelopes.
-- Returns (numeric_value, is_literal_bool).
function TapeEngine.resolve_literal(val)
    if type(val) == "string" and (val == "0" or val == "1" or val == "-1") then
        return tonumber(val), true
    elseif type(val) == "table" and val.is_literal then
        return val.value, true
    elseif type(val) == "table" and val[1] ~= nil and #val == 1 then
        return val[1], true
    elseif type(val) == "number" then
        return val, false
    else
        return val, false
    end
end

-- ============================================================================
-- BLOCK 4: CLOSING BLOCK
-- ( Integrity Seals & Commit Verification )
-- ============================================================================

---
-- Evaluates an invariant guard condition inside a roll action.
function TapeEngine.assert_guard(condition, ctx, error_reason)
    if condition then
        return TapeEngine.STATE_PROCEED
    else
        ctx.tripwire = TapeEngine.TRIPWIRE_TRIPPED
        ctx.error_reason = error_reason or "Invariant guard violation"
        return TapeEngine.STATE_ROLLBACK
    end
end

---
-- Verifies the final integrity of a context before closing commit.
function TapeEngine.verify_and_commit(ctx)
    if ctx.tripwire == TapeEngine.TRIPWIRE_TRIPPED
        or ctx.status == TapeEngine.STATE_ROLLBACK then
        return TapeEngine.STATE_ROLLBACK, "Commit rejected: tripwire tripped"
    end
    ctx.tripwire = TapeEngine.TRIPWIRE_DISARMED
    return TapeEngine.STATE_PROCEED, "Context sealed in PROCEED (+1)"
end

return TapeEngine
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/interpreted/lua/tests/test_common.lua

LUA | 51 LINES | 185 WORDS | 1.7 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & TEST HARNESS TOPOLOGY
-- ( Test Framework: < OmniCode Standard Library Universal Lua Testbed > )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Universal test harness for OmniCode Standard Library Lua Process Ribbons.
--   Provides structured assertions, test suite runner helpers, and tripwires.
--   - Witness: Scripture Proverbs 24:27 KJV
--     "Prepare thy work without, and make it fit for thyself in the field;
--      and afterwards build thine house."
--   - Semantics: { AssertionInterlock :: GranularFaultIsolation }
-- ================================================================================

local TestCommon = {
    failures = 0,
    tests_run = 0
}

function TestCommon.assert(condition, msg)
    TestCommon.tests_run = TestCommon.tests_run + 1
    if not condition then
        TestCommon.failures = TestCommon.failures + 1
        error("    [FAIL] Assertion failed: " .. tostring(msg), 2)
    end
end

function TestCommon.assert_eq(actual, expected, msg)
    TestCommon.tests_run = TestCommon.tests_run + 1
    if actual ~= expected then
        TestCommon.failures = TestCommon.failures + 1
        error(string.format("    [FAIL] %s: Expected [%s], got [%s]",
                            tostring(msg), tostring(expected),
                            tostring(actual)), 2)
    end
end

function TestCommon.run_test(name, fn)
    io.write(string.format("    -> %s ... ", name))
    local ok, err = pcall(fn)
    if ok then
        print("[PASS]")
        return true
    else
        print("[FAILED]")
        print(tostring(err))
        return false
    end
end

return TestCommon
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/interpreted/lua/tests/test_tape_engine.lua

LUA | 248 LINES | 798 WORDS | 9.3 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & TEST TOPOLOGY
-- ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Tape Engine } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Validates the universal Tape Engine, coroutine fibers, rollback snapshots,
--   and Quad-State connectives in isolation BEFORE being consumed by tapes.
--   - Witness: Scripture Proverbs 24:27 KJV
--     "Prepare thy work without, and make it fit for thyself in the field;
--      and afterwards build thine house."
--   - Semantics: { TapeEngineAxioms :: CoroutineFiber :: Connectives }
-- ================================================================================

local script_dir = debug.getinfo(1, "S").source:match("@?(.*[/\\])") or "./"
package.path = script_dir .. "?.lua;" ..
               script_dir .. "../?.lua;" ..
               package.path

local TestCommon = require("test_common")
local TapeEngine = require("tape_engine")

local Suite = {}

function Suite.test_tape_creation()
    local tape = TapeEngine.create_tape({
        metadata = { who = "Kernel", what = "Test" },
        constants = { LIMIT = 10 },
        rolls = {}
    })
    TestCommon.assert(tape ~= nil, "Tape must be instantiated")
    TestCommon.assert_eq(tape.metadata.who, "Kernel", "Metadata must match")
    TestCommon.assert_eq(tape.constants.LIMIT, 10, "Constants must match")
end

function Suite.test_roll_execution_and_hooks()
    local pre_count = 0
    local post_count = 0

    local tape = TapeEngine.create_tape({
        rolls = {
            {
                name = "Step1",
                action = function(ctx)
                    ctx.val = (ctx.val or 0) + 5
                    return TapeEngine.STATE_PROCEED
                end
            },
            {
                name = "Step2",
                action = function(ctx)
                    ctx.val = ctx.val * 2
                    return TapeEngine.STATE_PROCEED
                end
            }
        }
    })

    tape:before(function() pre_count = pre_count + 1 end)
    tape:after(function() post_count = post_count + 1 end)

    local ctx = { val = 10 }
    local status = tape:execute_all(ctx)
    TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED, "Execution PROCEED")
    TestCommon.assert_eq(ctx.val, 30, "Calculated value: (10 + 5) * 2 = 30")
    TestCommon.assert_eq(pre_count, 2, "2 pre-roll hooks fired")
    TestCommon.assert_eq(post_count, 2, "2 post-roll hooks fired")
end

function Suite.test_rollback_and_snapshots()
    local rollback_fired = false

    local tape = TapeEngine.create_tape({
        rolls = {
```

```
            {
                name = "Step1",
                action = function(ctx)
                    ctx.count = 100
                    return TapeEngine.STATE_PROCEED
                end
            },
            {
                name = "FaultyStep",
                action = function(ctx)
                    ctx.count = 999
                    return TapeEngine.STATE_ROLLBACK
                end
            }
        }
    })

    tape:on_rollback(function() rollback_fired = true end)

    local ctx = { count = 0 }
    local status = tape:execute_all(ctx)
    TestCommon.assert_eq(status, TapeEngine.STATE_ROLLBACK, "Yields ROLLBACK")
    TestCommon.assert(rollback_fired, "Rollback handler must be invoked")
    TestCommon.assert_eq(ctx.count, 100, "Context restored to last snapshot")
end

function Suite.test_dual_rail_substrate()
    TestCommon.assert_eq(TapeEngine.trit_to_pair(0), 0, "0 -> pair 0b00")
    TestCommon.assert_eq(TapeEngine.trit_to_pair(1), 1, "1 -> pair 0b01")
    TestCommon.assert_eq(TapeEngine.trit_to_pair(-1), 2, "-1 -> pair 0b10")

    local t0, ok0 = TapeEngine.pair_to_trit(0)
    TestCommon.assert(ok0 and t0 == 0, "0b00 decodes to 0")

    local t1, ok1 = TapeEngine.pair_to_trit(1)
    TestCommon.assert(ok1 and t1 == 1, "0b01 decodes to +1")

    local t2, ok2 = TapeEngine.pair_to_trit(2)
    TestCommon.assert(ok2 and t2 == -1, "0b10 decodes to -1")

    local _, ok3 = TapeEngine.pair_to_trit(3)
    TestCommon.assert(not ok3, "0b11 detects collision fault")

    -- SIMD rail swap negation
    local val = 1 -- bit 0 (positive rail)
    local neg = TapeEngine.negate_dual_rail(val)
    TestCommon.assert_eq(neg, 2, "Positive rail swapped to negative rail")
    local neg_neg = TapeEngine.negate_dual_rail(neg)
    TestCommon.assert_eq(neg_neg, val, "Double rail negation is identity")

    -- Collision detection
    TestCommon.assert(not TapeEngine.has_null_collision(val), "No collision")
    TestCommon.assert(TapeEngine.has_null_collision(3), "3 has collision")
end

function Suite.test_connective_operators()
    -- BUT (!)
    TestCommon.assert_eq(
        TapeEngine.op_but(TapeEngine.STATE_PROCEED, TapeEngine.STATE_PROCEED),
        TapeEngine.STATE_PROCEED, "PROCEED ! PROCEED == PROCEED"
    )
    TestCommon.assert_eq(
        TapeEngine.op_but(TapeEngine.STATE_PROCEED, TapeEngine.STATE_ROLLBACK),
        TapeEngine.STATE_ROLLBACK, "PROCEED ! ROLLBACK == ROLLBACK"
    )

    -- AND (::)
    TestCommon.assert_eq(
        TapeEngine.op_and(TapeEngine.STATE_PROCEED, TapeEngine.STATE_PROCEED),
        TapeEngine.STATE_PROCEED, "PROCEED :: PROCEED == PROCEED"
    )
    TestCommon.assert_eq(
        TapeEngine.op_and(TapeEngine.STATE_PROCEED, TapeEngine.STATE_AWAIT),
        TapeEngine.STATE_AWAIT, "PROCEED :: AWAIT == AWAIT"
    )

    -- OR (|)
    TestCommon.assert_eq(
```

```
        TapeEngine.op_or(TapeEngine.STATE_PROCEED, TapeEngine.STATE_ROLLBACK),
        TapeEngine.STATE_PROCEED, "PROCEED | ROLLBACK == PROCEED"
    )
    TestCommon.assert_eq(
        TapeEngine.op_or(TapeEngine.STATE_ROLLBACK, TapeEngine.STATE_PROCEED),
        TapeEngine.STATE_PROCEED, "ROLLBACK | PROCEED == PROCEED"
    )

    -- NOT (--)
    TestCommon.assert_eq(
        TapeEngine.op_not(TapeEngine.STATE_PROCEED),
        TapeEngine.STATE_ROLLBACK, "-- PROCEED == ROLLBACK"
    )
    TestCommon.assert_eq(
        TapeEngine.op_not(TapeEngine.STATE_ROLLBACK),
        TapeEngine.STATE_PROCEED, "-- ROLLBACK == PROCEED"
    )

    -- NEQ (!=)
    TestCommon.assert_eq(
        TapeEngine.op_neq(TapeEngine.STATE_PROCEED, TapeEngine.STATE_ROLLBACK),
        TapeEngine.STATE_PROCEED, "PROCEED != ROLLBACK == PROCEED"
    )
    TestCommon.assert_eq(
        TapeEngine.op_neq(TapeEngine.STATE_PROCEED, TapeEngine.STATE_PROCEED),
        TapeEngine.STATE_ROLLBACK, "PROCEED != PROCEED == ROLLBACK"
    )

    -- IS_NOT (=!)
    TestCommon.assert_eq(
        TapeEngine.op_is_not(TapeEngine.STATE_AWAIT,
                             TapeEngine.STATE_ROLLBACK),
        TapeEngine.STATE_PROCEED, "AWAIT =! ROLLBACK == PROCEED"
    )

    -- LIKEN (=~)
    TestCommon.assert_eq(
        TapeEngine.op_liken(TapeEngine.STATE_PROCEED, TapeEngine.STATE_AWAIT),
        TapeEngine.STATE_PROCEED, "PROCEED =~ AWAIT == PROCEED"
    )

    -- RELATE (:)
    TestCommon.assert_eq(
        TapeEngine.op_relate(TapeEngine.STATE_PROCEED, TapeEngine.STATE_AWAIT),
        TapeEngine.STATE_AWAIT, "PROCEED : AWAIT == AWAIT"
    )
    TestCommon.assert_eq(
        TapeEngine.op_relate(TapeEngine.STATE_ROLLBACK,
                             TapeEngine.STATE_PROCEED),
        TapeEngine.STATE_ROLLBACK, "ROLLBACK : PROCEED == ROLLBACK"
    )
end

function Suite.test_literal_disambiguation()
    TestCommon.assert(TapeEngine.is_state(TapeEngine.STATE_PROCEED), "State +1")
    TestCommon.assert(TapeEngine.is_state(TapeEngine.STATE_AWAIT), "State 0")
    TestCommon.assert(TapeEngine.is_state(TapeEngine.STATE_ROLLBACK),
                      "State -1")
    TestCommon.assert(TapeEngine.is_state(TapeEngine.STATE_HALT), "State NULL")
    TestCommon.assert(not TapeEngine.is_state(42), "42 is not a state")

    -- Air quotes "0" literal resolution
    local v0, is_lit0 = TapeEngine.resolve_literal("0")
    TestCommon.assert(is_lit0 and v0 == 0, "Air quote 0 resolves to literal 0")

    -- Cast { 0 } table literal resolution
    local vt, is_litt = TapeEngine.resolve_literal({ 0 })
    TestCommon.assert(is_litt and vt == 0, "{ 0 } resolves to literal 0")

    -- Plain state 0 is not a literal
    local vs, is_lits = TapeEngine.resolve_literal(TapeEngine.STATE_AWAIT)
    TestCommon.assert((not is_lits) and vs == 0, "STATE_AWAIT is not literal")
end

print("================================================================")
print("   OMNICODE LUA UNIVERSAL TAPE ENGINE UNIT TESTBED")
print("================================================================")
TestCommon.run_test("Tape Instantiation & Topology", Suite.test_tape_creation)
```

```
TestCommon.run_test("Roll Execution & Event Hooks",
                    Suite.test_roll_execution_and_hooks)
TestCommon.run_test("Transactional Rollback & Snapshots",
                    Suite.test_rollback_and_snapshots)
TestCommon.run_test("Dual-Rail Polar Substrate & SIMD",
                    Suite.test_dual_rail_substrate)
TestCommon.run_test("Quad-State Connective Operators",
                    Suite.test_connective_operators)
TestCommon.run_test("State vs Literal Disambiguation",
                    Suite.test_literal_disambiguation)

if TestCommon.failures > 0 then
    io.stderr:write(string.format("[!] %d test(s) failed\n",
                                  TestCommon.failures))
    os.exit(1)
end
print("-----------------------------------------------------------------")
print("[√] All Universal Tape Engine tests verified and passed!")
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/compiled/c/Makefile

TEXT | 17 LINES | 33 WORDS | 0.3 KB

```
CC ?= gcc
CFLAGS ?= -Wall -Wextra -std=c11 -Iinclude -Itests

all: test

test: test_universal_bin
    ./test_universal_bin

test_universal_bin: tests/test_universal.c include/omni_base.h \
                    tests/test_common.h
    $(CC) $(CFLAGS) $< -o $@

clean:
    rm -f test_universal_bin

.PHONY: all test clean
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/compiled/c/compile_flags.txt

TXT | 6 LINES | 5 WORDS | 0.0 KB

```
-Iinclude
-Itests
-Wall
-Wextra
-std=c11
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/compiled/c/test_universal_bin

DOC/BINARY | 1 LINES | 5 WORDS | 25.0 KB

[Binary Artifact: test_universal_bin (25.0 KB)]

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/compiled/c/tests/test_common.h

H | 59 LINES | 213 WORDS | 1.7 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & TEST HARNESS TOPOLOGY
 * ( Test Framework: < OmniCode Standard Library Universal Testbed > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Universal test harness for the OmniCode Standard Library C Transpile.
 *   Provides structured assertions, test suite runner macros, and tripwires.
 *   - Witness: Scripture Proverbs 24:27 KJV
 *     "Prepare thy work without, and make it fit for thyself in the field;
 *      and afterwards build thine house."
 *   - Semantics: { AssertionInterlock :: GranularFaultIsolation }
 * ============================================================================ */

#ifndef OMNI_TEST_COMMON_H
#define OMNI_TEST_COMMON_H

#include "omni_base.h"
#include <stdio.h>
#include <stdbool.h>

#ifdef __cplusplus
extern "C" {
#endif

/** Global test assertion failure counter */
extern int g_omni_test_failures;

/**
 * @brief Evaluates assertion expression; logs failure and returns 1 on error.
 */
#define OMNI_TEST_ASSERT(expr, msg) \
    do { \
        if (!(expr)) { \
            printf("    [FAIL] %s:%d: %s (Expression: %s)\n", \
                   __FILE__, __LINE__, (msg), #expr); \
            g_omni_test_failures++; \
            return 1; \
        } \
    } while (0)

/**
 * @brief Runs an individual test function and reports PASS / FAIL.
 */
#define OMNI_TEST_RUN(fn, test_name) \
    do { \
        printf("    -> %s ... ", (test_name)); \
        if ((fn)() != 0) { \
            printf("[FAILED]\n"); \
            return 1; \
        } \
        printf("[PASS]\n"); \
    } while (0)

#ifdef __cplusplus
}
#endif

#endif /* OMNI_TEST_COMMON_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/compiled/c/tests/test_universal.c

C | 213 LINES | 781 WORDS | 9.8 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & TEST TOPOLOGY
 * ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Universal Base } ] )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Validates the universal base axioms, balanced ternary ALU, tripwires, and
 *   Quad-State connectives in isolation BEFORE being consumed downstream.
 *   - Witness: Scripture Proverbs 24:27 KJV
 *     "Prepare thy work without, and make it fit for thyself in the field;
 *      and afterwards build thine house."
 *   - Semantics: { BaseAxioms :: BalancedTernaryALU :: ConnectiveGuards }
 * ============================================================================ */

#include "test_common.h"

int g_omni_test_failures = 0;

static int test_base_preconditions(void) {
    omni_quad_state_t s = omni_base_verify_preconditions();
    OMNI_TEST_ASSERT(s == OMNI_STATE_PROCEED,
                     "Preconditions must PROCEED on 64-bit platform");
    return 0;
}

static int test_balanced_ternary_arithmetic(void) {
    /* Negation */
    OMNI_TEST_ASSERT(omni_trit_negate(OMNI_TRIT_POS) == OMNI_TRIT_NEG,
                     "Negate(+1) == -1");
    OMNI_TEST_ASSERT(omni_trit_negate(OMNI_TRIT_NEG) == OMNI_TRIT_POS,
                     "Negate(-1) == +1");
    OMNI_TEST_ASSERT(omni_trit_negate(OMNI_TRIT_ZERO) == OMNI_TRIT_ZERO,
                     "Negate(0) == 0");

    /* Multiplication */
    OMNI_TEST_ASSERT(omni_trit_multiply(OMNI_TRIT_POS, OMNI_TRIT_NEG) ==
                     OMNI_TRIT_NEG, "(+1) * (-1) == -1");
    OMNI_TEST_ASSERT(omni_trit_multiply(OMNI_TRIT_NEG, OMNI_TRIT_NEG) ==
                     OMNI_TRIT_POS, "(-1) * (-1) == +1");

    /* Addition with Carry */
    omni_trit_t cout = 0;
    omni_trit_t sum = omni_trit_add(OMNI_TRIT_POS, OMNI_TRIT_POS, 0, &cout);
    OMNI_TEST_ASSERT(sum == OMNI_TRIT_NEG && cout == OMNI_TRIT_POS,
                     "(+1) + (+1) == -1 with carry +1 (Radix-3: 1*3 - 1 = 2)");

    sum = omni_trit_add(OMNI_TRIT_POS, OMNI_TRIT_NEG, 0, &cout);
    OMNI_TEST_ASSERT(sum == OMNI_TRIT_ZERO && cout == OMNI_TRIT_ZERO,
                     "(+1) + (-1) == 0 with carry 0 (Equilibrium)");

    return 0;
}

static int test_quad_state_connectives(void) {
    /* Connective: BUT (!) - Adversative Caveat */
    omni_quad_state_t act = OMNI_STATE_PROCEED;
    omni_quad_state_t cav_ok = OMNI_STATE_PROCEED;
    omni_quad_state_t cav_fail = OMNI_STATE_ROLLBACK;

    OMNI_TEST_ASSERT(omni_quad_state_but(act, cav_ok) == OMNI_STATE_PROCEED,
                     "PROCEED ! PROCEED must yield PROCEED");
    OMNI_TEST_ASSERT(omni_quad_state_but(act, cav_fail) == OMNI_STATE_ROLLBACK,
                     "PROCEED ! ROLLBACK must yield ROLLBACK");

    /* Connective: AND (::) - Conjunction */
    OMNI_TEST_ASSERT(omni_quad_state_and(OMNI_STATE_PROCEED, OMNI_STATE_PROCEED)
                     == OMNI_STATE_PROCEED, "PROCEED :: PROCEED == PROCEED");
    OMNI_TEST_ASSERT(omni_quad_state_and(OMNI_STATE_PROCEED, OMNI_STATE_AWAIT)
                     == OMNI_STATE_AWAIT, "PROCEED :: AWAIT == AWAIT");
    OMNI_TEST_ASSERT(
        omni_quad_state_and(OMNI_STATE_PROCEED, OMNI_STATE_ROLLBACK)
                     == OMNI_STATE_ROLLBACK, "PROCEED :: ROLLBACK == ROLLBACK");

    /* Connective: OR (|) - Disjunctive Alternative Valve */
```

```
    OMNI_TEST_ASSERT(omni_quad_state_or(OMNI_STATE_PROCEED, OMNI_STATE_ROLLBACK)
                     == OMNI_STATE_PROCEED, "PROCEED | ROLLBACK == PROCEED");
    OMNI_TEST_ASSERT(omni_quad_state_or(OMNI_STATE_ROLLBACK, OMNI_STATE_PROCEED)
                     == OMNI_STATE_PROCEED, "ROLLBACK | PROCEED == PROCEED");
    OMNI_TEST_ASSERT(
        omni_quad_state_or(OMNI_STATE_ROLLBACK, OMNI_STATE_ROLLBACK)
                     == OMNI_STATE_ROLLBACK, "ROLLBACK | ROLLBACK == ROLLBACK");

    /* Connective: NOT (--) - Truth Inversion */
    OMNI_TEST_ASSERT(omni_quad_state_not(OMNI_STATE_PROCEED) ==
                     OMNI_STATE_ROLLBACK, "-- PROCEED == ROLLBACK");
    OMNI_TEST_ASSERT(omni_quad_state_not(OMNI_STATE_ROLLBACK) ==
                     OMNI_STATE_PROCEED, "-- ROLLBACK == PROCEED");
    OMNI_TEST_ASSERT(omni_quad_state_not(OMNI_STATE_AWAIT) ==
                     OMNI_STATE_AWAIT, "-- AWAIT == AWAIT");

    /* Connective: NEQ (!=) - Inequality Gate */
    OMNI_TEST_ASSERT(omni_quad_state_neq(OMNI_STATE_PROCEED,
                                         OMNI_STATE_ROLLBACK) ==
                     OMNI_STATE_PROCEED, "PROCEED != ROLLBACK == PROCEED");
    OMNI_TEST_ASSERT(omni_quad_state_neq(OMNI_STATE_PROCEED,
                                         OMNI_STATE_PROCEED) ==
                     OMNI_STATE_ROLLBACK, "PROCEED != PROCEED == ROLLBACK");

    /* Connective: IS_NOT (=!) - Predicate Filter */
    OMNI_TEST_ASSERT(omni_quad_state_is_not(OMNI_STATE_AWAIT,
                                           OMNI_STATE_ROLLBACK) ==
                     OMNI_STATE_PROCEED, "AWAIT =! ROLLBACK == PROCEED");
    OMNI_TEST_ASSERT(omni_quad_state_is_not(OMNI_STATE_ROLLBACK,
                                           OMNI_STATE_ROLLBACK) ==
                     OMNI_STATE_ROLLBACK, "ROLLBACK =! ROLLBACK == ROLLBACK");

    /* Connective: LIKEN (=~) - Compatibility Match */
    OMNI_TEST_ASSERT(omni_quad_state_liken(OMNI_STATE_PROCEED,
                                         OMNI_STATE_AWAIT) ==
                     OMNI_STATE_PROCEED, "PROCEED =~ AWAIT == PROCEED");
    OMNI_TEST_ASSERT(omni_quad_state_liken(OMNI_STATE_PROCEED,
                                         OMNI_STATE_ROLLBACK) ==
                     OMNI_STATE_ROLLBACK, "PROCEED =~ ROLLBACK == ROLLBACK");

    /* Connective: RELATE (:) - Perspective Projection */
    OMNI_TEST_ASSERT(omni_quad_state_relate(OMNI_STATE_PROCEED,
                                           OMNI_STATE_AWAIT) ==
                     OMNI_STATE_AWAIT, "PROCEED : AWAIT == AWAIT");
    OMNI_TEST_ASSERT(omni_quad_state_relate(OMNI_STATE_ROLLBACK,
                                           OMNI_STATE_PROCEED) ==
                     OMNI_STATE_ROLLBACK, "ROLLBACK : PROCEED == ROLLBACK");

    /* State Validity */
    OMNI_TEST_ASSERT(omni_state_is_valid(OMNI_STATE_PROCEED), "PROCEED valid");
    OMNI_TEST_ASSERT(omni_state_is_valid(OMNI_STATE_AWAIT), "AWAIT valid");
    OMNI_TEST_ASSERT(omni_state_is_valid(OMNI_STATE_ROLLBACK),
                     "ROLLBACK valid");
    OMNI_TEST_ASSERT(omni_state_is_valid(OMNI_STATE_HALT), "HALT valid");
    OMNI_TEST_ASSERT(!omni_state_is_valid((omni_quad_state_t)42), "42 invalid");

    return 0;
}

static int test_dual_rail_substrate(void) {
    /* 1. 2-Bit Pair Encoding & Decoding */
    OMNI_TEST_ASSERT(omni_trit_to_pair(OMNI_TRIT_ZERO) == OMNI_TRIT_PAIR_ZERO,
                     "0 trit -> pair 0b00");
    OMNI_TEST_ASSERT(omni_trit_to_pair(OMNI_TRIT_POS) == OMNI_TRIT_PAIR_POS,
                     "+1 trit -> pair 0b01");
    OMNI_TEST_ASSERT(omni_trit_to_pair(OMNI_TRIT_NEG) == OMNI_TRIT_PAIR_NEG,
                     "-1 trit -> pair 0b10");

    omni_trit_t t = 0;
    bool ok = omni_pair_to_trit(OMNI_TRIT_PAIR_ZERO, &t);
    OMNI_TEST_ASSERT(ok && t == OMNI_TRIT_ZERO, "pair 0b00 -> 0 trit");

    ok = omni_pair_to_trit(OMNI_TRIT_PAIR_POS, &t);
    OMNI_TEST_ASSERT(ok && t == OMNI_TRIT_POS, "pair 0b01 -> +1 trit");

    ok = omni_pair_to_trit(OMNI_TRIT_PAIR_NEG, &t);
    OMNI_TEST_ASSERT(ok && t == OMNI_TRIT_NEG, "pair 0b10 -> -1 trit");

```

```
    ok = omni_pair_to_trit(OMNI_TRIT_PAIR_NULL, &t);
    OMNI_TEST_ASSERT(!ok, "pair 0b11 -> NULL collision fault");

    /* 2. Dual-Rail SIMD Negation */
    uint64_t val = 0x0000000000000001ULL; /* 1 trit POS in bit 0 */
    uint64_t neg = omni_dual_rail_negate(val);
    OMNI_TEST_ASSERT(neg == 0x0000000000000002ULL, "Negation swaps P->N rail");
    uint64_t neg_neg = omni_dual_rail_negate(neg);
    OMNI_TEST_ASSERT(neg_neg == val, "Double negation restores identity");

    /* 3. Collision Trap */
    OMNI_TEST_ASSERT(!omni_dual_rail_has_collision(val),
                     "val has no collision");
    uint64_t trap = 0x0000000000000003ULL; /* bits 0 & 1 set -> collision */
    OMNI_TEST_ASSERT(omni_dual_rail_has_collision(trap),
                     "trap reports collision");

    return 0;
}

static int test_tripwire_lifecycle(void) {
    omni_tripwire_state_t tw = OMNI_TRIPWIRE_DISARMED;
    omni_tripwire_arm(&tw);
    OMNI_TEST_ASSERT(tw == OMNI_TRIPWIRE_ARMED, "Tripwire must be ARMED");

    omni_tripwire_disarm(&tw);
    OMNI_TEST_ASSERT(tw == OMNI_TRIPWIRE_DISARMED, "Tripwire must be DISARMED");

    omni_tripwire_arm(&tw);
    omni_tripwire_trip(&tw);
    OMNI_TEST_ASSERT(tw == OMNI_TRIPWIRE_TRIPPED, "Tripwire must be TRIPPED");

    omni_quad_state_t integrity = omni_base_verify_integrity(tw,
        OMNI_STATE_PROCEED);
    OMNI_TEST_ASSERT(integrity == OMNI_STATE_ROLLBACK,
                     "Integrity must fail if tripwire was TRIPPED");

    return 0;
}

int main(void) {
    printf("================================================================\n");
    printf("   OMNICODE C UNIVERSAL BASE UNIT TESTBED\n");
    printf("================================================================\n");
    OMNI_TEST_RUN(test_base_preconditions,
                  "Hardware Preconditions Verification");
    OMNI_TEST_RUN(test_balanced_ternary_arithmetic,
                  "Balanced Ternary ALU Math");
    OMNI_TEST_RUN(test_dual_rail_substrate, "Dual-Rail Polar Substrate & SIMD");
    OMNI_TEST_RUN(test_quad_state_connectives, "Quad-State Connectives");
    OMNI_TEST_RUN(test_tripwire_lifecycle, "Tripwire Lifecycle & Integrity");

    if (g_omni_test_failures > 0) {
        fprintf(stderr, "[!] Universal base test suite failed with %d errors\n",
                g_omni_test_failures);
        return 1;
    }
    printf("----------------------------------------------------------------\n");
    printf("[√] All Universal Base Axioms verified and sealed!\n");
    return 0;
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/compiled/c/include/omni.h

H | 45 LINES | 176 WORDS | 1.4 KB

```c
/* ============================================================================
 * OMNICODE FREESTANDING STANDARD LIBRARY (C MASTER UMBRELLA HEADER)
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Master header exposing all primitive invariant types and operational
 *   primitives for OmniCode bare-metal and hosted runtimes.
 *   - Category: Standard Library Track 2 C Transpile Variant
 *   - Invariant Types Included:
 *       < Base >      : Substrates, Balanced Trits, Quad-State & Connectives
 *       < Character > : Atomic Glyph / Letter Rune (1 Cell / 5 Trits)
 *       < Number >    : Scalar Quantity / Measure (1 Word / 32 Trits)
 *       < Container > : Generic Bounded Space Envelope / Slice
 *       < Site >      : Silicon Coordinate / Address (RAM @ vs MMIO ::@)
 *       < State >     : Quad-State Causal Truth & Loop Stepper
 * ============================================================================ */

#ifndef OMNI_H
#define OMNI_H

#include "omni_base.h"

/* Modular Type Substrates (Character & Number) */
#if __has_include("character.h")
#include "character.h"
#endif

#if __has_include("number.h")
#include "number.h"
#endif

/* Transitional Primitive Types */
#if __has_include("container.h")
#include "container.h"
#endif

#if __has_include("site.h")
#include "site.h"
#endif

#if __has_include("state.h")
#include "state.h"
#endif

#endif /* OMNI_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/universal/transpiled/compiled/c/include/omni_base.h

H | 403 LINES | 1567 WORDS | 14.5 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT AXIOMS BLOCK
 * ( Invariant Root: < Lexicon { Types of Ideas } > & [ Noun { Base Axioms } ] )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Foundational bare-metal substrates, balanced ternary types, and Quad-State
 *   truth logic governing the OmniCode Standard Library Track 2 C Transpile.
 *   - Environment: Freestanding C99/C11 (Zero libc/OS dependency)
 *   - Word Size: 64-Bit Silicon Scalar / 32 Balanced Trits
 *   - Cell Size: 8-Bit Byte / 5 Balanced Trits
 *   - System: OmniCode { Lexicon }
 *   - Witness: Scripture Proverbs 11:1 KJV
 *     "A false balance is abomination to the LORD: but a just weight is his
 *      delight."
 *   - Semantics: { BalancedTernary :: QuadStateTruth :: TripwireGuards }
 * ============================================================================ */

#ifndef OMNI_BASE_H
#define OMNI_BASE_H

#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>

#ifdef __cplusplus
extern "C" {
#endif

/* ----------------------------------------------------------------------------
 * ARCHITECTURAL CONSTANTS & SUBSTRATE CAPACITIES
 * ---------------------------------------------------------------------------- */

/** Number of balanced trits packed into an 8-bit memory Cell (3^5 = 243). */
#define OMNI_TRITS_PER_CELL         5

/** Number of balanced trits packed into a 64-bit machine Word. */
#define OMNI_TRITS_PER_WORD         32

/** Footprint of the standard bounded Container vessel exemplar (in bytes). */
#define OMNI_VESSEL_DEFAULT_CELLS   64

/* ----------------------------------------------------------------------------
 * 1. BALANCED TERNARY SUBSTRATE: omni_trit_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief Discrete balanced ternary digit (-1, 0, +1).
 *
 * Balanced ternary is inherently symmetrical around zero:
 *   - Negative (-1) : Backward causation, negation, deficit, or restoration.
 *   - Zero     ( 0) : Equilibrium, standby, neutral, or unperturbed balance.
 *   - Positive (+1) : Forward causation, affirmative assertion, or surplus.
 */
typedef int8_t omni_trit_t;

#define OMNI_TRIT_NEG   ((omni_trit_t)-1)
#define OMNI_TRIT_ZERO  ((omni_trit_t) 0)
#define OMNI_TRIT_POS   ((omni_trit_t) 1)

/* ----------------------------------------------------------------------------
 * BALANCED TERNARY DUAL-RAIL SUBSTRATE (POLAR BIT-PLANES)
 * ---------------------------------------------------------------------------- */

/** Dual-Rail Positive Bit-Plane Mask (all even bits: 0, 2, 4, ... 62). */
#define OMNI_BITPLANE_POS_MASK      (0x5555555555555555ULL)

/** Dual-Rail Negative Bit-Plane Mask (all odd bits: 1, 3, 5, ... 63). */
#define OMNI_BITPLANE_NEG_MASK      (0xAAAAAAAAAAAAAAAAULL)

/** Dual-Rail 2-Bit Trit Constants on Binary Silicon. */
#define OMNI_TRIT_PAIR_ZERO         ((uint8_t)0x00) /* 00: Equilibrium (0) */
#define OMNI_TRIT_PAIR_POS          ((uint8_t)0x01) /* 01: Positive (+1)   */
#define OMNI_TRIT_PAIR_NEG          ((uint8_t)0x02) /* 10: Negative (-1)   */
```

```c
#define OMNI_TRIT_PAIR_NULL         ((uint8_t)0x03) /* 11: Collision (NULL)*/

/* ---------------------------------------------------------------------------
 * 2. QUAD-STATE CAUSAL TRUTH: omni_quad_state_t
 * ------------------------------------------------------------------------- */

/**
 * @brief The fourfold causal truth states of computation in OmniCode.
 *
 * Unifies boolean logic, error handling, scheduling, and clock detents:
 *   - OMNI_STATE_PROCEED  (+1) : Forward Causation (+t). Equilibrium.
 *   - OMNI_STATE_AWAIT    ( 0) : Equilibrium Standby (t=t). Pending async.
 *   - OMNI_STATE_ROLLBACK (-1) : Retroactive Reversal (-t). Guard tripped.
 *   - OMNI_STATE_HALT     (-2) : Terminal Detent (NULL). Clock stopped.
 */
typedef enum {
    OMNI_STATE_ROLLBACK = -1,   /**< -1 : Retroactive Reversal / Error */
    OMNI_STATE_AWAIT    =  0,   /**<  0 : Equilibrium Standby / Hold */
    OMNI_STATE_PROCEED  =  1,   /**< +1 : Forward Causation / Advance */
    OMNI_STATE_HALT     = -2    /**< NULL: Terminal Detent / Clock Stop */
} omni_quad_state_t;

/**
 * @brief Tertiary execution status subset for non-terminal checkpoints.
 */
typedef enum {
    OMNI_STATUS_ROLLBACK = -1,
    OMNI_STATUS_AWAIT    =  0,
    OMNI_STATUS_PROCEED  =  1
} omni_status_t;

/* ---------------------------------------------------------------------------
 * 3. HARDWARE COGNITIVE SITES (EXEMPLARS)
 * ------------------------------------------------------------------------- */

/** Unmapped null pointer origin. */
#define OMNI_SITE_NULL          ((uintptr_t)0x00000000)

/** Real-mode Master Boot Record physical origin. */
#define OMNI_SITE_MBR_BOOT      ((uintptr_t)0x00007C00)

/** Hardware UART serial port base (COM1) for kernel telemetry. */
#define OMNI_SITE_COM1          ((uintptr_t)0x000003F8)

/** Physical memory-mapped VGA text-mode display buffer (80x25 characters). */
#define OMNI_SITE_VGA_BUFFER    ((uintptr_t)0x000B8000)

/* ---------------------------------------------------------------------------
 * 4. TRIPWIRE & INTEGRITY DETENTS: omni_tripwire_state_t
 * ------------------------------------------------------------------------- */

/**
 * @brief Safety interlock state preventing silent memory corruption.
 */
typedef enum {
    OMNI_TRIPWIRE_DISARMED = 0, /**< Interlock inactive; safe quiescent */
    OMNI_TRIPWIRE_ARMED    = 1, /**< Active guard monitoring bounds */
    OMNI_TRIPWIRE_TRIPPED  = 2  /**< Violation intercepted; blocked */
} omni_tripwire_state_t;

/* ===========================================================================
 * BLOCK 2: OPENING BLOCK
 * ( Subsystem Preconditions, Verification & Initializers )
 * ========================================================================= */

/**
 * @brief Verifies that host hardware meets OmniCode's bare-metal preconditions.
 */
static inline omni_quad_state_t omni_base_verify_preconditions(void) {
    if (sizeof(uintptr_t) != 8) return OMNI_STATE_ROLLBACK;
    if (sizeof(uint8_t)   != 1) return OMNI_STATE_ROLLBACK;

    /* Prove Proverbs 11:1 Law { Equilibrium }: (+1) plus (-1) <=> 0 */
    omni_trit_t pos = OMNI_TRIT_POS;
    omni_trit_t neg = OMNI_TRIT_NEG;
    if ((pos + neg) != OMNI_TRIT_ZERO) return OMNI_STATE_ROLLBACK;

    return OMNI_STATE_PROCEED;
```

```
}

static inline void omni_tripwire_arm(omni_tripwire_state_t *tw) {
    if (tw) *tw = OMNI_TRIPWIRE_ARMED;
}

static inline void omni_tripwire_disarm(omni_tripwire_state_t *tw) {
    if (tw) *tw = OMNI_TRIPWIRE_DISARMED;
}

static inline void omni_tripwire_trip(omni_tripwire_state_t *tw) {
    if (tw) *tw = OMNI_TRIPWIRE_TRIPPED;
}

/* ============================================================================
 * BLOCK 3: BODY BLOCK
 * ( Balanced Ternary Algebra, Quad-State Logic & Connective Inlines )
 * ============================================================================ */

static inline omni_trit_t omni_trit_negate(omni_trit_t t) {
    return (omni_trit_t)(-t);
}

static inline omni_trit_t omni_trit_multiply(omni_trit_t a, omni_trit_t b) {
    return (omni_trit_t)(a * b);
}

static inline omni_trit_t omni_trit_add(omni_trit_t a, omni_trit_t b,
                                        omni_trit_t cin, omni_trit_t *cout) {
    int32_t sum = (int32_t)a + (int32_t)b + (int32_t)cin;
    omni_trit_t carry = 0;

    if (sum > 1) {
        sum -= 3;
        carry = 1;
    } else if (sum < -1) {
        sum += 3;
        carry = -1;
    }

    if (cout) *cout = carry;
    return (omni_trit_t)sum;
}

/* ----------------------------------------------------------------------------
 * BALANCED TERNARY DUAL-RAIL HARDWARE CONVERSIONS
 * ---------------------------------------------------------------------------- */

static inline uint8_t omni_trit_to_pair(omni_trit_t trit) {
    if (trit == OMNI_TRIT_ZERO) return OMNI_TRIT_PAIR_ZERO;
    if (trit == OMNI_TRIT_POS)  return OMNI_TRIT_PAIR_POS;
    if (trit == OMNI_TRIT_NEG)  return OMNI_TRIT_PAIR_NEG;
    return OMNI_TRIT_PAIR_NULL;
}

static inline bool omni_pair_to_trit(uint8_t pair, omni_trit_t *out_trit) {
    switch (pair & 0x03) {
        case OMNI_TRIT_PAIR_ZERO:
            if (out_trit) *out_trit = OMNI_TRIT_ZERO;
            return true;
        case OMNI_TRIT_PAIR_POS:
            if (out_trit) *out_trit = OMNI_TRIT_POS;
            return true;
        case OMNI_TRIT_PAIR_NEG:
            if (out_trit) *out_trit = OMNI_TRIT_NEG;
            return true;
        default: /* OMNI_TRIT_PAIR_NULL */
            if (out_trit) *out_trit = OMNI_TRIT_ZERO;
            return false;
    }
}

static inline uint64_t omni_dual_rail_negate(uint64_t packed) {
    uint64_t pos = packed & OMNI_BITPLANE_POS_MASK;
    uint64_t neg = packed & OMNI_BITPLANE_NEG_MASK;
    return ((pos << 1) & OMNI_BITPLANE_NEG_MASK) |
           ((neg >> 1) & OMNI_BITPLANE_POS_MASK);
}
```

```
230
231 static inline bool omni_dual_rail_has_collision(uint64_t packed) {
232     uint64_t pos = packed & OMNI_BITPLANE_POS_MASK;
233     uint64_t neg = (packed >> 1) & OMNI_BITPLANE_POS_MASK;
234     return (pos & neg) != 0;
235 }
236
237 /* ---------------------------------------------------------------------------
238  * 5. BALANCED TERNARY CONNECTIVE OPERATORS (!, ::, |, --)
239  * ------------------------------------------------------------------------- */
240
241 /**
242  * @brief Adversative caveat gate: A ! B (A executes, BUT caveat B must hold).
243  */
244 static inline omni_quad_state_t omni_quad_state_but(omni_quad_state_t action,
245                                                     omni_quad_state_t caveat) {
246     if (caveat == OMNI_STATE_ROLLBACK) return OMNI_STATE_ROLLBACK;
247     if (caveat == OMNI_STATE_HALT)     return OMNI_STATE_HALT;
248     return action;
249 }
250
251 /**
252  * @brief Cumulative conjunction: A :: B (A AND B simultaneous).
253  */
254 static inline omni_quad_state_t omni_quad_state_and(omni_quad_state_t a,
255                                                     omni_quad_state_t b) {
256     if (a == OMNI_STATE_HALT || b == OMNI_STATE_HALT) return OMNI_STATE_HALT;
257     if (a == OMNI_STATE_ROLLBACK || b == OMNI_STATE_ROLLBACK) {
258         return OMNI_STATE_ROLLBACK;
259     }
260     if (a == OMNI_STATE_AWAIT || b == OMNI_STATE_AWAIT) return OMNI_STATE_AWAIT;
261     return OMNI_STATE_PROCEED;
262 }
263
264 /**
265  * @brief Disjunctive alternative valve: A | B (A OR B).
266  */
267 static inline omni_quad_state_t omni_quad_state_or(omni_quad_state_t a,
268                                                    omni_quad_state_t b) {
269     if (a == OMNI_STATE_PROCEED || b == OMNI_STATE_PROCEED) {
270         return OMNI_STATE_PROCEED;
271     }
272     if (a == OMNI_STATE_AWAIT || b == OMNI_STATE_AWAIT) return OMNI_STATE_AWAIT;
273     if (a == OMNI_STATE_ROLLBACK && b == OMNI_STATE_ROLLBACK) {
274         return OMNI_STATE_ROLLBACK;
275     }
276     return OMNI_STATE_HALT;
277 }
278
279 /**
280  * @brief Balanced ternary truth inversion: -- A (NOT A).
281  */
282 static inline omni_quad_state_t omni_quad_state_not(omni_quad_state_t a) {
283     if (a == OMNI_STATE_PROCEED)  return OMNI_STATE_ROLLBACK;
284     if (a == OMNI_STATE_ROLLBACK) return OMNI_STATE_PROCEED;
285     return a;
286 }
287
288 /**
289  * @brief Inequality verification: A != B (A not to be B).
290  */
291 static inline omni_quad_state_t omni_quad_state_neq(omni_quad_state_t a,
292                                                     omni_quad_state_t b) {
293     if (a == OMNI_STATE_HALT || b == OMNI_STATE_HALT) return OMNI_STATE_HALT;
294     return (a != b) ? OMNI_STATE_PROCEED : OMNI_STATE_ROLLBACK;
295 }
296
297 /**
298  * @brief Predicate is-not check: A =! B (State A is not target B).
299  */
300 static inline omni_quad_state_t omni_quad_state_is_not(
301     omni_quad_state_t a,
302     omni_quad_state_t target)
303 {
304     return (a != target) ? OMNI_STATE_PROCEED : OMNI_STATE_ROLLBACK;
305 }
306
307 /**
```

```c
 * @brief Liken to / compatibility match: A =~ B.
 */
static inline omni_quad_state_t omni_quad_state_liken(omni_quad_state_t a,
                                                      omni_quad_state_t b) {
    if (a == OMNI_STATE_HALT || b == OMNI_STATE_HALT) return OMNI_STATE_HALT;
    if (a == b) return OMNI_STATE_PROCEED;
    if ((a == OMNI_STATE_PROCEED || a == OMNI_STATE_AWAIT) &&
        (b == OMNI_STATE_PROCEED || b == OMNI_STATE_AWAIT)) {
        return OMNI_STATE_PROCEED;
    }
    return OMNI_STATE_ROLLBACK;
}

/**
 * @brief Relation projection valve: Context : Aspect.
 */
static inline omni_quad_state_t omni_quad_state_relate(omni_quad_state_t ctx,
                                                       omni_quad_state_t asp) {
    if (ctx == OMNI_STATE_HALT)     return OMNI_STATE_HALT;
    if (ctx == OMNI_STATE_ROLLBACK) return OMNI_STATE_ROLLBACK;
    return asp;
}

/**
 * @brief Verifies whether a given state is one of the defined causal states.
 */
static inline bool omni_state_is_valid(omni_quad_state_t s) {
    return (s == OMNI_STATE_PROCEED ||
            s == OMNI_STATE_AWAIT   ||
            s == OMNI_STATE_ROLLBACK||
            s == OMNI_STATE_HALT);
}

static inline omni_quad_state_t omni_guard_assert(bool condition,
                                                  omni_tripwire_state_t *tw,
                                                  omni_quad_state_t *status) {
    if (condition) {
        if (status) *status = OMNI_STATE_PROCEED;
        return OMNI_STATE_PROCEED;
    } else {
        if (tw) *tw = OMNI_TRIPWIRE_TRIPPED;
        if (status) *status = OMNI_STATE_ROLLBACK;
        return OMNI_STATE_ROLLBACK;
    }
}

static inline bool omni_is_proceed(omni_quad_state_t s) {
    return (s == OMNI_STATE_PROCEED);
}
static inline bool omni_is_await(omni_quad_state_t s) {
    return (s == OMNI_STATE_AWAIT);
}
static inline bool omni_is_rollback(omni_quad_state_t s) {
    return (s == OMNI_STATE_ROLLBACK);
}
static inline bool omni_is_halt(omni_quad_state_t s) {
    return (s == OMNI_STATE_HALT);
}

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK
 * ( Integrity Seals, Detents & Commit Status Functions )
 * ============================================================================ */

static inline omni_quad_state_t omni_base_verify_integrity(
    omni_tripwire_state_t tw,
    omni_quad_state_t status)
{
    if (tw == OMNI_TRIPWIRE_TRIPPED || status == OMNI_STATE_ROLLBACK) {
        return OMNI_STATE_ROLLBACK;
    }
    return OMNI_STATE_PROCEED;
}

static inline omni_quad_state_t omni_commit_proceed(omni_tripwire_state_t *tw) {
    omni_tripwire_disarm(tw);
    return OMNI_STATE_PROCEED;
}
```

```
386
387 static inline omni_quad_state_t omni_commit_rollback(
388     omni_tripwire_state_t *tw) {
389     omni_tripwire_trip(tw);
390     return OMNI_STATE_ROLLBACK;
391 }
392
393 static inline omni_quad_state_t omni_commit_halt(omni_tripwire_state_t *tw) {
394     omni_tripwire_disarm(tw);
395     return OMNI_STATE_HALT;
396 }
397
398 #ifdef __cplusplus
399 }
400 #endif
401
402 #endif /* OMNI_BASE_H */
```